Introduction
Architecture des ordinateurs

R1.03

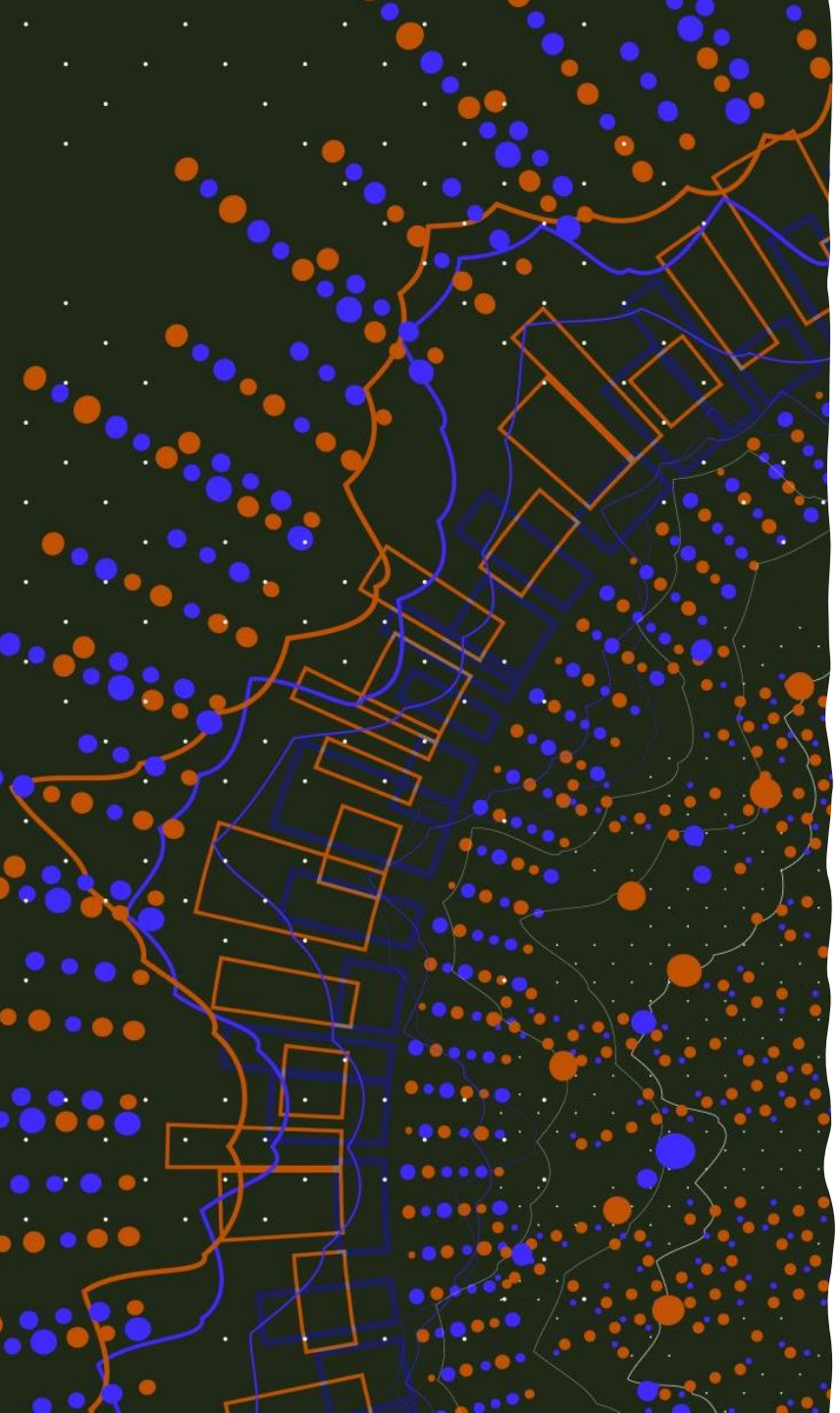
7 C, 7 TD, 3 TP + Saé

Philippe Roose, Nathalie Vallès



Objectifs – Comprendre…

- De quoi est composé un ordinateur :
  - microprocesseur, des mémoires, disque dur …?
- Quels sont les modèles sous-jacents au fonctionnement d'une machine ?
- Comment s'exécutent des programmes sur un ordinateur ?
- Quel est le lien entre le logiciel et le matériel ?
- Comment se fait l'interface avec l'extérieur ?
  - fonctionnement des divers périphériques



Plan

- Qu’est-ce qu’un ordinateur – Architecture Von Neumann
- Types mémoires
- Fonctionnement d’un processeur
- Représentation de l’information

Qu’est ce qu’un ordinateur ?

Un ordinateur est une machine de traitement de l’information.

Capable de réaliser les opérations suivantes :

- acquisition de l’information
- stockage de l’information
- transformation de l’information par son traitement
- restitution de l’information

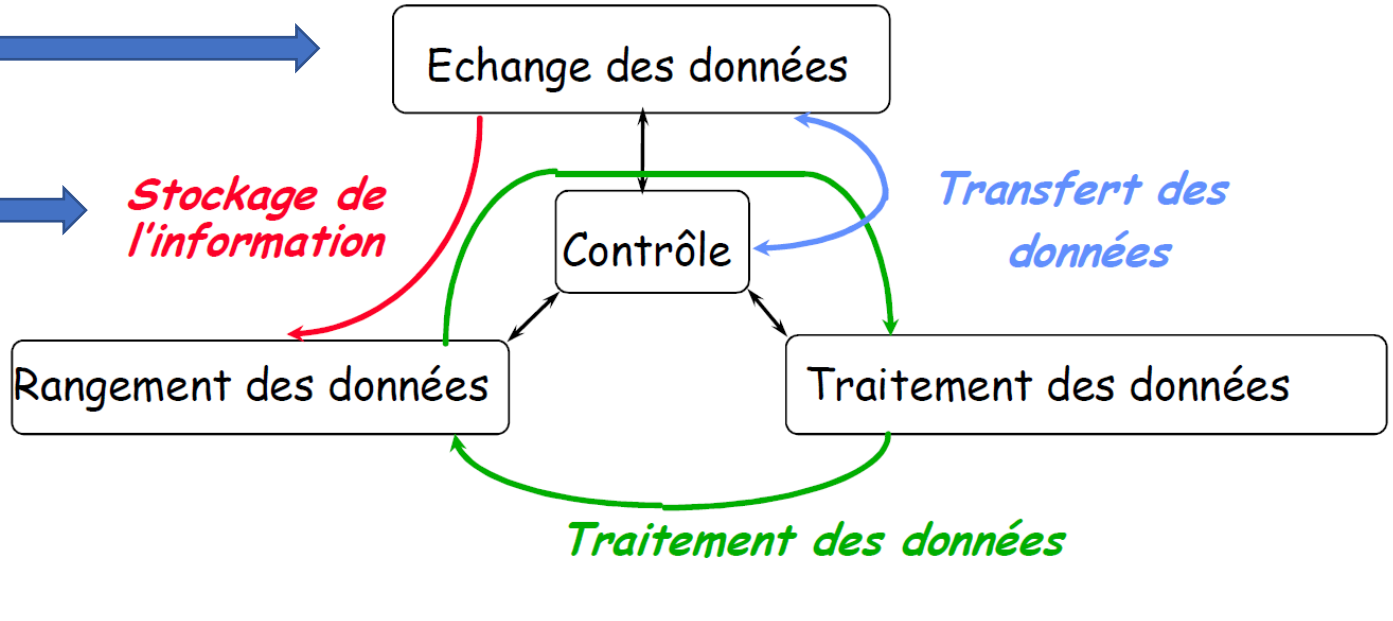
Différents types d’informations :

- données sous forme de texte, de nombres, de son ou d’images
- les instructions composant un programme.

Rq: Le mot informatique vient de la contraction des deux mots : information et automatique.

Intro

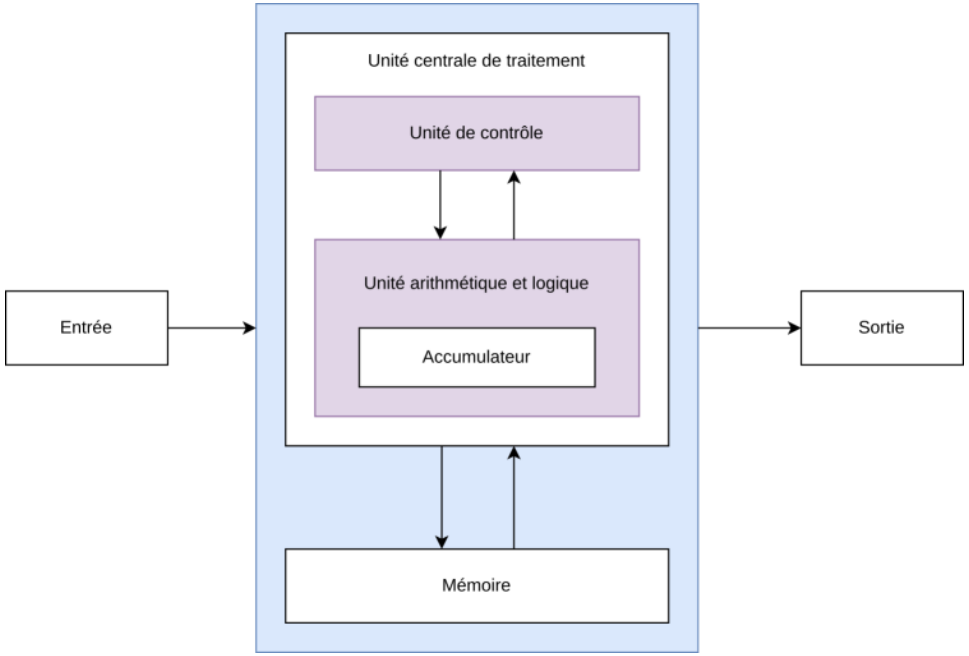
- Les différentes « vues » d'un ordinateur
  - Service
  - Fonctionnelle
  - Matérielle



Architecture Von Neumann (1903-1957)

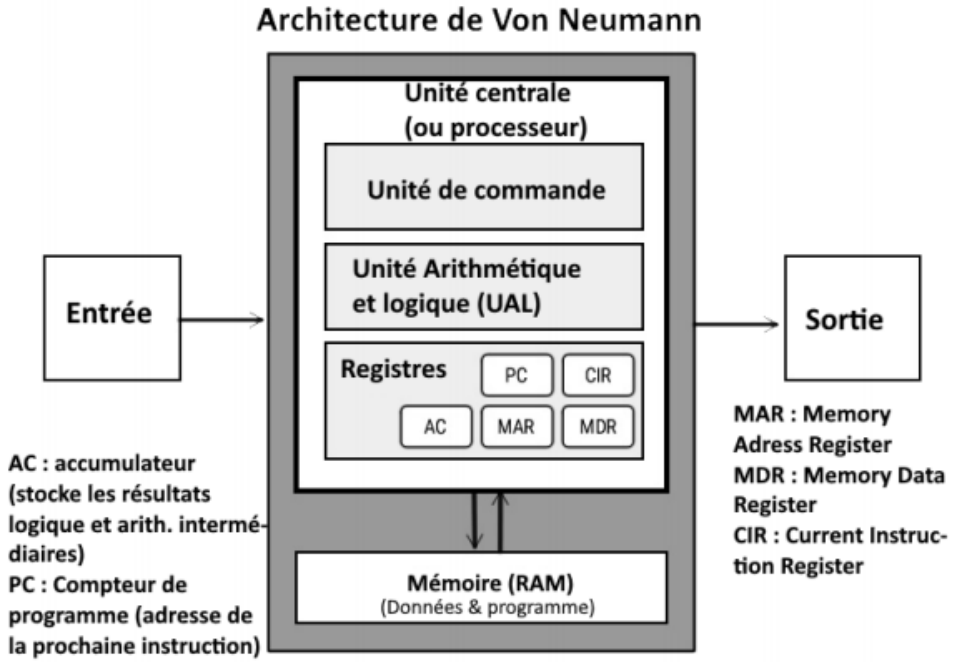


- L'architecture d'un système à microprocesseur = organisation de ses unités + leurs interconnexions.
- Pour traiter une information, un microprocesseur seul ne suffit pas, il faut l'insérer au sein d'un système de traitement de l'information.
- 1945 : John Von Neumann propose l'architecture interne d'une machine universelle de traitement de l'information. Elle est composé des éléments suivants :
  - Le microprocesseur (Unité centrale de traitement=CPU) ;
  - Une mémoire principale (MP)
    - contient les instructions et les données lors de l'exécution d'un programme ;
  - Des interfaces d'E/S pour communiquer avec l'extérieur.



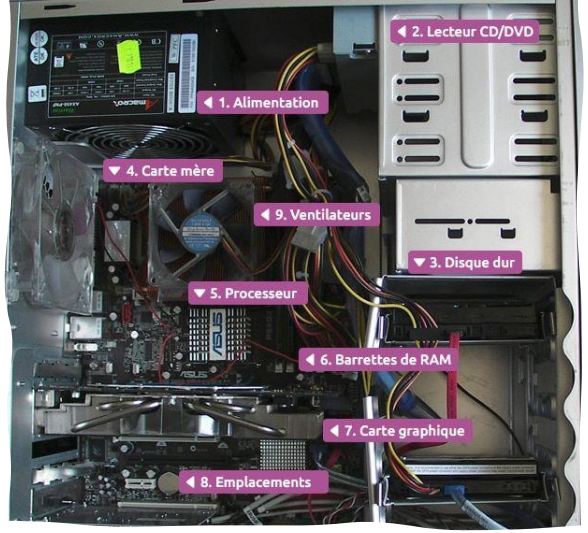
Architecture Von Neumann

- L'architecture d'un système à microprocesseur représente l'organisation de ses différentes unités et de leurs interconnexions.
- Le choix d'une architecture est toujours le résultat d'un compromis :
  - entre performances et coûts
  - entre efficacité et facilité de construction
  - entre performances d'ensemble et facilité de programmation

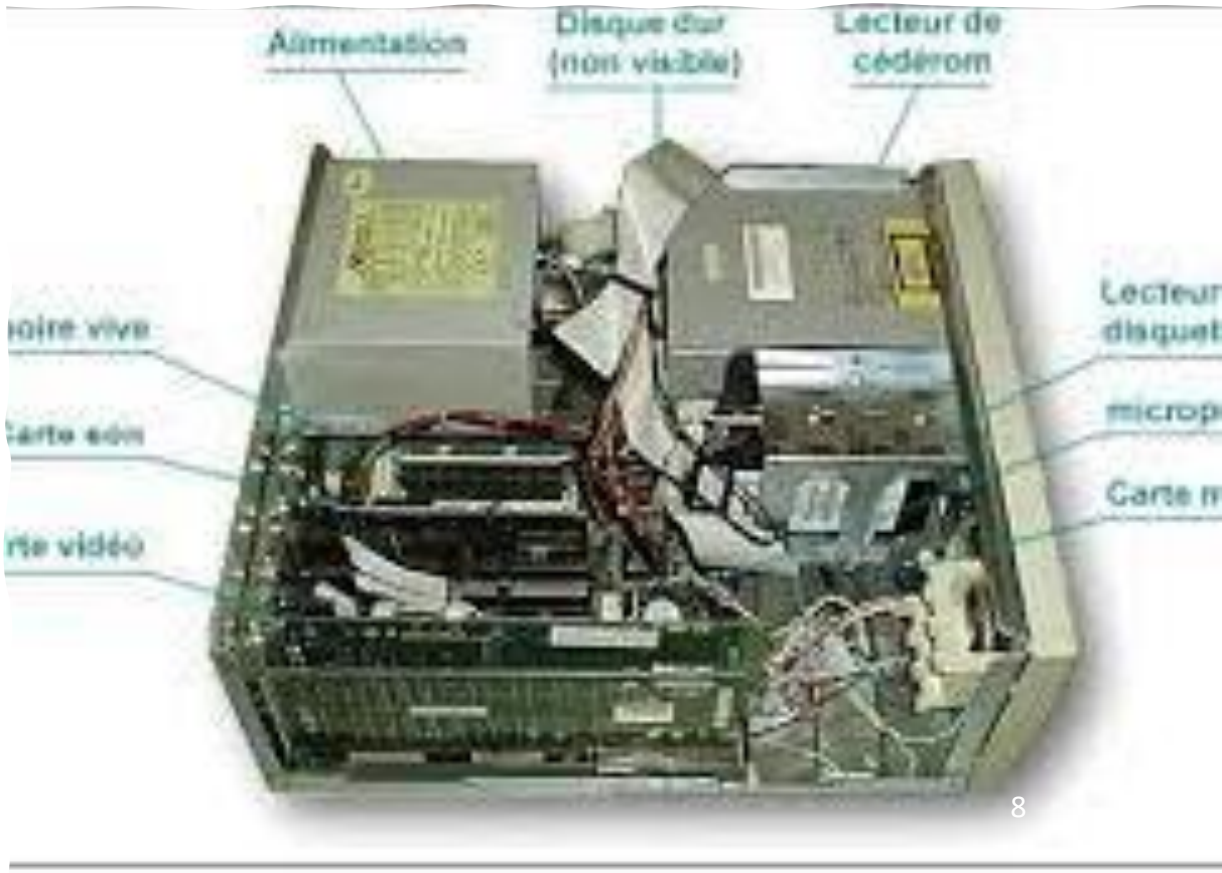


Matériel

- Un ordinateur est constitué de plusieurs parties :
  - Souris, écran, clavier, unité centrale, etc.
- A l'intérieur de l'unité centrale
  - une carte mère, une carte vidéo, des disques



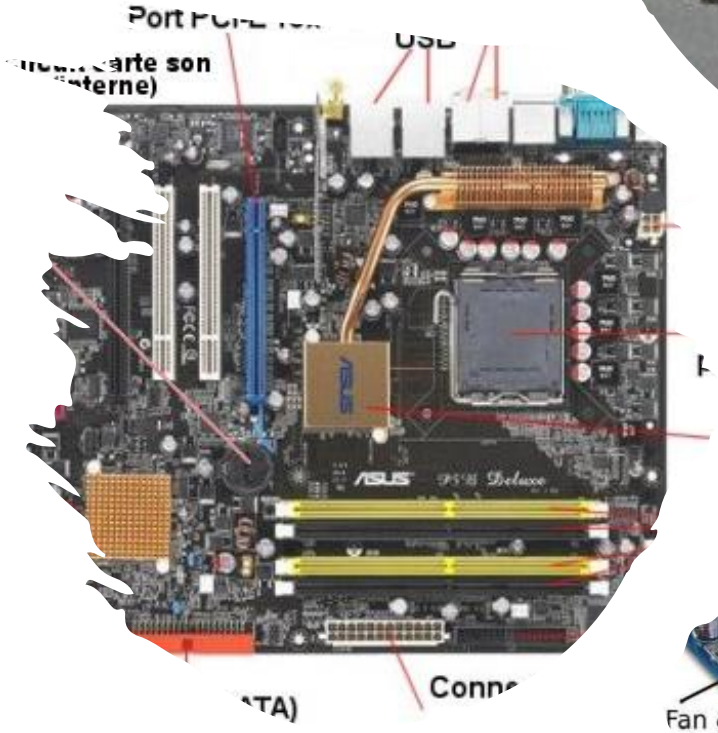


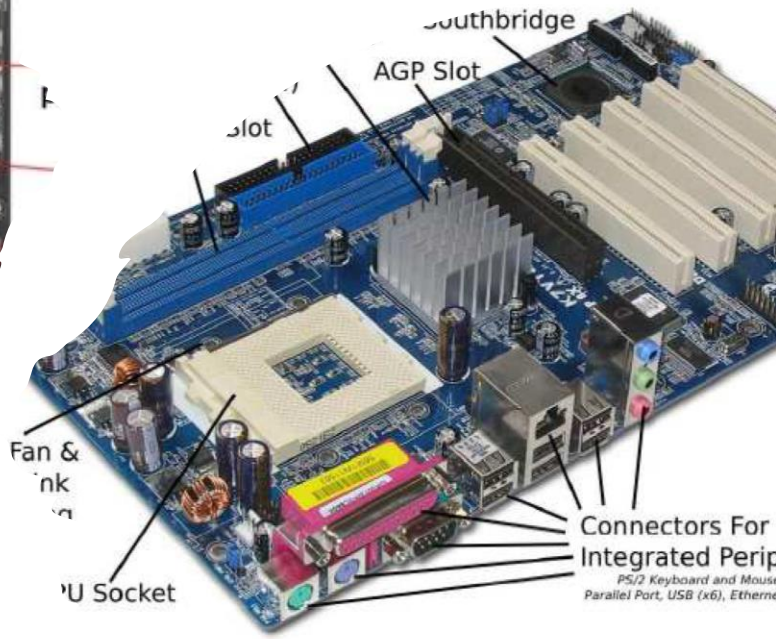


Matériel

- Sur la carte mère
  - un microprocesseur,
  - de la mémoire (ROM, RAM) ...
- Dans le microprocesseur, la puce



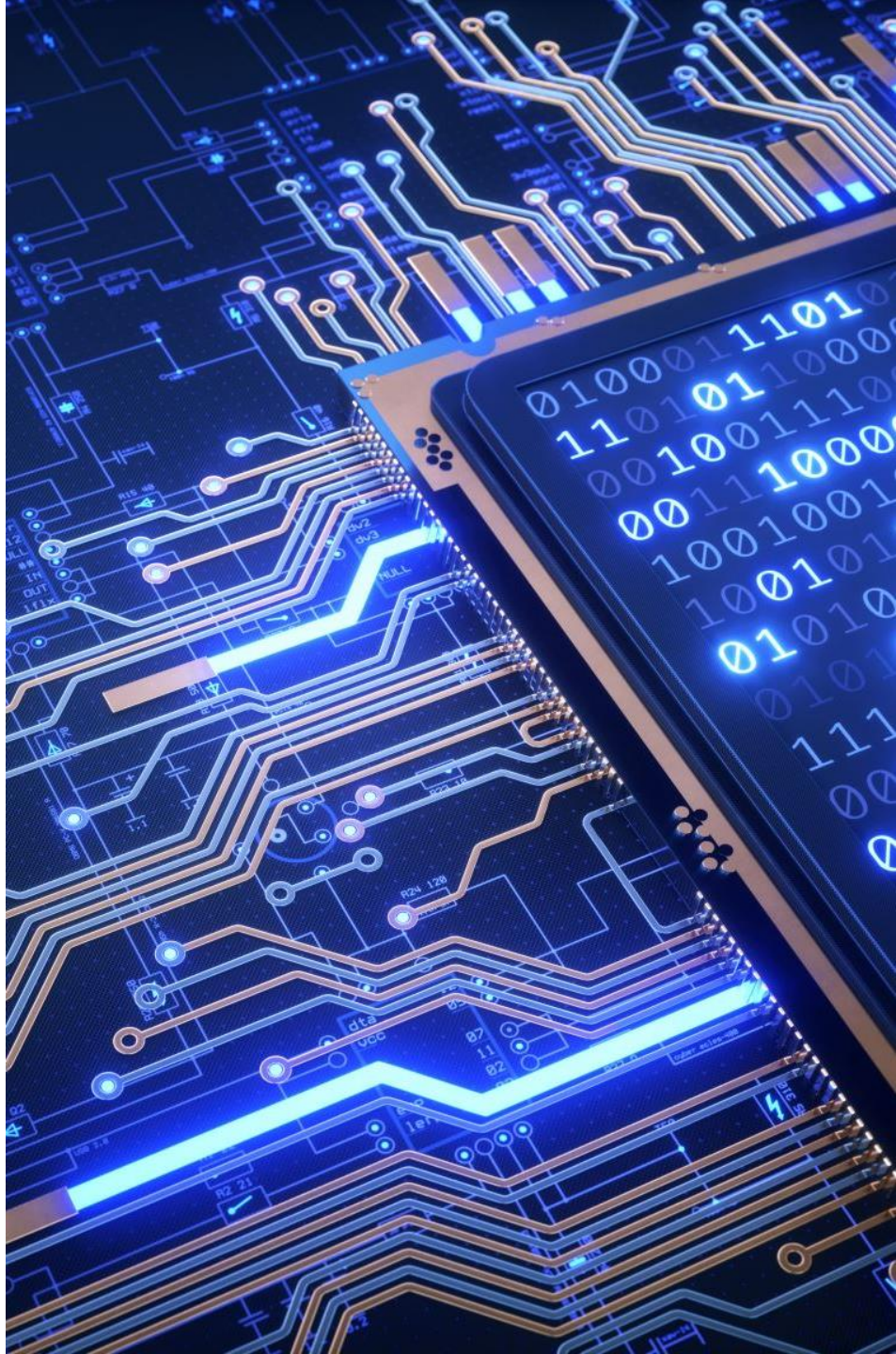




Récapitulons

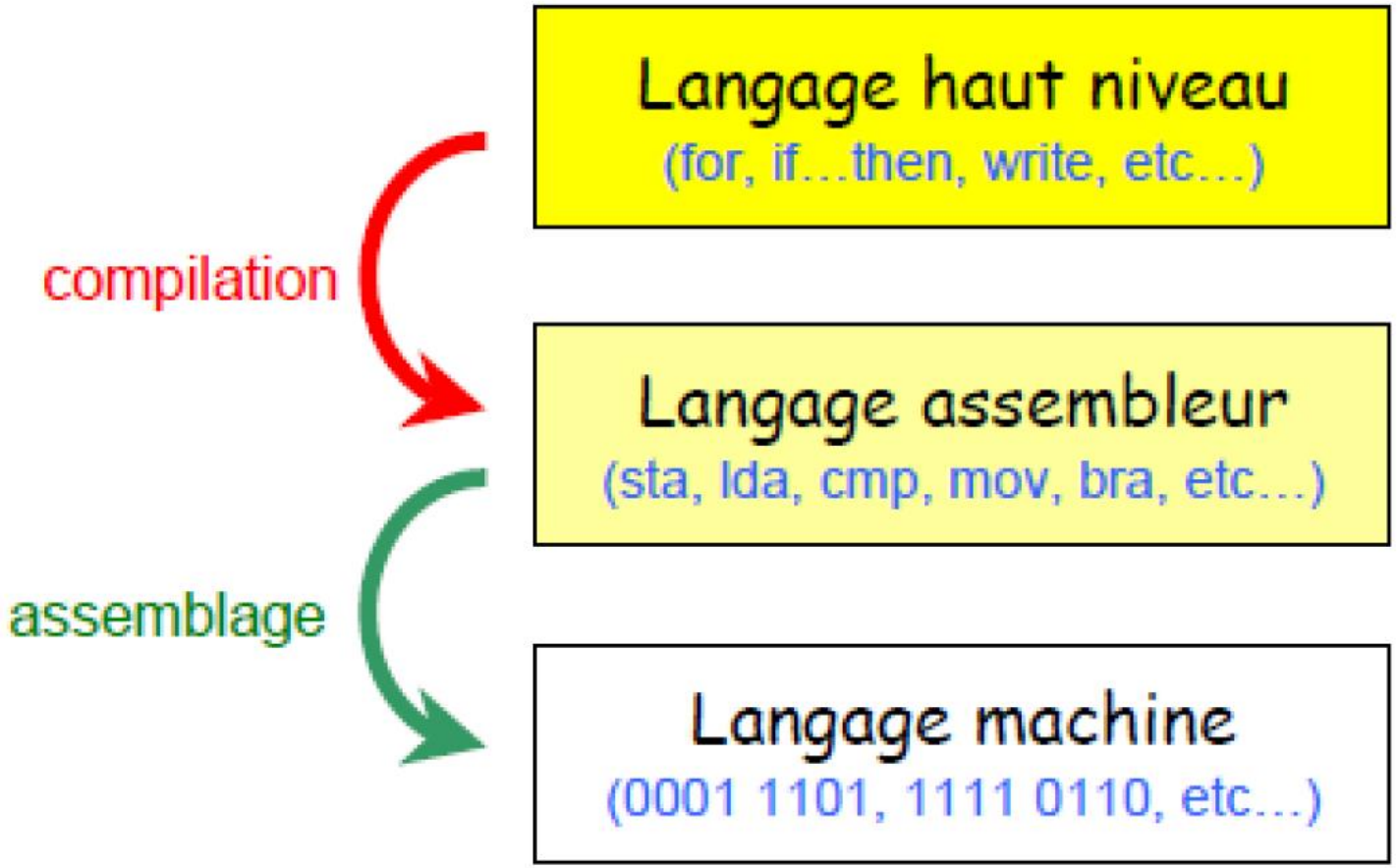
Ordinateur = **machine de traitement automatique d'informations**

- Machine ⇒ technologie (électronique, mécanique …)
- Traitement ⇒ opérations
- Automatique ⇒ autonome
- Information = connaissance sur quelque chose ou quelqu'un



Pour une exécution sur une architecture de type Von Neumann

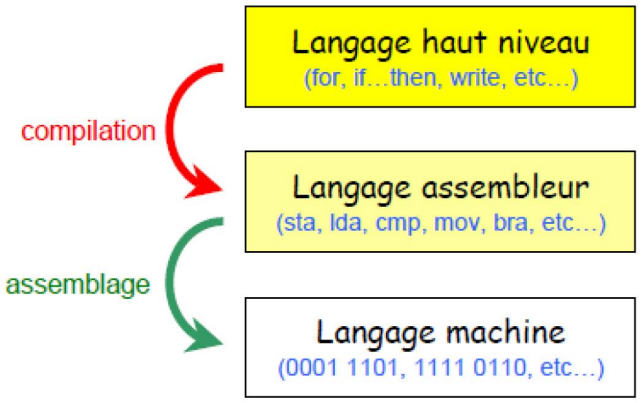
Du programme à son exécution



Compilation

- Lorsque l'on écrit un programme on utilise un langage de programmation (C, C++, java, python, …)
- Les langages de programmation ont une syntaxe (caractères spéciaux, noms de variables …) et une grammaire très précise : *if (condition) …*

- La machine ne comprend que des instructions (code d'opération + opérandes)
  - Opérande:
    - Donnée, quantité ou valeur, entrant dans une opération arithmétique ou logique
    - Valeur utilisée dans un traitement
- Il faut donc traduire le programme en instructions
- C'est le rôle du compilateur
- Lorsqu'il ne peut pas traduire il indique des erreurs




```
.model small

.stack 100h

.data
   bonjour  db  "Hello world!$"

.code
   main  proc
        mov  AX,@data
        mov  DS, AX
        mov  DX, offset bonjour
        mov  AH,09h
        int  21h
        mov  AX,4C00h
        int  21h
   main endp
end main
```

Représentation de l'information

- Par langage :
  - Alphabet = ensemble de symboles
  - Constitution de mots (concaténation, superposition)
  - Format des mots (fixe ou variable)
  - Taille des mots (taille ou taille max)
  - Règles syntaxiques (facultatives ⇒ mots interdits)

```
.model small

.stack 100h

.data
   bonjour  db  "Hello world!$"

.code
   main  proc
       mov  AX,@data
       mov  DS, AX
       mov  DX, offset bonjour
       mov  AH,09h
       int  21h
       mov  AX,4C00h
       int  21h
   main endp
end main
```

- Alphabet de k symboles
  - Format fixe taille n ⇒ on a k^n mots possibles
  - Format variable taille max n ⇒ on a $\frac{k^{n+1}-k}{k-1}$ mots possibles

Traitement d'informations

Traitement d'informations « à la main »

- Informations
  - données du problème
  - résultats
- Le **traitement** consiste à passer des données du problème au résultats par une méthode quelconque (*comme on peut*)

Traitement par une machine

- Codage : on représente les données du problème dans un langage compréhensible par la machine (par exemple en binaire)
- Traitement automatique : la machine effectue les traitements qui produisent les résultats dans le même type de représentation. Il faut donc que la méthode soit connue.
- Décodage : il suffit de décoder ces informations pour les présenter sous une forme compréhensible par l'utilisateur (texte, image, …)

Choix de la représentation

```
.model small

.stack 100h

.data
   bonjour  db  "Hello world!$"

.code
   main  proc
       mov  AX,@data
       mov  DS, AX
       mov  DX, offset bonjour
       mov  AH,09h
       int  21h
       mov  AX,4C00h
       int  21h
   main endp
end main
```

- Langage : binaire pour des raisons techniques (pas obligatoire)
- Assignation sémantique = associer un mot du langage à chaque information (arbitraire)
  - MOV = 110011; INT = 001100; etc.
- Non ambigüité : un mot ne peut pas désigner plusieurs informations
  - MOV = 0 ; INT = 00; => si 000 ? MOV puis INT ? 3 MOV ? INT puis MOV ?
- Choix de l'assignation sémantique arbitraire mais il vaut mieux choisir une assignation qui facilite les traitements
- Représentation des caractères :
  - Codes standards respectant l'ordre alphabétique (UTF: *Universal Character Set Transformation Format*)
- Représentation des nombres :
  - Faciliter les opérations arithmétiques, mathématiques, …
- Représentation des autres informations :
  - Au coup par coup (vous aurez à le faire)
  - Parfois plusieurs représentations selon les opérations (ex : couleurs)

Les nombres

- Entiers naturels :
  - En base 2 : N s'écrit $a_{n-1}\ a_{n-2} \dots a_1\ a_0$ avec a_i qui vaut 0 ou 1

    et N vaut : $a_{n-1} * 2^{n-1} + a_{n-2} * 2^{n-2} + \dots a_1 * 2 + a_0$
  - Les opérations classiques fonctionnent bien
- Entiers relatifs :
  - Le principe est d'utiliser la moitié des mots possibles pour les positifs et l'autre moitié pour les négatifs (sur n bits on a 2^n mots possibles)
  - On cherche une représentation dans laquelle la somme d'un nombre et de son opposé donne 0
  - Représentation en complément à 2 :
    - Un nombre positif est représenté comme un entier naturel mais le premier bit doit être 0 (première moitié)
    - Un nombre négatif N est représenté comme l'entier naturel $2^n + N$ (seconde moitié)
  - Les opérations classiques fonctionnent bien
- Réels
  - Notation scientifique (mantisse et exposant) $\Rightarrow$ 2 entiers relatifs
  - Mais les opérations ne sont pas simples

Ordinateur

- But : concevoir une machine automatique de traitement d'informations
- Théorie : machine de Turing (Alan Turing, 1912-1954; *The Imitation Game*)
  - Machine théorique qui peut calculer de façon autonome tout ce qui est calculable
  - Tout n'est pas calculable (Kurt Gödel 1906-1978) - le **théorème d'incomplétude de Gödel**, « *n'importe quel système logique suffisamment puissant pour décrire l'arithmétique des entiers admet des propositions sur les nombres entiers ne pouvant être ni infirmées ni confirmées à partir des axiomes de la théorie. Ces propositions sont qualifiées d'indécidables* » => Exemple PI !
- Il suffit de concevoir une machine qui peut se substituer à une machine de Turing
- Elle doit donc savoir faire au moins l'équivalent de ce que fait la machine de Turing (mais elle peut savoir faire plus).
- On n'évitera pas les limites dues au matériel (coût, fiabilité, tailles limités, temps de traitement, …)
- On cherchera à les minimiser
- Von Neumann propose une architecture simple qui est équivalente à la machine de Turing $\Rightarrow$ le problème est résolu

Machine de Turing



- Un ruban infini divisé en cases consécutives.
  - Chaque case contient un symbole d'un alphabet fini donné. L'alphabet contient un symbole spécial appelé « symbole blanc » ('0' dans les exemples qui suivent), et un ou plusieurs autres symboles.
  - Le ruban est supposé être de longueur illimitée vers la gauche ou vers la droite, en d'autres termes la machine doit toujours avoir assez de longueur de ruban pour son exécution.
- Une tête de lecture/écriture qui peut lire et écrire les symboles sur le ruban, et se déplacer vers la gauche ou vers la droite du ruban ;
- Un registre d'état qui mémorise l'état courant de la machine de Turing. Le nombre d'états possibles est toujours fini, et il existe un état spécial appelé « état de départ » qui est l'état initial de la machine avant son exécution ;
- Une table d'actions qui indique à la machine quel symbole écrire sur le ruban, comment déplacer la tête de lecture (par exemple « ← \leftarrow » pour une case vers la gauche, « → \rightarrow » pour une case vers la droite), et quel est le nouvel état, en fonction du symbole lu sur le ruban et de l'état courant de la machine.

Représentation de l’information

Représentation de l'information

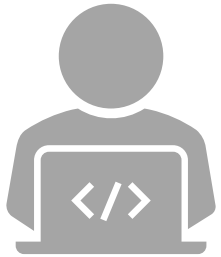


Information externe (*manipulées en Système; R 1.04*)

Formats multiples et variés

textes, images, sons ...

Systèmes d'acquisition des données (micros, capteurs, scanners)



Information interne (*manipulées en Système; R 1.03*)

Binaire 0101111...

Nécessité d’avoir des unités d'échanges : transformation de l'information en binaire

Représentation de l'information

- Ensemble de composants = quelques centaines de milliers de transistors /composants
- Ils fonctionnent selon deux états logiques notés 0 et 1 (logique binaire)
  - correspond à deux niveaux électriques 0 et + 5 /3,3 V
- Information logique (0,1) représente
  - des chiffres, nombres (entiers, réels)
  - caractères
  - chaîne de caractères
- Autres possibilités de codage (extension du binaire)
  - Hexadécimal, Binaire, UTF, ...

Représentation de l'information

Un **système de numération** est un ensemble de règles permettant de représenter les nombres. Dans les systèmes numériques, on utilise principalement les systèmes suivants :

Le système de numération **décimale**	• utilise les dix chiffres **0**, **1**, **2**, **3**, **4**, **5**, **6**, **7**, **8**, **9**.
Le système de numération **binaire**	• utilise exclusivement les deux chiffres **0** et **1**. (que l'on appelle alors **bit** – *binary digit*)
Le système de numération **hexadécimale**	• utilise les seize chiffres **0**, **1**, **2**, **3**, **4**, **5**, **6**, **7**, **8**, **9**, **A**, **B**, **C**, **D**, **E**, **F**.

Représentation de l'information

- Par convention, l'écriture d'un nombre **N** s'effectue par la juxtaposition de **chiffres** possédant chacun un **poids** égal à une puissance entière de la base de numération.
- Les chiffres s'écrivent de la gauche vers la droite par valeur décroissante de leur poids. Le poids d'un chiffre dépend donc de son rang et du système de numération adopté

$N = a_n b^n + a_{n-1} b^{n-1} + \ldots + a_1 b^1 + a_0 b^0$

Poids fort — Poids faible

a_i b^i — Rang de a_i — base — Poids de a_i

a_i = 0 ou 1 => bit

- **Exemple :** Base 2

$10_{10} = 1.2^3 + 0.2^2 + 1.2^1 + 0.2^0$ ← 8+2

$10_{10} = 1010_2$

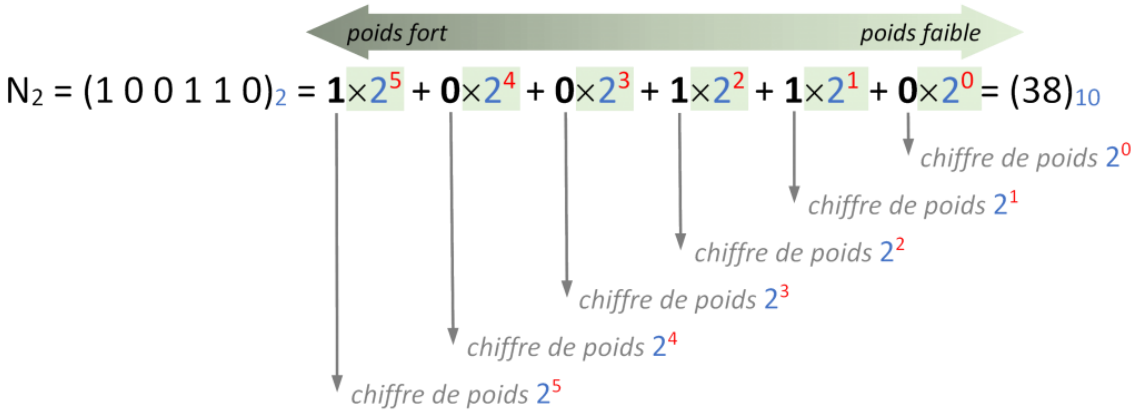
Indique la base

poids fort — poids faible

$N_1 = (7248,5)_{10} = 7\times10^3 + 2\times10^2 + 4\times10^1 + 8\times10^0 + 5\times10^{-1}$

chiffre de poids 10^3 — chiffre de poids 10^2 — chiffre de poids 10^1 — chiffre de poids 10^0 — chiffre de poids 10^{-1}

Représentation des nombres entiers naturels

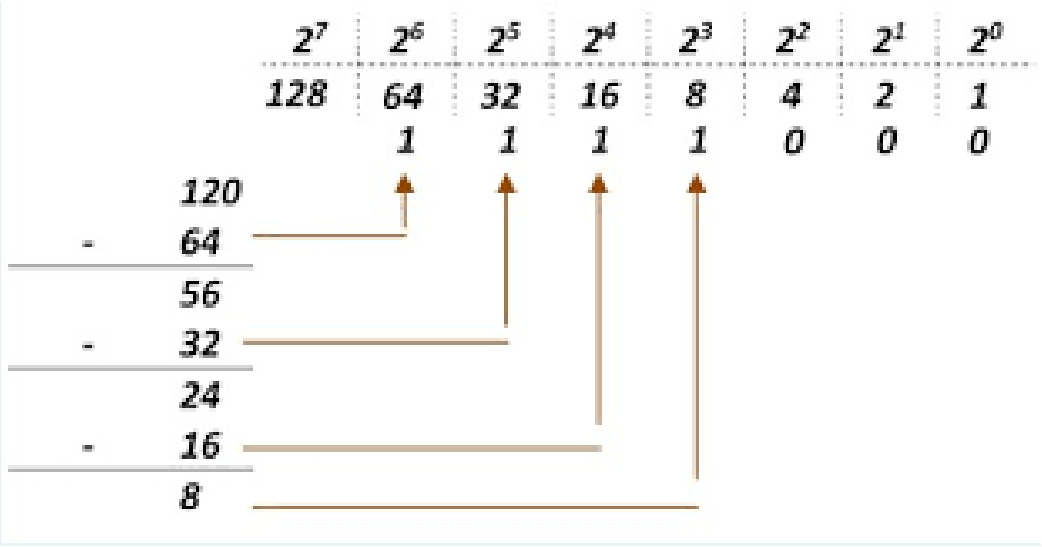


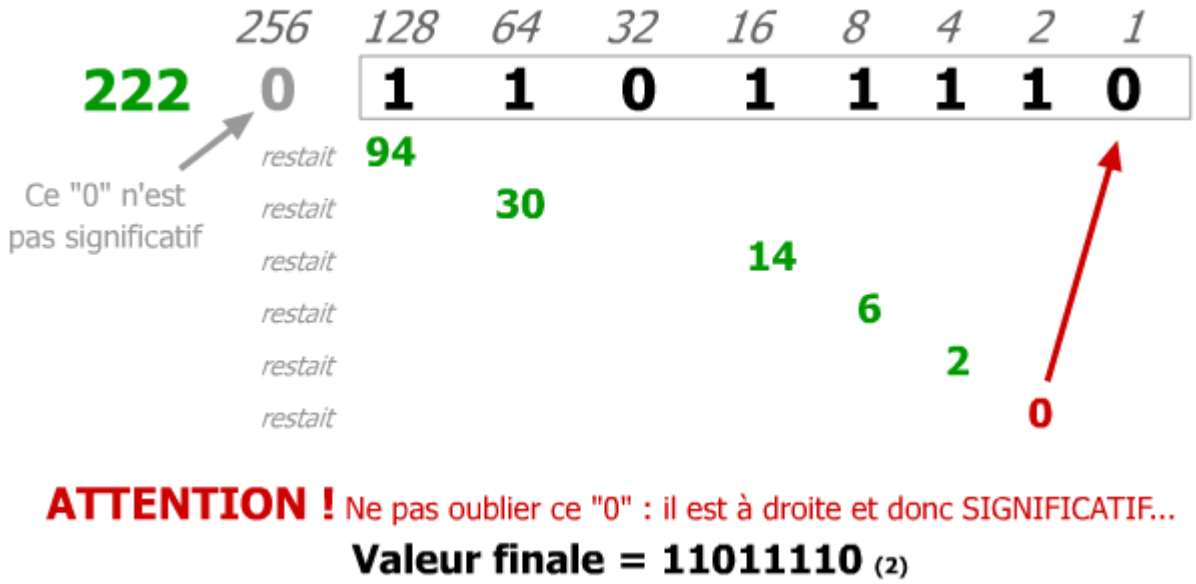
Codage binaire naturel des 10 chiffres de la base 10 :

	2 3	**2 2**	**2 1**	**2 0**
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Représentation de l'information

- Calcul binaire





Blague d'informaticien : « *dans le monde, il y a 10 catégories de personnes: celles qui connaissent le binaire et celles qui ne le connaissent pas* »

Le code hexadécimal

- Le **code hexadécimal** permet une représentation simplifiée des nombres binaires, à l'usage des humains !

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Tableau récapitulatif : **Décimal**, **Hexadécimal**, **Binaire**

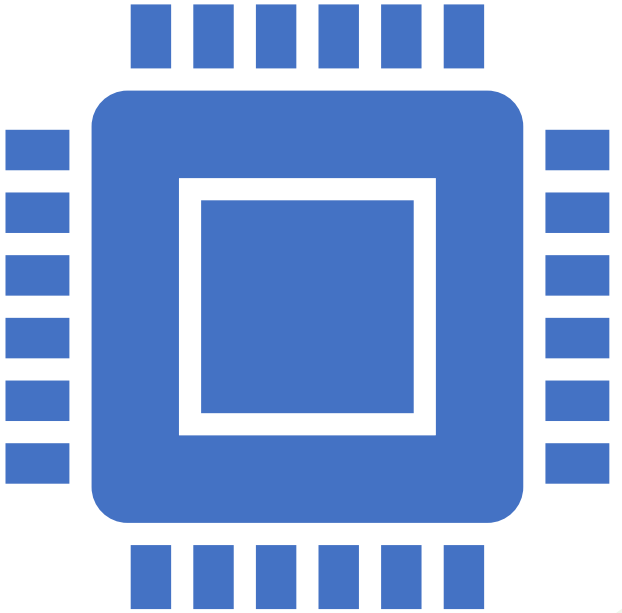
26

en puissance	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
en décimal	128	64	32	16	8	4	2	1
valeur du groupe	8	4	2	1	8	4	2	1
nombre à convertir	0	0	0	1	1	0	1	0
résultat				1	8		2	
Calcul en décimal conversion en hexa	1 = 1 1				8 + 2 = 10 10 = A			

Représentation des nombres

- Ordinateurs travaillent sur un nombre fixe de bits
- La notion d'octet
  - un octet = 8 bits
- La notion de mot (unité)
  - un mot = 8, 16, 32, 64 bits (dépend du processeur)
  - Zilog Z80 -> 8 bits (10101010)
  - Intel 80386 et 80486 -> 32 bits (10101010101010101010101010101010)
  - Core i7 -> 64 bits (10101010101010101010101010101010)
  - Certaines cartes vidéo => 256, 512 voire 1 Ko bits

- Taille des bus = 1 mot (selon processeur)

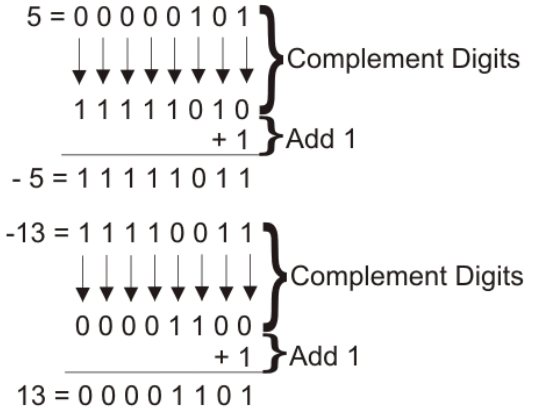


Représentation du signe

- Valeur Absolue
  - Sur n bits, le bit de poids fort représente le signe (0, positif; 1, négatif)
  - Les autres bits représentent la valeur absolue
  - Exemple sur 4 bits



- Complément à 2
  - Inverser tout
  - +1
  - ou… recopie jusqu'au 1er 1 puis on inverse



- Conséquence des signes
  - Autant de possibilités
  - Mais… Valeur Max / 2

Utilité de savoir compter.. Exemple avec C/C++

TYPE	DESCRIPTION	TAILLE
int	**entier standard signé**	**4 octets: - 2^{31} à 2^{31}-1**
unsigned int	**entier positif**	**4 octets: 0 à 2^{32}**
short	**entier court signé**	**2 octets: - 2^{15} à 2^{15}-1**
unsigned short	**entier court non signé**	**2 octets: 0 à 2^{16}**
char	**caractère signé**	**1 octet : - 2^{7} à 2^{7}-1**
unsigned char	**caractère non signé**	**1 octet : 0 à 2^{8}**



Représentation des données

- Caractères à coder, alphabet + , ! * " %

(old school)

- Code ASCII (7 bits + 1 bit de parité)
  - American Standard Code for Information Interchange
  - permet 128 combinaisons différentes utilisation de cette table ...
- Code EBCDIC (8 bits, 256 caractères)
  - Extended Binary Coded Decimal Interchange Code
  - Utilisé principalement par IBM
- Code ANSI
  - American National Standard Institute
  - Utilisés par certains logiciels (Windows)
  - Code ASCII + extensions multilingue alphabets occidentaux

Représentation des données

- Bien que de moins en moins fréquent, on trouve parfois des phrases dans lesquelles certains caractères sont remplacés par d'autres
  - connus sous le nom de **mojibakes**) qui n'ont rien à voir et qui empêchent la lecture et la compréhension du texte.
- Il s'agit ici d'un problème d'encodage et de décodage.
- La personne qui écrit le texte utilise une norme différente de celle utilisée par celui qui le lit
- UTF (8, 16, 32)



Représentation des données

- La **norme Unicode** a donc été créée pour permettre le codage de textes écrits quel que soit le système d'écriture utilisé.
- L'Unicode est une table de correspondance Caractère-Code (Charset), et l'UTF-8 est l'encodage correspondant (Encoding) le plus répandu.
- Unicode :
  - UTF-8 (1 à 4 octets)
  - UTF-16 (2 ou 4 octets)
  - UTF-32 (4 octets)

Caractères codés	Représentation binaire UTF-8	Signification
U+0000 à U+007F	0xxxxxxx	1 octet, codant 7 bits
U+0080 à U+07FF	110xxxxx 10xxxxxx	2 octets, codant 11 bits
U+0800 à U+FFFF	1110xxxx 10xxxxxx 10xxxxxx	3 octets, codant 16 bits
U+10000 à U+FFFFF	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx	4 octets, codant 21 bits

Table UTF

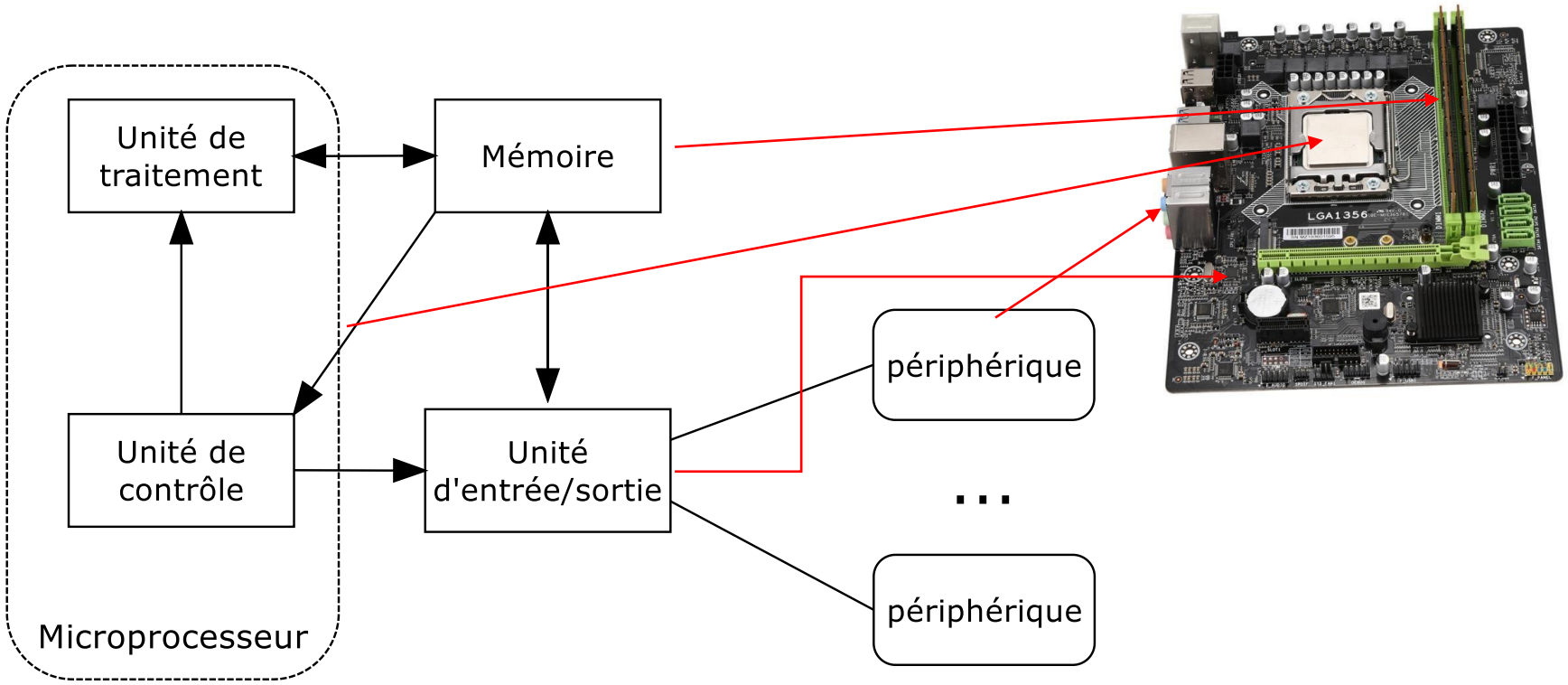
Caractères nationaux

Jeu de commandes

BITS				b_7	0	0	0	0	1	1	1	1
				b_6	0	0	1	1	0	0	1	1
				b_5	0	1	0	1	0	1	0	1
b_4	b_3	b_2	b_1		0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	à	P		p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	£	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	'	6	F	V	f	v
0	1	1	1	7	BEL	ETB	(	7	G	W	g	w
1	0	0	0	8	BS	CAN	)	8	H	X	h	x
1	0	0	1	9	HT	EM	.	9	I	Y	i	y
1	0	1	0	A	LF	SUB		:	J	Z	j	z
1	0	1	1	B	VT	ESC	,	;	K		k	é
1	1	0	0	C	FF	ES		<	L	ç	l	ù
1	1	0	1	D	CR	GS		¨	M	§	m	è
1	1	1	0	E	SO	RS		>	N	^	n	¨
1	1	1	1	F	SI	US	/	?	O	–	o	DEL

Architecture d’un ordinateur

Architecture de Von Neumann



Unité de traitements

Unité de traitement

- Rôle : effectuer les opérations sur les informations

- Machine de Turing :
  - Peut calculer tout ce qui est calculable
  - Les opérations qu'elle sait faire sont donc suffisantes pour un ordinateur
  - Il suffit que l'unité de traitement sache faire au moins l'équivalent de ces opérations là (il y en a très peu)
  - En général l'unité de traitement sait en faire beaucoup plus pour être plus efficace et plus rapide

L’unité centrale de traitement (UC)

Son Rôle

- Exécute les programmes en mémoire centrale
- Cycle de chargement, décodage et exécution des instructions
- Stocke les résultats intermédiaires dans des registres

Sa Composition

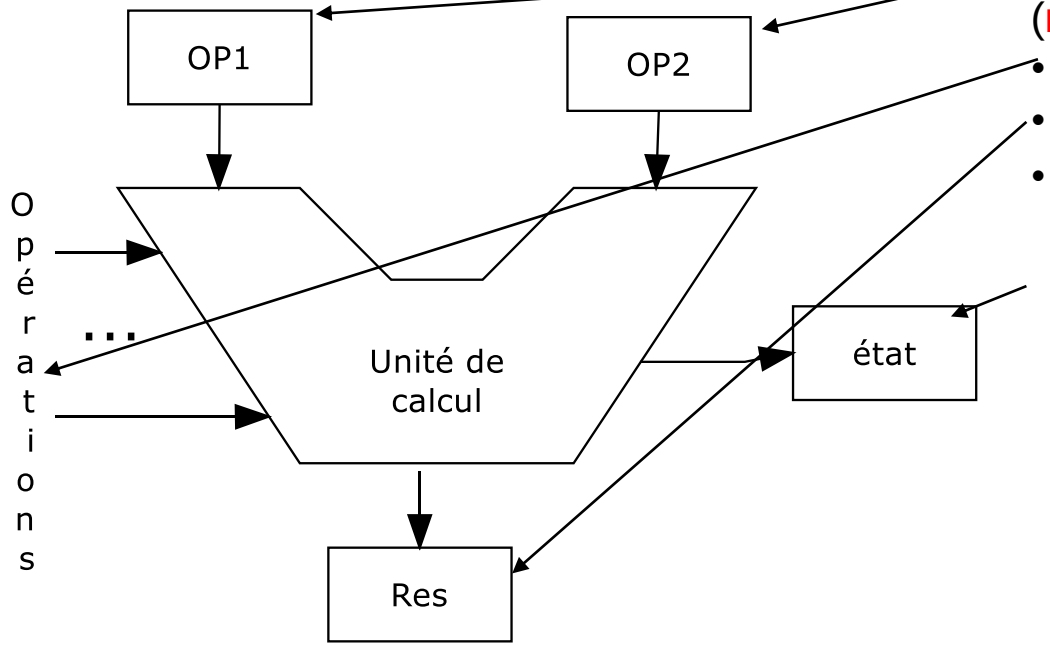
- Registres : mémorisation temporaire
- Unité Arithmétique et Logique : effectue les opérations
- Unité de commande : orchestre les opérations
- Bus : Communication des résultats
- Jeu d’Instructions : opérations

Unité de traitement

- L’assignation sémantique est choisie pour faciliter les opérations :
  - Entiers naturels en base 2
  - Entiers relatifs en complément à 2
  - Réels en notation scientifique (mantisse + exposant)
  - Caractères dans l’ordre alphabétique
  - …

- Opérations implémentées :
  - Arithmétiques (+ - * /)
  - Logiques (ET OU NON OUExclusif)
  - Décalages (Logiques, arithmétiques et cycliques)
  - Comparaison (= ≠ < ≤ > ≥)
  - Mathématiques (sin, log, …)
  - Vectorielles (+ - * / …)
  - …

Unité de traitement



- Opérandes mis dans les registres ‘OP1’ et ‘OP2’ (registre = mémoire ne contenant qu’une information)
- Commande d’opération
- Résultat de l’opération dans le registre ‘Res’
- L’état dans le registre ‘état’ indique :
  - Le résultat si l’opération est une comparaison
  - Les erreurs pour les autres opérations (division par 0 par exemple)

Les opérandes respectent une assignation sémantique adaptée aux opérations

Unités d’E/S

Unité d’entrée/sortie

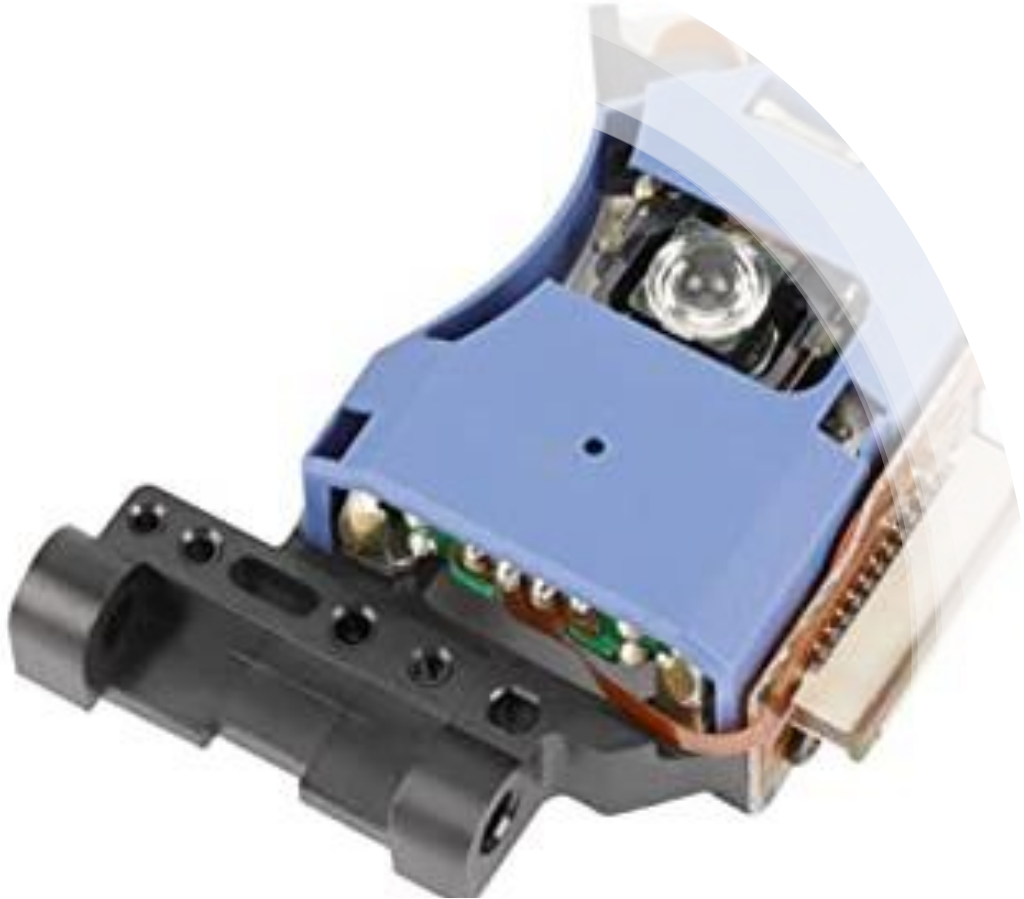
- Pilote les périphériques
- Son architecture dépend beaucoup des périphériques

- Il existe des standards de connexion (USB, HDMI, SATA, …) selon le périphérique
- Parfois on doit utiliser des extensions de l’UE/S (cartes) spécialisées









Périphériques de stockage

- Optique (CD/DVD)
  - Un laser éclaire la surface du disque
  - Un capteur mesure la lumière réfléchie
  - Un trou à la surface du disque $\Rightarrow$ pas de lumière réfléchie
  - Gravure en faisant des trous au laser
- Magnétique (disque dur)
  - Le disque est recouvert d'un revêtement magnétique
  - Un capteur lit les orientations du champ magnétique
  - Enregistrement par création d'un champ magnétique
- Utilisent de la mécanique (usure, fragilité)
- Supports peu chers au regard de la capacité.

Périphériques de stockage

- Charges électriques (flash, carte SD, disque SSD)
  - Des charges électriques sont piégées dans la puce
  - Lecture par le champ électrique crée par ces charges
  - Ecriture par création d'un champ électrique qui piège des charges
  - Nécessité d'effacement avant écriture pour libérer les charges piégées.
- Pas de mécanique : **plus rapide mais plus cher à capacité égale**

- Solutions mixtes :
  - Disque magnétique + disque SSD (plus petit)
  - Le système d'exploitation copie sur le disque SSD les fichiers les plus utilisés (plus rapide)



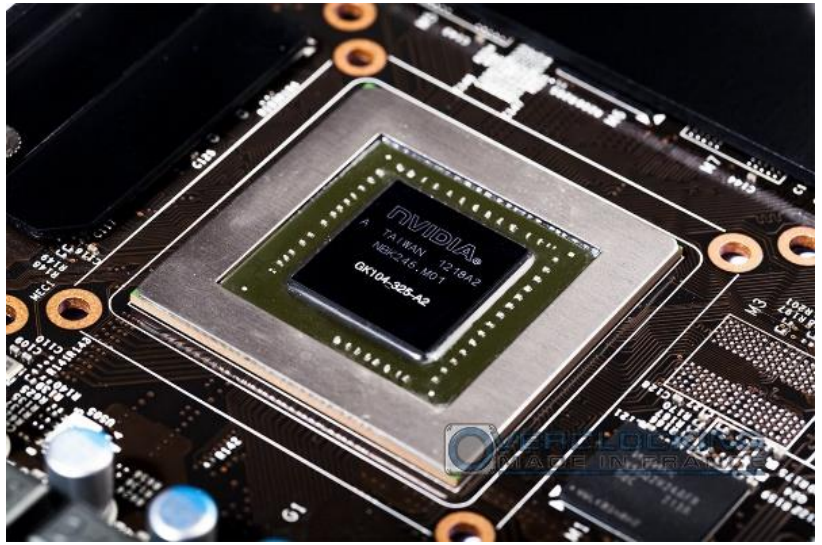
Périphériques d’entrée/sortie

- Transmission d’informations dans un sens ou dans les deux sens:
  - Clavier (touches appuyées / lâchées)
  - Souris (déplacements en x et y + boutons)
  - Imprimante (caractères + codes de commande)
  - Scanner (pixels)
  - Caméra (pixels ou vidéos encodées MP4 …)
  - Ecran : c’est le plus complexe de tous

Cartes graphiques

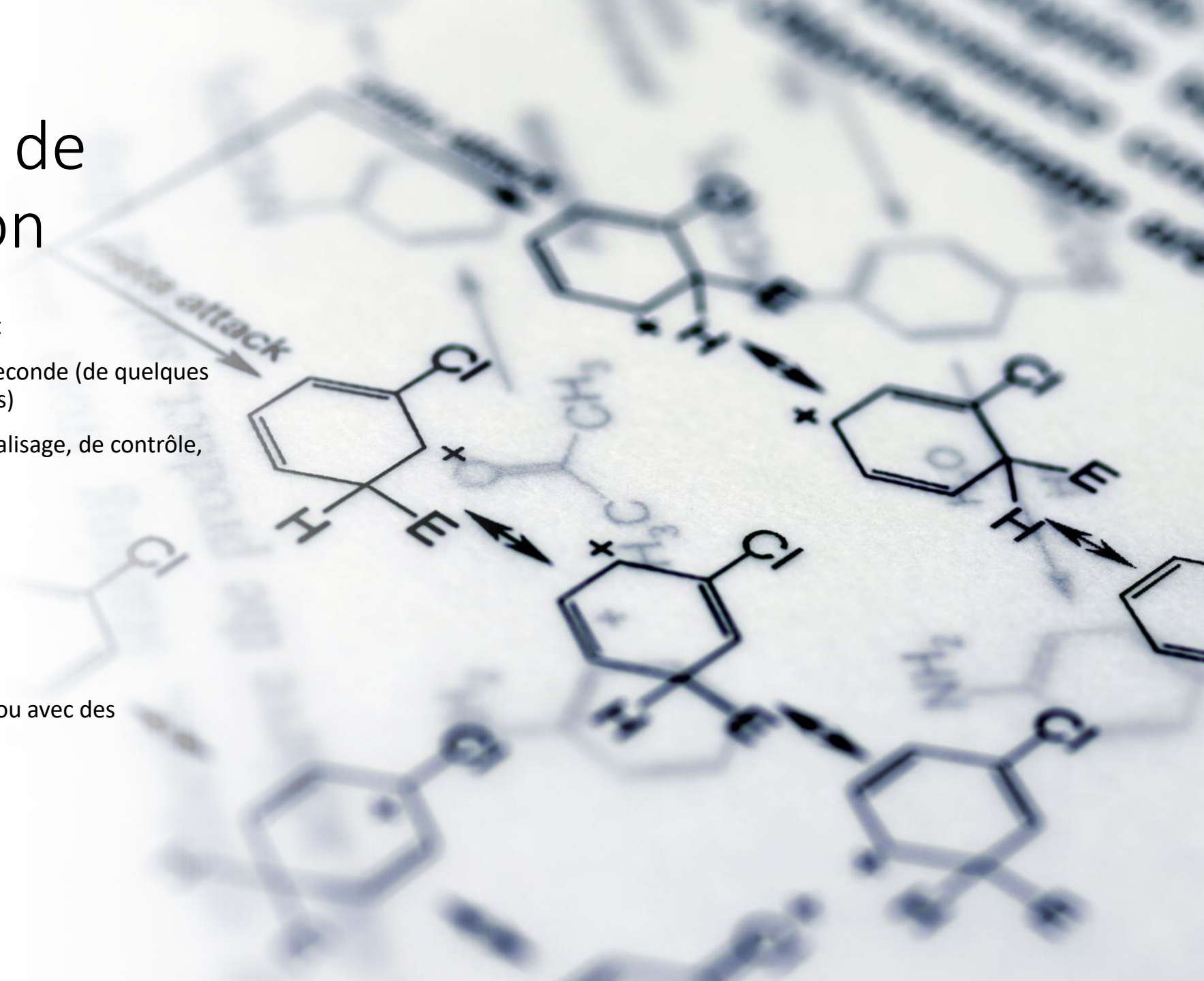


- L'écran est le périphérique qui demande le plus de calculs (3D, jeux …)
- Pendant un temps on a doté les unités de traitement d'opérations spécialisées pour ça
- Mais la solution la plus efficace est de doter l'UE/S de sa propre unité de traitement spécialisée (GPU : Graphics Processing Unit)
- Les GPU sont très performants (beaucoup plus que l'UT mais très spécialisés : calcul vectoriel et matriciel pour translations, rotations …) de l'ordre de 500 milliards d'opérations/s



Périphériques de communication

- Transmission d'informations bit par bit
- **Débit** = nombre de bits transmis par seconde (de quelques centaine de milliers à quelque milliards)
- **Protocoles** : ajout d'informations de balisage, de contrôle, de correction d'erreurs, …
- **Supports :**
  - Fil
  - Onde
  - Fibre optique
  - Infra rouge
- Pour communiquer entre ordinateurs ou avec des périphériques :
  - Ethernet
  - Wifi
  - Bluetooth
  - USB
  - Mais aussi HDMI, firewire, …



Les Bus

- Ensemble de “fils” (50 à 100) connectant des unités fonctionnelles au sein d’un ordinateur
  - Le **bus d'adresses** (appelé parfois *bus d'adressage* ou *bus mémoire*) transporte les adresses mémoire auxquelles le processeur souhaite accéder pour lire ou écrire une donnée. Il s'agit d'un bus unidirectionnel.
  - Le **bus de données** véhicule les instructions en provenance ou à destination du processeur. Il s'agit d'un bus bidirectionnel.
  - Le **bus de contrôle** (parfois *bus de commandes*) transporte les ordres et les signaux de synchronisation en provenance de l'unité de commande et à destination de l'ensemble des composants matériels. Il s'agit d'un bus directionnel dans la mesure où il transmet également les signaux de réponse des éléments matériels.

Les Bus

- On distingue généralement sur un ordinateur deux principaux bus :
- le **bus système** (appelé aussi *bus interne*, en anglais *internal bus* ou *front-side bus*, noté *FSB*).
    - Permet au processeur de communiquer avec la mémoire centrale du système (RAM)
- le **bus d'extension** (parfois appelé *bus d'entrée/sortie*) permet aux divers composants de la carte-mère (USB, série, parallèle, cartes branchées sur les connecteurs PCI, disques durs,, etc.) de communiquer entre eux mais il permet surtout l'ajout de nouveaux périphériques grâce aux connecteurs d'extension (appelés **slots**) connectés sur le bus d'entrées-sorties.



Exemples de bus

- Bus interne/local : Ex. PCI-Express (ou PCIe), 100 MHz

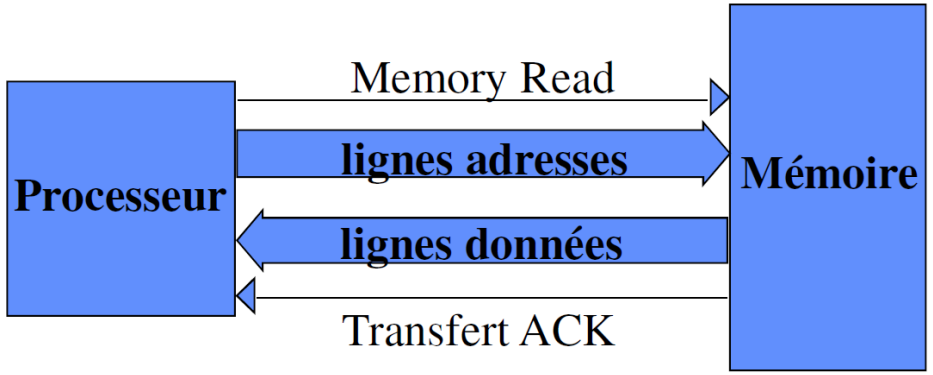
Génération	x1	x4	x8	x16
PCIe 1.1	250 Mo/s	1 Go/s	2 Go/s	4 Go/s
PCIe 2.0	500 Mo/s	2 Go/s	4 Go/s	8 Go/s
PCIe 3.0	1 Go/s	4 Go/s	8 Go/s	16 Go/s
PCIe 4.0	2 Go/s	8 Go/s	16 Go/s	32 Go/s
PCIe 5.0	4 Go/s	16 Go/s	32 Go/s	64 Go/s
PCIe 6.0	8 Go/s	32 Go/s	64 Go/s	128 Go/s

Bus externe

- Connexion carte mère / contrôleur disque
- USB 1,0: 1,5 Mbit/s; USB 1.1: 12 Mbit/s; USB 2.0: 480 Mbits/s; USB 3.0: 4,8 Gbit/s.

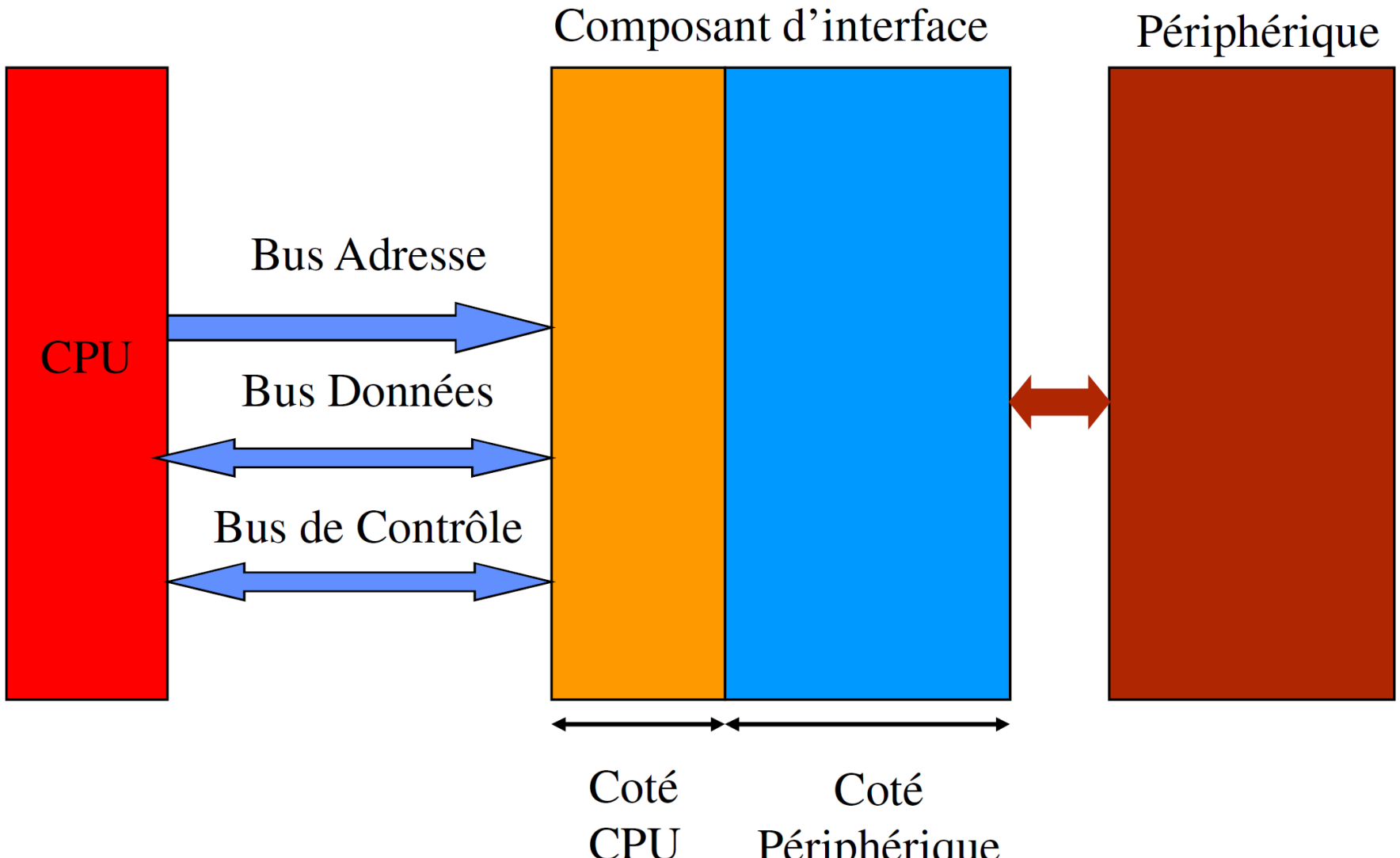
Schéma de fonctionnement du bus

- Connexion entre le processeur et la mémoire
  - lecture d'un mot de la mémoire



- Memory Read : le processeur signale qu'il a placé l'adresse sur la ligne
- Transfert ACK : la mémoire répond que les données sont disponibles

Exemple


Composant d'interface
Périphérique
CPU
Bus Adresse
Bus Données
Bus de Contrôle
Coté CPU
Coté Périphérique

Handshaking et Buffering

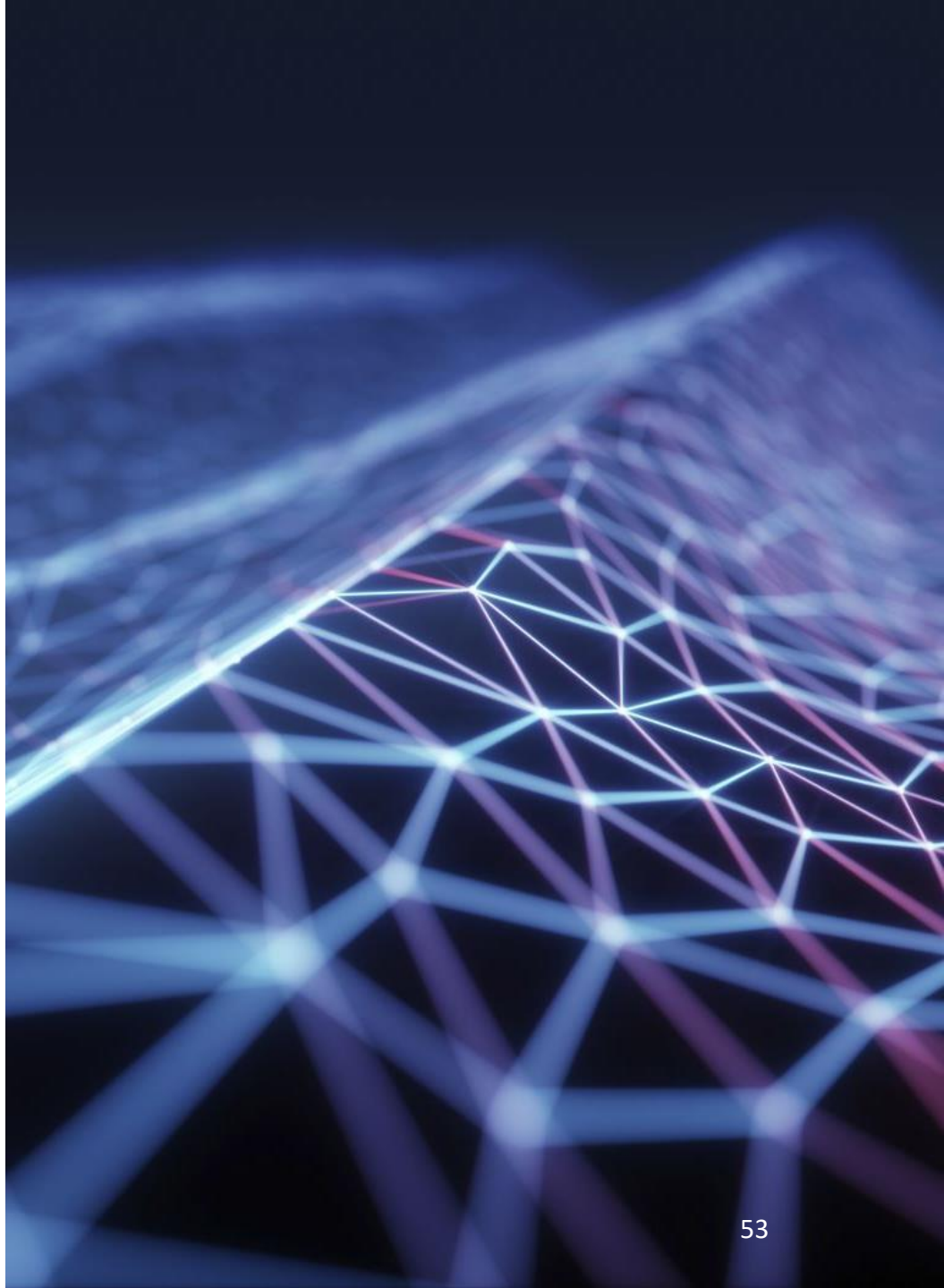
Les transferts de données d'E/S sont de 2 types :

Boucle ouverte : les données sont transférées et leur réception correcte est supposée après un certain délai

- Exemple du service de mail
- Généralement suffisant

Boucle fermée : il y a accusé de réception des messages (acknowledgement)

- Exemple de procédure d'approche d'un aéroport



Bufferisation des E/S

- Temps de traitement des interfaces ?
  - **Rapide** => problème de maintient des données
  - **Lent** => perte de temps à accéder aux données
  - **Exemple** : problème du temps moyen d'attente chez les médecins
- Buffer d'interface => FIFO
  - Détermination de la taille de la FIFO
- « Buffering »: stockage des données avant leur utilisation



Mémoire



Mémoire

- Cahier des charges :
  - On doit pouvoir y stocker des informations
  - Elle doit les conserver
  - On doit pouvoir les retrouver

- Quelles informations ?
  - Toutes celles manipulées par l'ordinateur (données du problème, résultats intermédiaires, résultats finals)

La mémoire centrale

Mémorise les programmes utilisateurs, le système d'exploitation

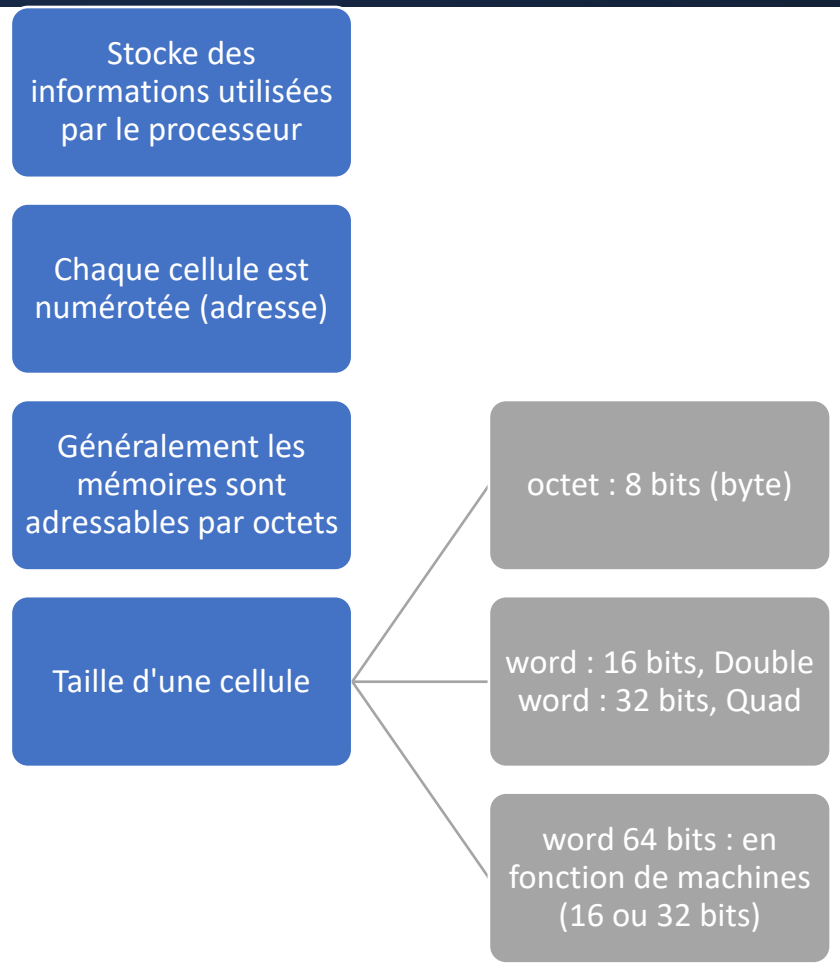
Taille

- Aujourd'hui en Go (Gigas) d'un PC (4, 8 16 voire 32 Go)
- Première approximation….Giga = milliards… Go => milliards de lettres

Située sur une carte mémoire

Communique avec le processeur (carte mère) via des bus (de données, d'adresse)

La mémoire


Stocke des informations utilisées par le processeur
Chaque cellule est numérotée (adresse)
Généralement les mémoires sont adressables par octets
Taille d'une cellule
octet : 8 bits (byte)
word : 16 bits, Double word : 32 bits, Quad
word 64 bits : en fonction de machines (16 ou 32 bits)

Types de mémoire

- La mémoire en informatique est le lieu de stockage permanent ou non de l'information. **Elle est multiple.**
- **Les Registres**: mémoire non permanente interne au CPU. On compte :
  - Registre mot (registre d'instruction): Sa taille classique actuelle est 32 ou 64 bits.
  - Registre adresse (compteur Ordinal): contient l'adresse d'un mot.

- **La mémoire cache**: stocke des données récemment accédées. Sa taille varie de quelques centaines de Ko à quelques Mo mais elle est rapide
  - peut être interne ou externe au CPU. Elle se situe entre le CPU et la RAM,
  - entre le CPU et un disque,
  - entre le CPU et une autre cache. par rapport à la MP.

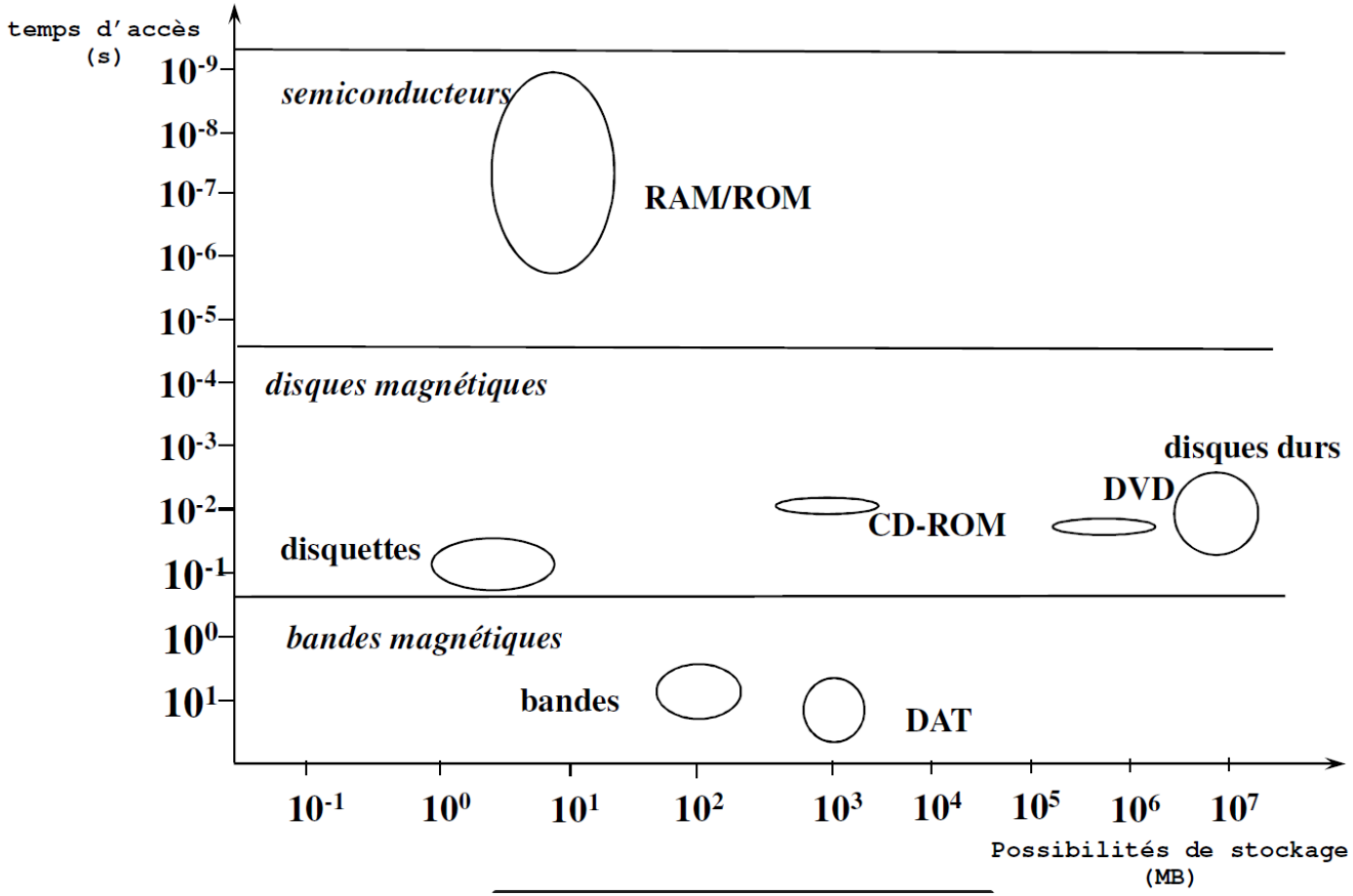
- **La mémoire centrale ou principale (MP) :** mémoire à semi-conducteurs. Elle comporte la mémoire vive (RAM) et la mémoire morte (ROM) ainsi que ses dérivées (PROM, EPROM, …).

- **La mémoire de masse**, permanente, secondaire ou auxiliaire : son rôle est le stockage permanent des données et est caractérisée par sa grande capacité (ex. supports magnétiques (disques durs), optiques (CD, DVD, Blu-ray) ou électriques (flash disques)).

Capacité et temps d’accès

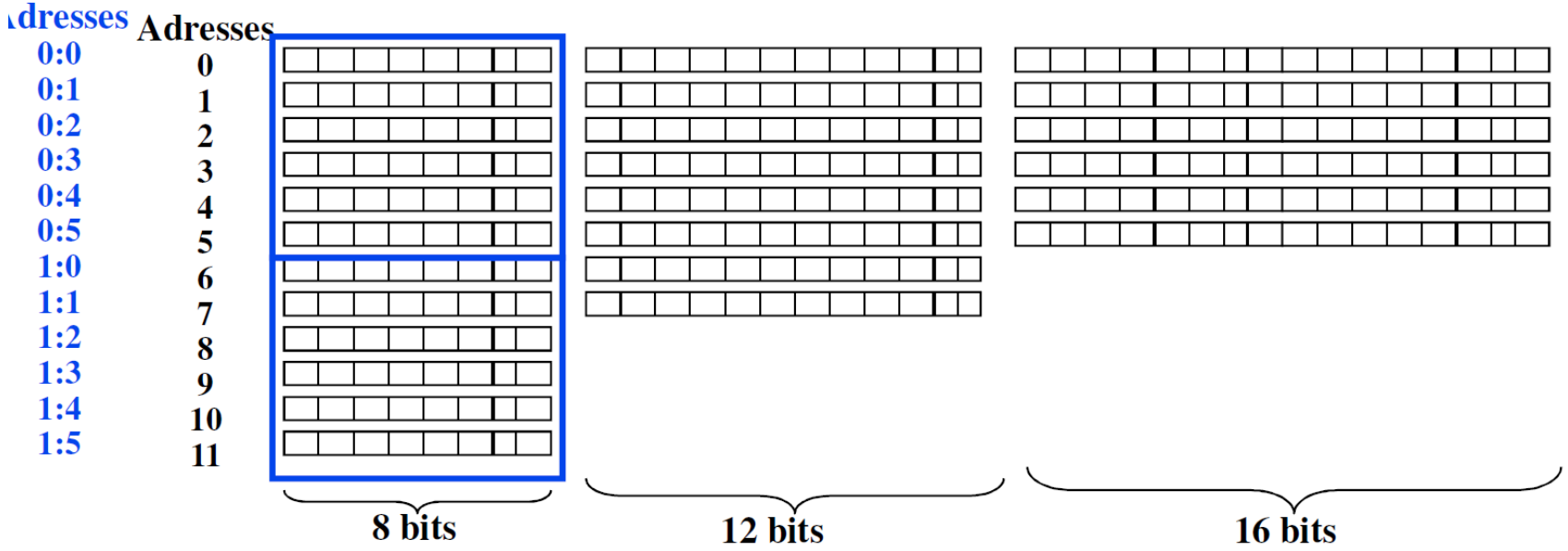
- Mot mémoire [word]: désigne le nombre de bits pouvant être lus ou écrits simultanément par le processeur. Sa longueur classique était de 32 bits et elle est passée actuellement à 64 bits.
- Capacité (taille) d'une mémoire: c’est le nombre de bits (registres) ou d'octets [bytes] (1 octet = 8 bits) qu'elle peut contenir
  - Bit: {0,1}; Octet = 8 bits {01010101};
  - ex. volumétrie 1 à 2 Téra octets (To) pour certains disques durs internes, 3 à 5 To pour les disques durs externes et 4Go pour certaines Mémoires principales (RAM).
  - Temps accès disque durs : 10ms (HDD); 70 à 300 µs (microsecondes, soit 0,07 à 0, ms)
- Temps d'accès mémoire
  - Temps d’accès pour la lecture ou l’écriture d'un mot mémoire. De l’ordre de la nanosecondes (10-9 s).
- Cycle mémoire: c’est le temps minimal entre 2 accès mémoire.
- Débit: nombre d'informations lues ou écrites par seconde.
- Temps d'accès registre: environ 10 fois plus court que le temps d'accès mémoire..

Types de mémoire



Organisation d’une mémoire

- Exemple: 96 octets selon taille mémoire (8, 12, 16 bits)



- Aujourd’hui plutôt 64, 128 voire 256 et même 512 octets

Caractéristiques des mémoires

Capacité : quantité d'informations qu'elle peut stocker

- kilo-octet -> Ko = 1024 octets
- méga-octet -> Mo = 1024 Ko
- giga-octet -> Go = 1024 Mo
- téra-octet -> To = 1024 Go

Exemple

Une mémoire centrale de 1 Méga mots de 16 bits

- 1 x 1024 K-mots
- 1 x 1024 x1024 mots
- 1 x 1024 x1024 x 2 octets
- 1 x 1024 x1024 x 2 x8 bits = 16 777 216 bits

Caractéristique des mémoires



Mémoire vive: RAM

Mémoire volatile

Cycle de lecture/écriture (@ bus d'adresse, V bus de donnée, CS, R/W)

Mémoire vive statique -> SRAM

Mémoire vive dynamique -> DRAM

Mémoire vive dynamique Synchrone -> SDRAM



Mémoire morte: ROM, PROM, EPROM, EEPROM

A lecture seule,

Conserve l'information

ROM : information stockée de manière définitive,

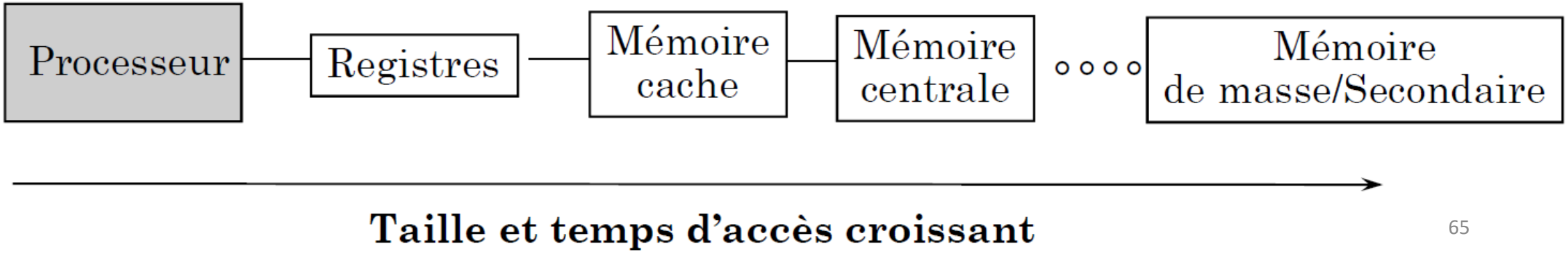
PROM : Programmable par l'utilisateur

EPROM -EEPROM : Programmable + effacement par UV ou électriquement

Hiérarchie des mémoires

- Machine idéale
  - taille mémoire illimitée
  - temps d'attente nul
  - Mais… cela n'existe pas !!

- Problème de coût => Hiérarchie de mémoires
- Le processeur va chercher dans les différentes mémoires
- L'optimisation est faite par le gestionnaire de mémoire



Mémoire cache

Mémoire située sur la carte mère
=> accès direct au processeur

Idée principale du cache: diminuer le temps d'accès à certaines données en les ramenant de la mémoire centrale dans le cache.

Gestion du cache se fait par microprogramme

Mémoire cache

Le préchargeur d'instruction : un petit cache d'anticipation

Un cache externe

- réduction des accès à la mémoire principale à des accès sans états d'attente
- les accès mémoires sont encore limités par la vitesse du bus

Un cache interne unifié instructions /données

- Cache sur la puce du processeur
- Occupation du bus
- Compétition interne instructions/données

Des caches séparés pour les données et les instructions

La mémoire centrale

Elle mémorise les programmes utilisateurs, le système d'exploitation

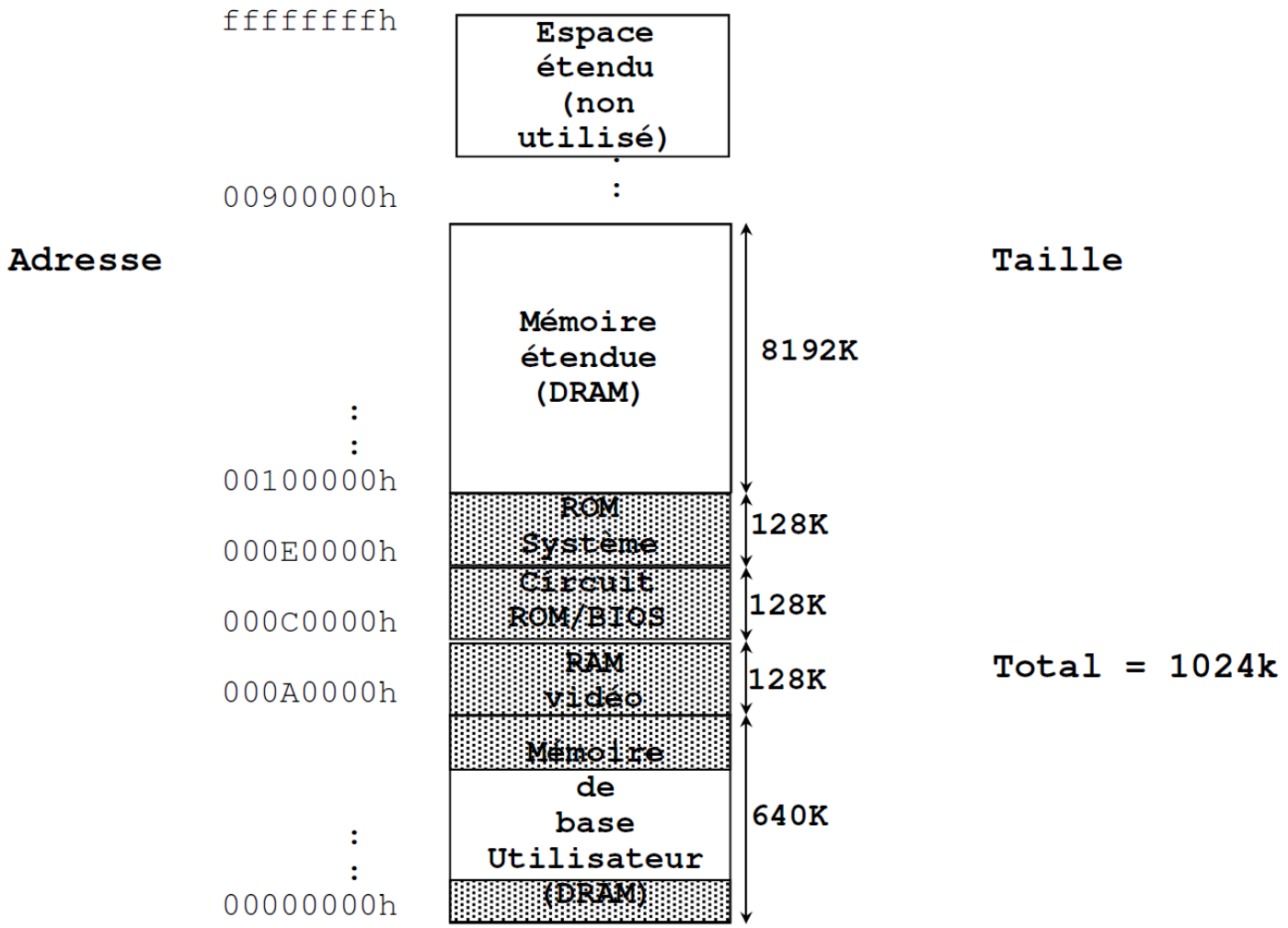
Sa taille

- Aujourd'hui en Go (Gigas) d'un PC (4, 8 16 voire 32 Go)

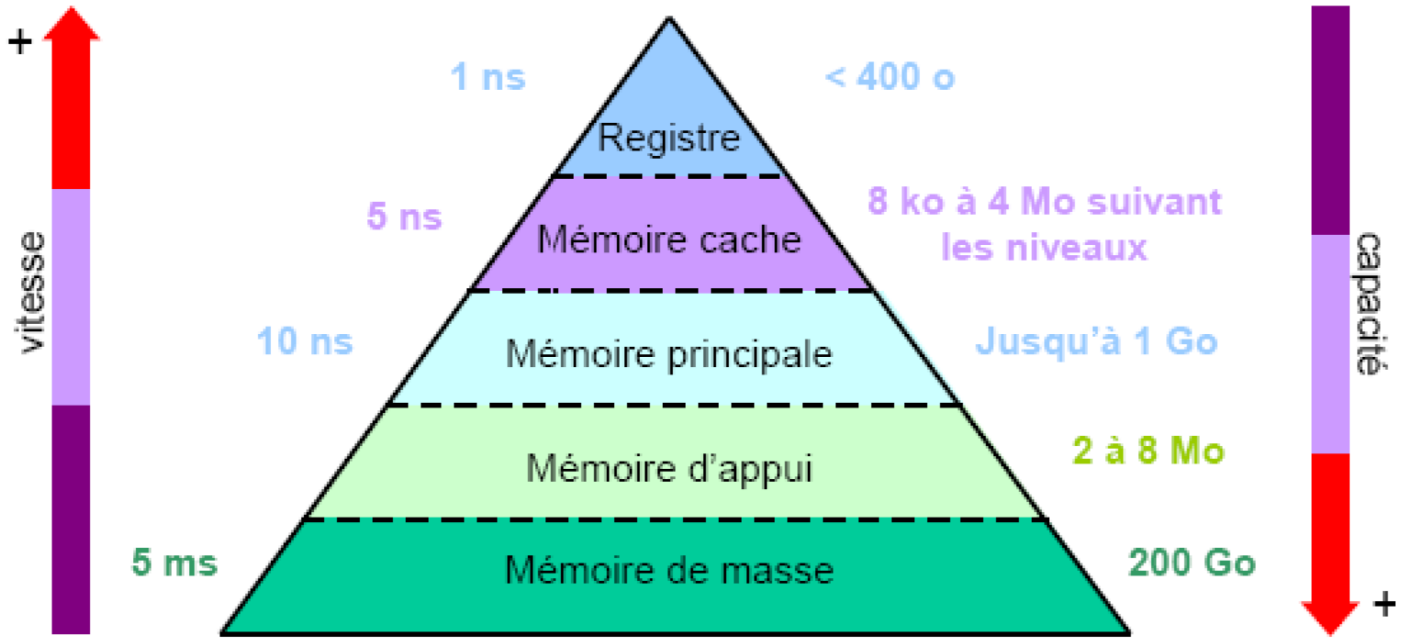
La mémoire centrale est située sur une carte mémoire

Elle communique avec le processeur (carte mère) via des bus (de données, d'adresse)

Exemple configuration mémoire



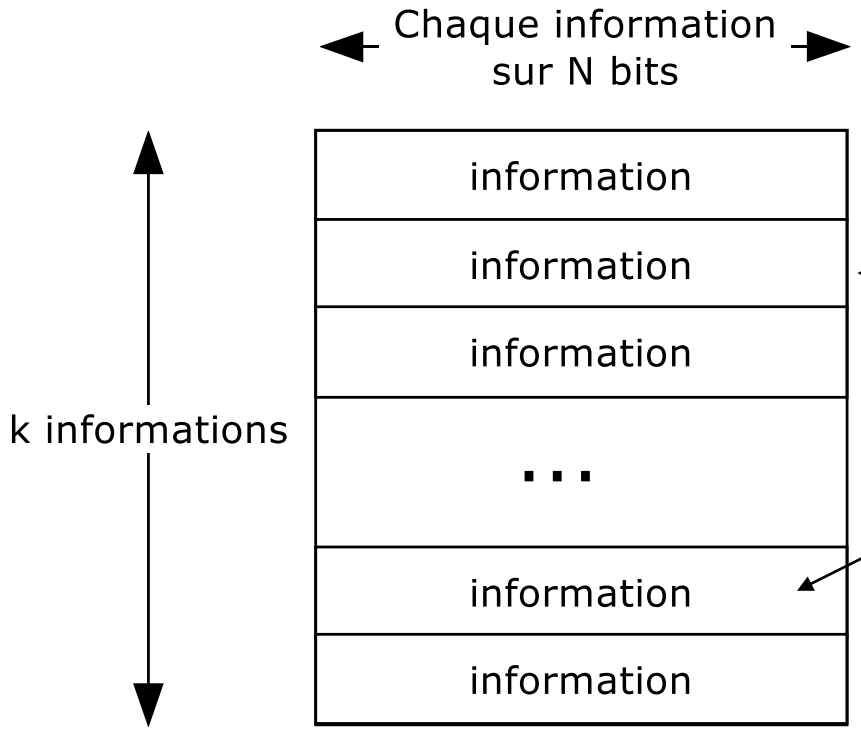
Synthèse sur les mémoires/accès



Mécanisme d'accès à la mémoire

- Les informations sont représentées par des langages binaires (déjà vu)
- On utilise le format fixe parce que c'est plus simple
- Les informations sont donc toutes des mots binaires de N bits
- La taille N de ces mots binaires peut être quelconque actuellement on utilise le plus souvent 64 bits. $2^{64} \approx 18\ 10^{18}$ (18 milliards de milliards) c'est le plus souvent largement suffisant pour les utilisations courantes des ordinateurs
- Lorsque ça ne suffit pas on peut toujours représenter une information sur k fois N bits en la découpant en plusieurs mots mais le traitement sera plus long puisqu'il faudra le faire par parties.

Mémoire



Stocker des informations :

On dispose d'une zone pouvant contenir K informations (zone constituée de k registres)

Chaque information est un mot binaire sur N bits (contenu dans un registre)

Mémoire

- Il faut pouvoir retrouver les informations stockées

- Solutions :
  - Balayer les K informations pour trouver la bonne mais comment savoir quelle est la bonne ?
  - Savoir où elle est parmi les K informations stockées (adresse)

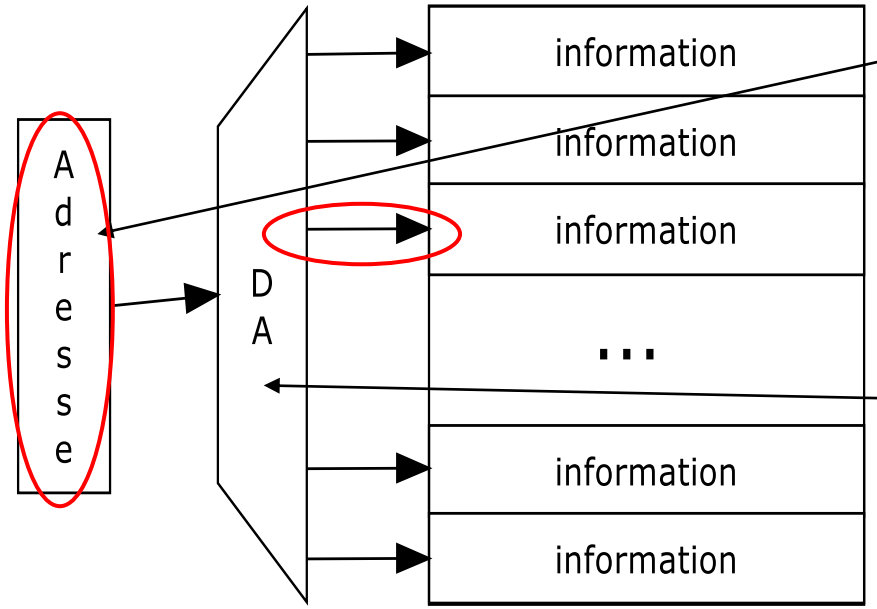
- La première solution est possible (mémoire associative) on cherche une information à partir d'un modèle (par exemple une information qui commence par 0110 et qui finit par 0000) c'est utilisé dans des cas très particuliers.

- Le seconde est plus simple et plus rapide (mémoire adressable) : **c'est celle qui a été retenue**

Mémoire adressable

- Chaque information stockée a une adresse qui correspond à sa position dans la zone de stockage
- Cette adresse est exprimée dans un langage binaire (comme toute information)
- On choisit le format fixe parce que c'est du matériel (plus simple)
- On doit avoir K adresses différentes donc il faut utiliser un langage binaire en format fixe de taille t tel que : $2^t \geq K$
- La taille des adresses dépend donc du nombre d'informations que la mémoire peut stocker et pas de la taille de ces informations.
- Par exemple si la mémoire peut stocker 1000 informations la taille des adresses est 10 ($2^{10} \geq 1000$)
- En général on fabrique des mémoires dont le nombre d'informations est une puissance de 2 pour que toutes les adresses possibles soient utilisées :
  - K (kilo) = $2^{10} \approx 1000$
  - M (méga) = $2^{20} \approx 1$ million
  - G (giga) = $2^{30} \approx 1$ milliard (ordre de grandeur pour les mémoires d'ordinateur)
  - T (téra) = $2^{40} \approx 1000$ milliards (ordre de grandeur pour les disques)

Mémoire adressable



- L'adresse est mise dans un registre (*registre = mémoire ne contenant qu'une seule information*)
- Elle est traitée par un décodeur d'adresses (DA) qui associe cette adresse à la position de l'information dans la zone de stockage (association unique)

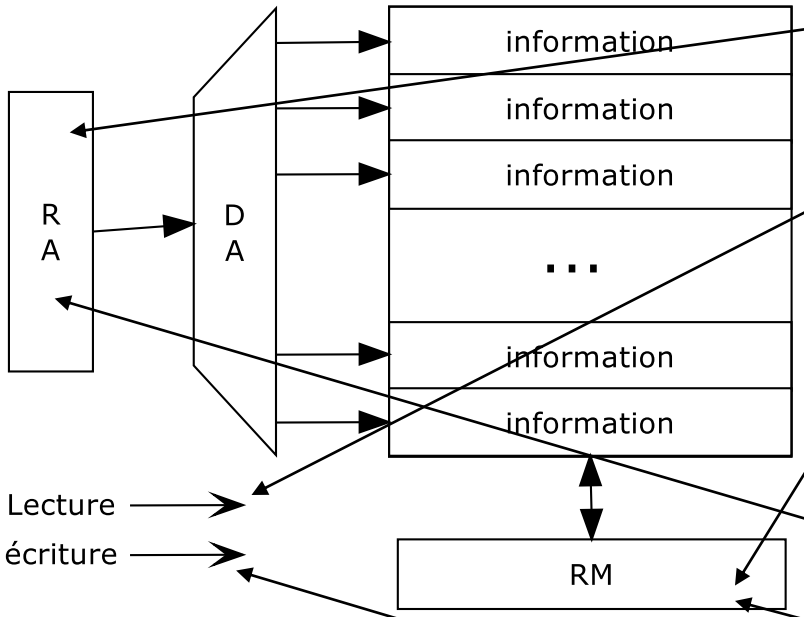
Mémoire adressable

- Il faut pouvoir maintenant :
  - Déposer des informations dans la mémoire (écrire)
  - Récupérer des informations dans la mémoire (lire)

- On a donc besoin :
  - D'un registre où mettre les informations à écrire
  - D'un registre où récupérer les informations lues
  - De commandes pour lire et écrire

  Remarque le registre où mettre les informations à écrire peut être le même que celui où récupérer les informations lues puisqu'on ne fait jamais les 2 opérations en même temps

Mémoire adressable

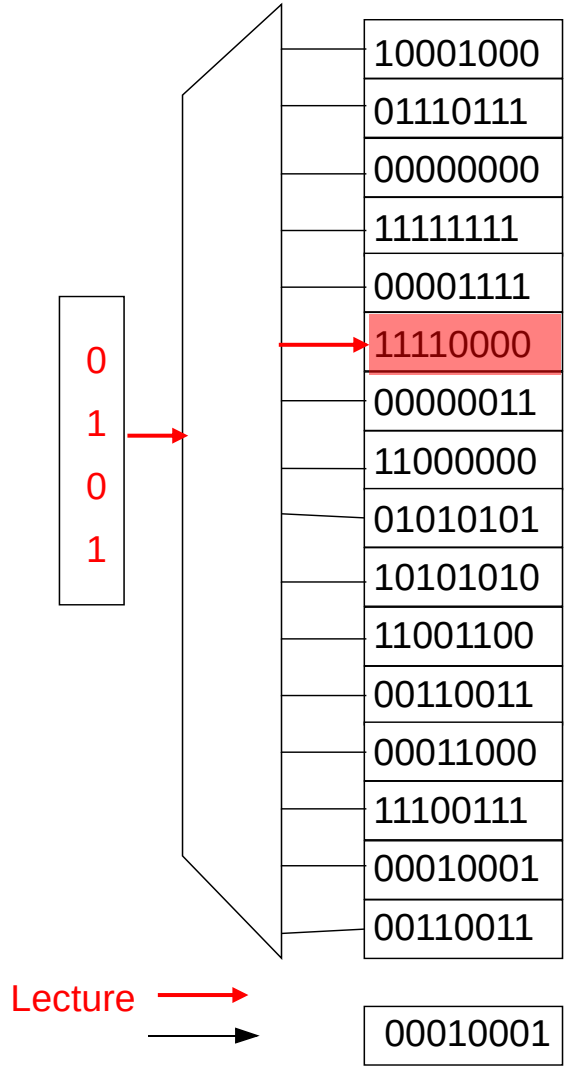


- L'adresse est mise dans le registre RA
- Une commande de lecture est donnée à la mémoire
- L'information lue est récupérée dans le registre RM

OU

- L'adresse est mise dans le registre RA
- L'information à écrire est mise dans le registre RM
- Une commande d'écriture est donnée à la mémoire

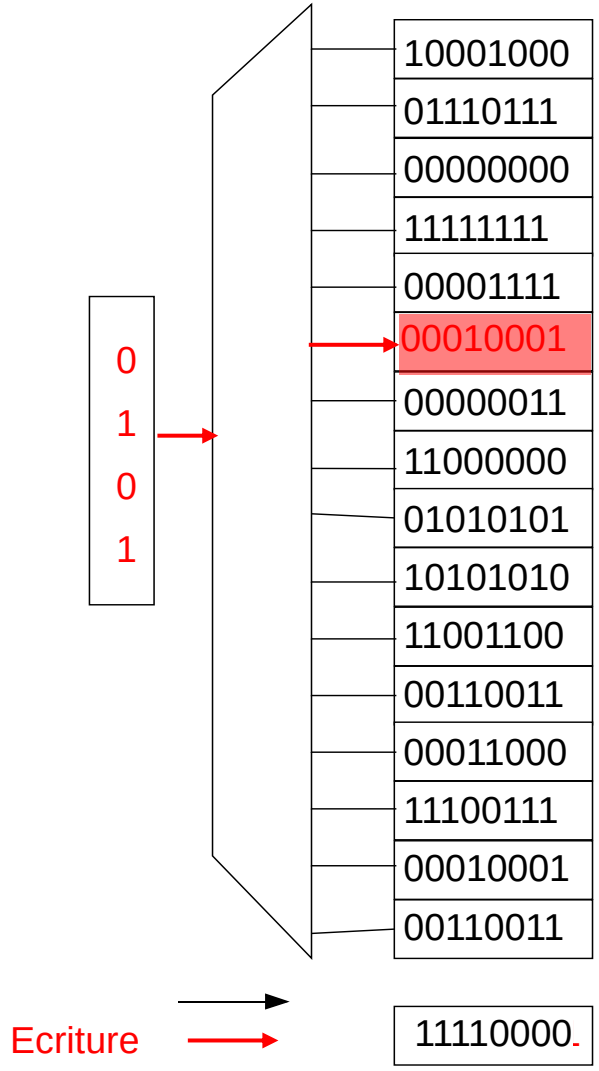
Fonctionnement de la mémoire



Lecture d'une information :

1. Mettre l'adresse de l'information dans RA
2. *Le décodeur d'adresses associe l'adresse à la position de l'information*
3. Donner l'ordre de lecture à la mémoire
4. *La mémoire copie l'information dans RM*
5. Récupérer cette information dans RM

Fonctionnement de la mémoire



Ecriture d'une information :

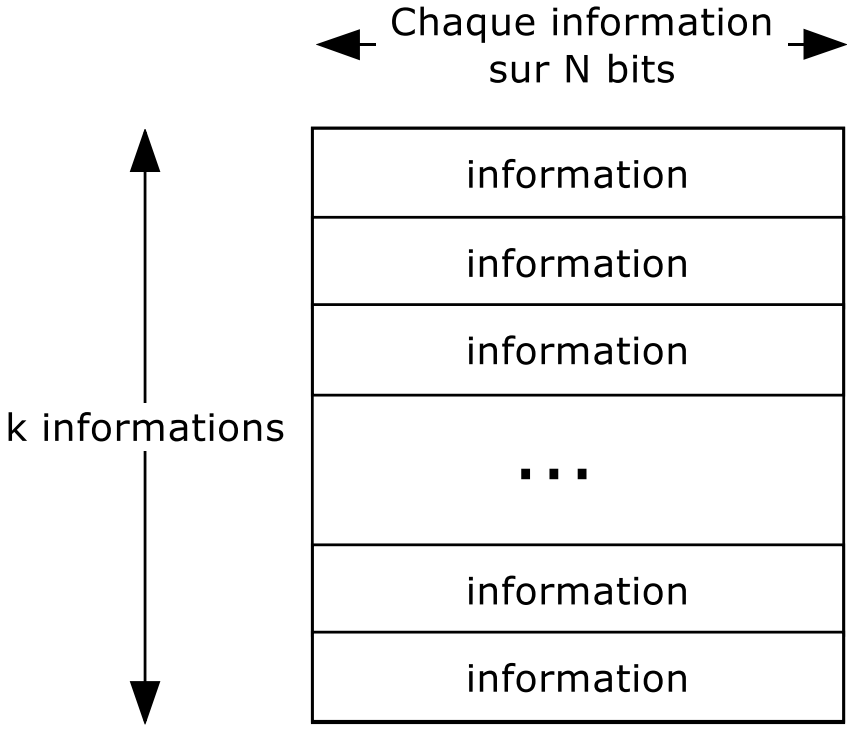
1. Mettre l'adresse de l'information dans RA
2. *Le décodeur d'adresses associe l'adresse à la position de l'information*
3. Mettre l'information à écrire dans RM
4. Donner l'ordre d'écriture à la mémoire
5. *La mémoire remplace l'information stockée à cette position par celle contenue dans RM*

Conséquences de ce fonctionnement

- Chaque information doit être désignée par une adresse unique
  - Nom de variable (toute variable doit avoir un nom)
  - A ce nom (adresse) est associé une information (contenu)
- Quand on lit une information on récupère une copie (l'original est toujours dans la zone de stockage)
  - Quand on utilise le contenu d'une variable sa valeur n'est pas perdue (l'affectation a ← x ne modifie pas x)
- Quand on stocke une information à une adresse celle qui était avant à cette adresse est remplacée
  - Quand on modifie le contenu d'une variable son ancienne valeur est perdue (l'affectation a ← x perd l'ancienne valeur de a)

Caractéristiques d'une mémoire

- Capacité
  - Nombre d'informations qu'elle peut contenir (K) et taille de ces informations (N)
  - L'habitude veut qu'on la mesure en octets (mot de 8 bits) plutôt qu'en bits. C'est donc K*N/8
- Exemple :
  - Une mémoire contenant 2^{30} (1 Giga) informations de 64 bits aura une capacité de 8 Giga octets (1 Giga informations de 8 octets chacune)
  - Sur un PC une mémoire de 8 Go ne contient qu' 1G informations



Caractéristiques d'une mémoire

- Opérations possibles
  - Mémoires ne pouvant qu'être lues (ROM = Read Only Memory) les informations sont stockées à la construction de la mémoire
  - Mémoire ne pouvant qu'être lues mais pouvant écrites une seule fois (PROM = Programmable Read Only Memory) les informations sont stockées une seule fois par un dispositif spécial (programmateur de PROM)
  - Mémoire ne pouvant être que lues mais pouvant écrites plusieurs fois après effacement (EPROM = Erasable Programmable Read Only Memory ou EEPROM = Electrically Erasable Programmable Read Only Memory ou Flash) les informations sont stockées par un dispositif spécial mais on peut les effacer (totalement ou par parties) et recommencer (clés USB ou cartes SD par exemple)
  - Mémoires pouvant être lues et écrites autant de fois qu'on veut (RAM = Random Access Memory) : celles que l'on trouve dans les ordinateurs



Caractéristiques d'une mémoire

- Conservation des informations (volatilité)

  Combien de temps les informations mises dans la mémoire y sont-elles conservées ?

  - Toujours
    - Mémoires de type ROM ou PROM (non effaçables)
    - Mémoires de type Flash (clés USB, cartes SD … effaçables)
    - Mémoires de type EPROM ou EEPROM (effaçables)
  - Tant que la mémoire est sous tension : mémoires de type RAM
  - Très peu de temps (quelques ms) : mémoires de type DRAM qui doivent être réécrites à intervalles réguliers (celles que l'on a dans nos ordinateurs)

Caractéristiques d'une mémoire

- Vitesse (temps d'accès)

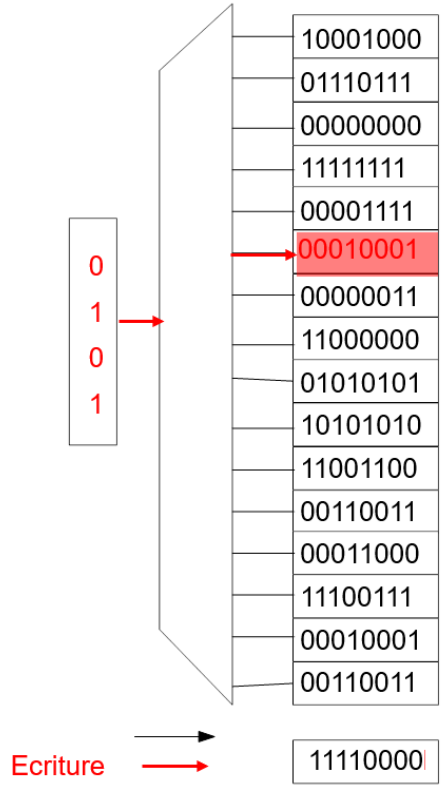
  Ou a vu que les opérations de lecture et d'écriture comportaient 2 étapes effectuées par la mémoire. Par exemple lors d'une lecture :

  - *Le décodeur d'adresses associe l'adresse à la position de l'information*
  - *La mémoire copie l'information dans RM*

  Ces opérations prennent du temps

  - La première opération est bien plus longue que la seconde

  On peut aller plus vite si on accède à des informations se trouvant à des adresses consécutives *(on ne fait la première opération que pour le premier accès, pour les suivants le décodeur d'adresses se contente de sélectionner l'information suivante sans devoir réassocier l'adresse à la position)*



Caractéristiques d'une mémoire

- Prix

  Le prix dépend bien entendu de la capacité

  Mais il dépend aussi beaucoup de la technologie utilisée et en particulier de la vitesse

  Par exemple une mémoire de PC de 8 Go peut coûter entre 30 € et 120 € selon sa vitesse (et la vitesse est loin d'être multipliée par 4)

  La technologie DRAM (DDR4) qui doit être réécrite à intervalles réguliers est bien moins chère à produire et permet une meilleure intégration (grande capacité pour un encombrement et une consommation faible) de plus elle est rapide : c'est pour ça qu'on l'utilise

Mémoires…et antémoire (ou cache)

Mémoire

- On a vu que le temps de la mémoire est surtout lié au décodage d'adresse
- Plus une mémoire est grande, plus l'adresse est grande, plus il faut de temps
- A la limite une mémoire ne contenant qu'une information (registre) n'a pas d'adresse et est très rapide
- On sait fabriquer des mémoires de petite taille (quelques dizaines de Ko) très rapides
- Mais :
  - Elles sont de petite taille
  - Elles coûtent beaucoup plus cher que les mémoires classiques (DRAM)

Mémoire

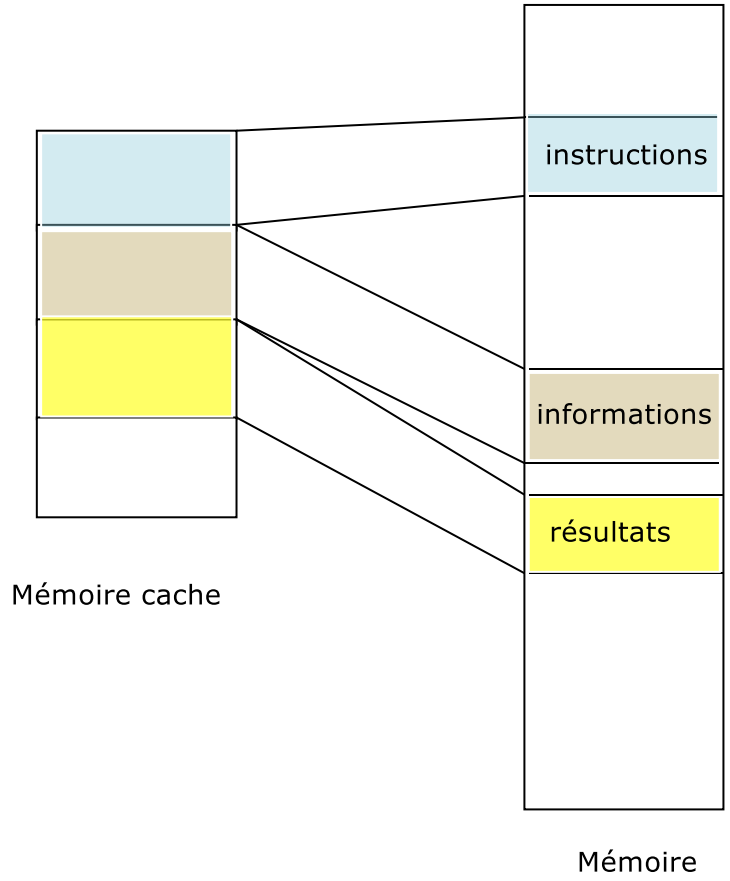
- Quand un programme s'exécute il faut :
  - Aller chercher
    - les instructions
    - les informations à traiter
  - Ranger les résultats obtenus
- Mais :
  - Les instructions sont à des adresses aléatoires (mais aussi consécutives)
  - Les informations traitées par un programme peuvent être placées dans la mémoire dans des adresses aléatoires (mais aussi consécutives)
- Donc :
  - On n'utilise pendant un temps que de toutes petites parties de la mémoire
    - celles où sont les instructions en cours
    - celles où sont les informations à traiter
    - celles où l'on range les résultats

Mémoire

- Idée :
  - Puisqu'on n'a besoin que de quelques parties de la mémoire à un moment donné on pourrait utiliser une mémoire plus petite et plus rapide.
  - Il suffit de copier au préalable dans cette mémoire les parties dont on a besoin
  - On perd du temps à copier ces parties mais on en gagne ensuite en accédant à ces informations de façon plus rapide
  - Si le temps perdu est faible devant le temps gagné ça vaut le coup

- On parle d'antémémoire ou de mémoire cache

Mémoire cache



- Les informations utiles à un moment donné sont copiées dans la mémoire cache
- Les unités accèdent à cette mémoire plus rapidement
- Il faut que les copies ne prennent pas trop de temps

Mémoire cache

- Taille :
  - Petite pour être rapide
  - Mais si elle est petite on n'aura pas souvent les bonnes informations $\Rightarrow$ il faudra faire une autre copie $\Rightarrow$ on perd du temps
- Quand l'information nécessaire n'est pas dans le cache :
  - Il faut l'y copier
  - Mais il faut trouver une place où la mettre dans le cache et si le cache est plein ?
  - Il faut enlever des informations qu'il contient pour mettre les nouvelles

Gestion du cache

- Au départ le cache ne contient rien
- On y met les informations dont on a besoin
- Au bout d'un moment (assez vite puisqu'il est petit) le cache est plein
- Il faut remplacer des informations par les nouvelles
- Mais si ces informations ont été modifiées par le programme il faut d'abord les **recopier en mémoire** pour ne pas les perdre
- Donc un remplacement c'est
  1. Choisir la zone à libérer
  2. Recopier cette zone du cache vers la mémoire
  3. Copier les nouvelles informations de la mémoire vers le cache

Choisir la zone à libérer

- Il vaut mieux choisir une zone dont on n'aura plus besoin oui mais comment le savoir ?
- Least Frequently Used :
  - On prend la zone la moins souvent utilisée puisque c'est celle qui risque moins de nous manquer si on ne l'a plus
- Least Recently Used :
  - On prend la zone qui n'a pas été utilisée depuis le plus longtemps puisque c'est peut être une zone qui ne sert plus et donc qui ne nous manquera pas

- Quelle est la bonne solution ?
  - La première constate le présent (on s'en sert peu)
  - La seconde mise sur le futur (on ne s'en servira peut-être plus)
- Pas de réponse théorique mais on peut essayer les 2
- C'est ce qu'on a fait et les 2 se valent.

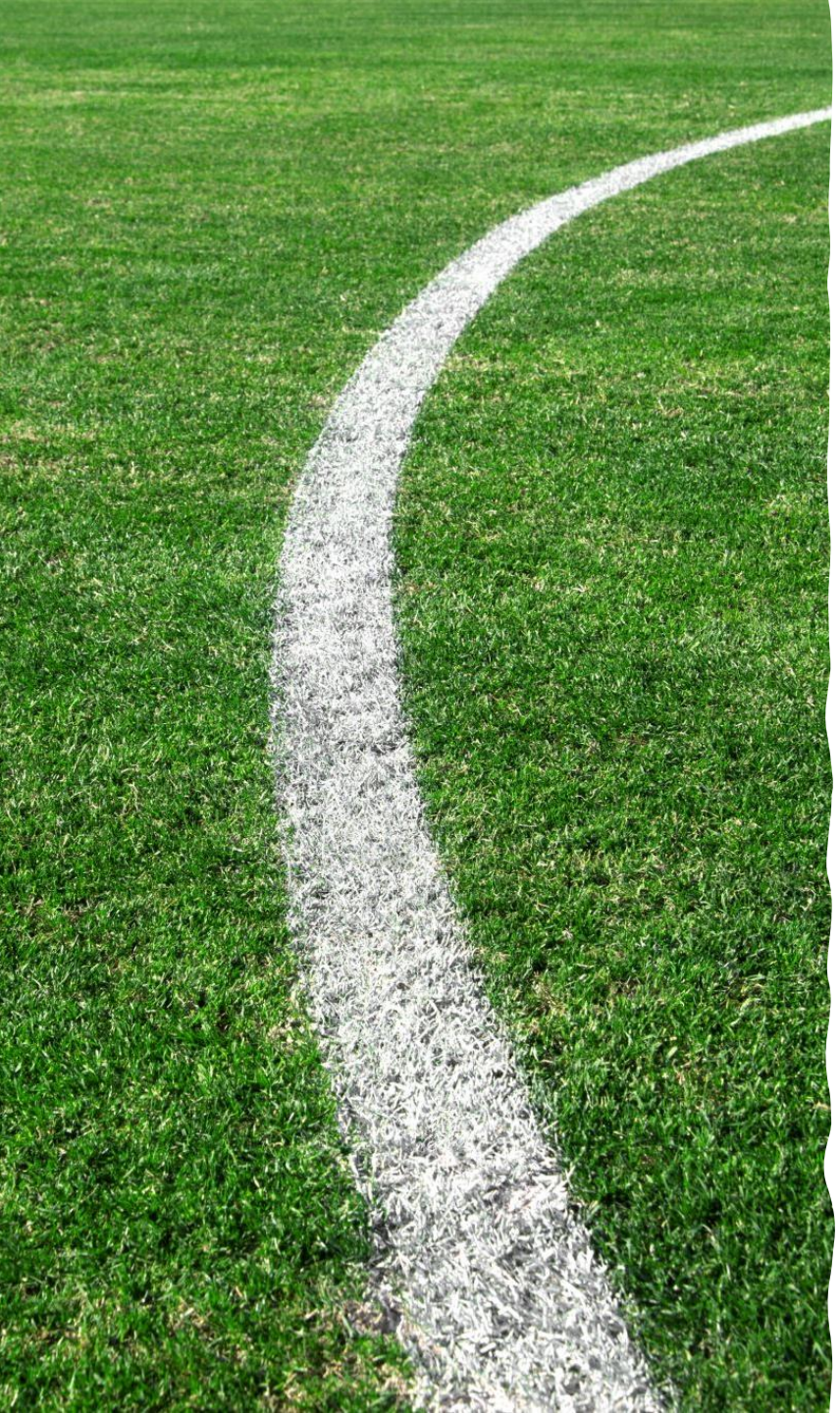
Transférer les informations

- Les copies entre mémoire et cache (dans un sens comme dans l'autre) se font sur des blocs d'informations placées à des adresses consécutives

- On a vu que le décodeur d'adresse prend du temps pour associer l'adresse à la position de l'information mais que si ensuite on passe à l'information suivante ça vite.

- Les copies peuvent donc se faire à des vitesse élevées (plusieurs Go/s)
- Comme on ne copie que des petits blocs (quelques centaines d'octets) ça va vite

- De plus si un bloc n'a pas été modifié on peut éviter la copie cache $\rightarrow$ mémoire (c'est le cas pour les blocs contenant des instructions mais aussi parfois pour des blocs contenant des informations par exemple un tableau dans lequel on a cherché une valeur sans rien modifier)



Faire plus

- L'idéal serait d'avoir un cache grand (pour éviter de faire trop de remplacements) et très rapide
- On peut le faire mais c'est très cher

- La solution retenue c'est d'utiliser des **caches de caches** :
  - Un cache de niveau 1 auquel accèdent les unités petit mais très rapide
  - Un cache de niveau 2 de taille moyenne mais moins rapide
  - Un cache de niveau 3 plus grand et encore moins rapide mais quand même plus rapide que la mémoire elle même

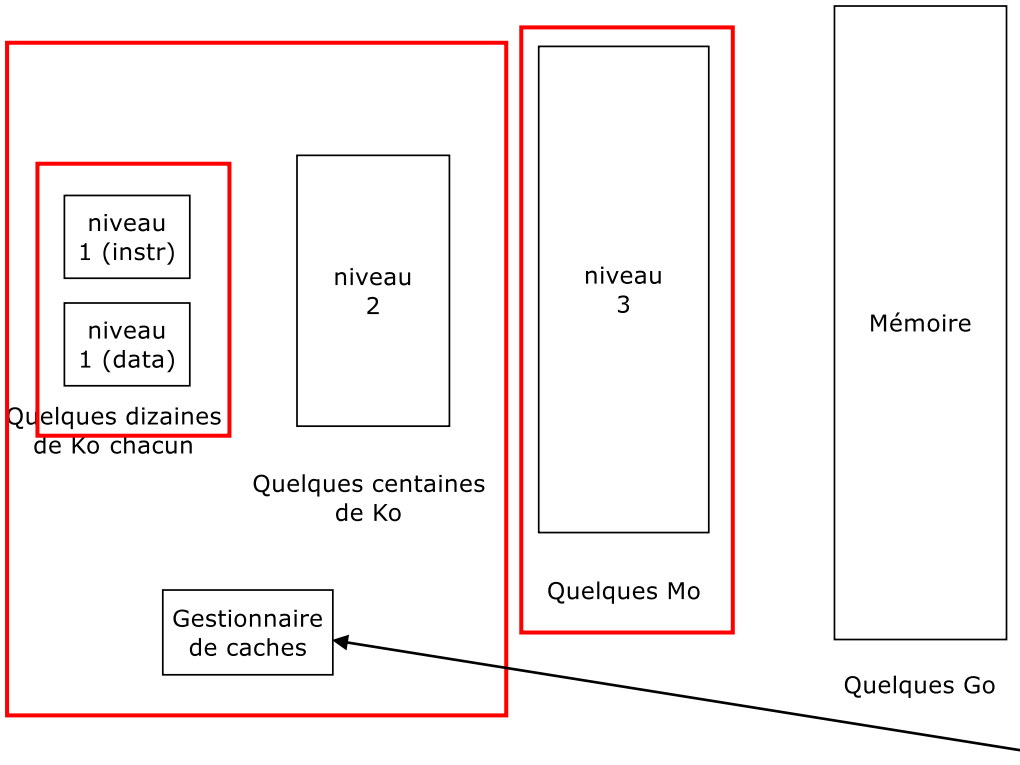
Niveaux de cache

- Quand une information n’est pas disponible dans le cache de niveau 1 il est possible qu’elle le soit dans celui de niveau 2 qui est plus grand. La copie sera très rapide car le cache de niveau 2 est rapide

- Si elle n’est pas dans celui de niveau 2 elle est peut-être dans celui de niveau 3 qui est encore plus grand. La copie sera rapide car le cache de niveau 2 est plus rapide que la mémoire

- Sinon tant pis on ira la chercher dans la mémoire mais assez vite puisqu’on accède à des adresses consécutives.

Caches



Au niveau 1 on utilise 2 caches : 1 pour les instructions et 1 pour le reste. Le 1er n'a jamais besoin d'être copié en mémoire lors d'un remplacement

Les caches de niveau 1 et 2 et le gestionnaire sont inclus dans la même puce que l'UT et l'UC (microprocesseur)

Le cache de niveau 3 peut l'être aussi ou être externe

Le gestionnaire de cache applique la politique de remplacement et provoque les copies

Unité de Contrôle

Unité de contrôle

Rôle :

- Piloter les autres unités pour aboutir au résultat attendu en fonction des informations relatives au problème à résoudre

- Doit être autonome car on veut une machine automatique de traitement d'informations

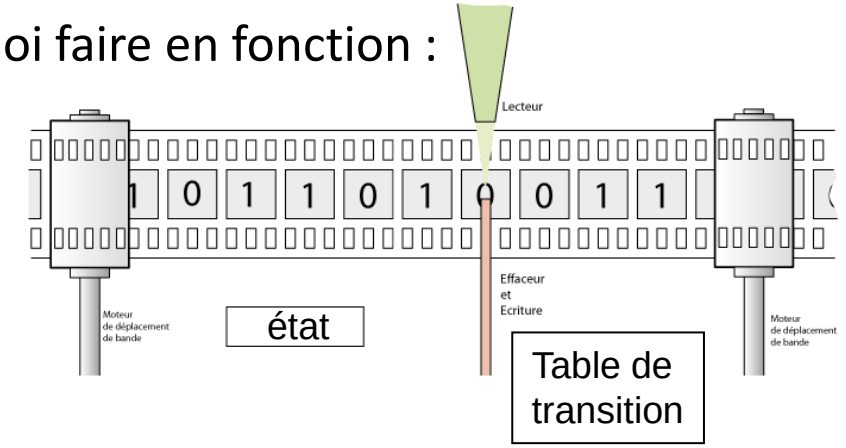
- Elle dispose :
  - De la mémoire pour stocker/retrouver les informations
  - De l'unité de traitement pour traiter les informations
  - De l'unité d'entrée/sortie pour collecter les informations relatives au problème à résoudre et pour présenter les résultats obtenus

Retour sur la machine de Turing

- La machine de Turing utilise une bande qui sert :
  - De mémoire (la machine lit des informations sur la bande et en écrit)
  - D'unité d'entrée sortie (on écrit le problème à résoudre sur la bande et on lit le résultat sur la bande)

- Elle sait effectuer des traitements :
  - Ecrire sur la bande
  - Décaler la bande d'un cran à droite ou à gauche
  - S'arrêter

- Elle possède une table de transition qui lui dit quoi faire en fonction :
  - De son état actuel
  - Et de ce qu'elle lit sur la bande



Retour sur la machine de Turing

- Au départ la machine est dans un état initial (toujours le même)

- A chaque étape (indéfiniment ou jusqu'à ce qu'elle s'arrête) :

  - Elle lit ce qu'il y a sur la bande (là où se trouve le tête de lecture/écriture)
  - Selon son état actuel et ce qu'elle a lu :
    - Elle écrit quelque chose sur la bande là où se trouve la tête de lecture/écriture (éventuellement la même chose)
    - Elle effectue un traitement (décaler la bande ou s'arrêter)
    - Elle change d'état

Retour sur la machine de Turing

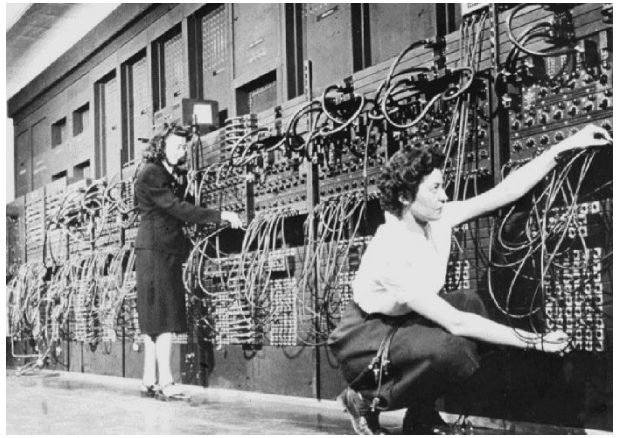
- La machine de Turing est construite pour résoudre un type problème et un seul puisque sa table définit son travail (c'est son programme)

- On fabrique une machine avec une table donnée et elle ne sait faire que ça !

- On peut fabriquer des ordinateurs qui ne savent faire qu'une seule chose. Par exemple celui qui gère l'ABS dans une voiture ne sait rien faire d'autre que doser le freinage en fonction de la vitesse des roues de la voiture.

- Mais on veut avoir une machine universelle

Von Neumann et Turing



Dialogue imaginaire dont j'assume la responsabilité

- Von Neumann : « l'ordinateur doit être une machine universelle capable de traiter automatiquement des informations »

- Turing : « ma machine peut être universelle il suffit d'écrire la table de transition sur la bande au lieu de la figer à la construction. En changeant la table écrite elle fera autre chose »

- Von Neumann : « donc il suffit que l'équivalent de la table (c'est-à-dire le programme) soit mis dans la mémoire pour que la machine soit universelle »

- Remarque : l'ENIAC n'était pas une machine universelle car son programme était représenté par des fils qu'il fallait rebrancher à chaque programme.

Unité de contrôle

- L'unité de contrôle trouve « la table » (le **programme**) dans la mémoire.

- A chaque étape (indéfiniment ou jusqu'à ce qu'on l'éteigne *ou qu'elle tombe en panne*):

    - Elle va chercher une ligne de cette table (une instruction du programme)
    - Elle donne des ordres aux autres unités et à la mémoire pour exécuter cette instruction
    - Elle passe à l'instruction suivante

Instruction

- Dans la machine de Turing une instruction (une ligne de la table) est de la forme :
  - Etat actuel
  - Caractère lu
  - Caractère à écrire
  - Traitement à faire
  - Etat suivant

- Les 2 premiers éléments permettent de trouver dans la table la ligne correspondant à l'état actuel et au caractère lu (trouver l'instruction à exécuter)

- Les 3 suivants permettent d'écrire le caractère, d'effectuer le traitement et de changer l'état actuel (exécuter l'instruction)

Instruction

- Dans une machine de Von Neumann une instruction doit indiquer quel traitement doit être effectué sur quelle(s) information(s)

- On doit donc y trouver :
  - L'opération à faire (qui doit réalisable par la mémoire et/ou par l'unité de traitement et/ou par l'unité d'entrée/sortie)
  - La ou les informations à traiter

- Et tout ça doit se trouver dans la mémoire donc être représenté par un mot binaire

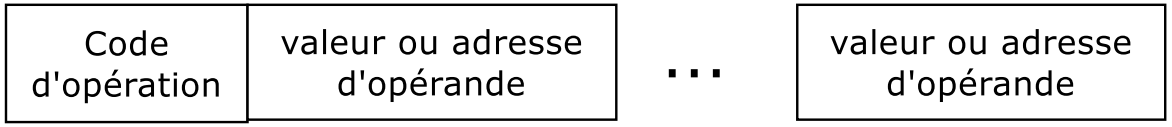
- Il restera à résoudre le problème de trouver l'instruction à exécuter

Instruction (contenu)

- Désignation de l'opération à faire :
  - On dispose d'un nombre fini d'opérations possibles (celles de la mémoire + celles de l'UT + celles de l'UE/S). On peut combiner ces opérations (par exemple additionner puis stocker le résultat).
  - Le constructeur définit un nombre fini d'instructions possibles.
  - Il peut donc définir un langage binaire (arbitraire) en format fixe ou variable pour représenter chacune d'entre elles en binaire

- Désignation des informations à traiter
  - Soit il s'agit d'une valeur fixe (par exemple ajouter 10) et on peut la représenter en binaire de façon classique (entier naturel, entier relatif en complément à 2, code UTF …)
  - Soit il s'agit d'une information stockée et elle est désignée par son adresse dans la mémoire

Format d'instruction




```
.model small

.stack 100h

.data
   bonjour  db  "Hello world!$"

.code
   main  proc
       mov  AX,@data
       mov  DS, AX
       mov  DX, offset bonjour
       mov  AH,09h
       int  21h
       mov  AX,4C00h
       int  21h
   main endp
end main
```

- Combien d'opérandes ?
  - Dépend de l'opération
  - En général on se limite à 2 (exceptionnellement 3)

- Taille du mot binaire
  - La taille du code d'opération dépend du nombre d'opérations définies (en général de l'ordre de quelques centaines)
  - La taille des opérandes dépend de celle des valeurs ou des adresses

- Si la taille peut être limitée à 64 bits l'instruction tient dans un seul mot de la mémoire sinon on la découpe en N mots (c'est plus lent puisqu'il faut faire N lectures dans la mémoire pour récupérer une instruction)

- En général la plupart des instructions contiennent sur un seul mot et quelques unes occupent plusieurs mots

Trouver l’instruction à faire

- L’unité de contrôle va chercher l’instruction à faire en mémoire elle doit donc connaître son **adresse**
- Au démarrage de l’ordinateur on prend une **adresse fixe** toujours la même
- Il faut trouver une instruction à cette adresse
- Mais la mémoire est de type RAM donc perd tout quand on l’éteint
- Il faut donc une mémoire de type ROM ou flash où placer la première instruction
- C’est le rôle du **BIOS** (mémoire de type flash contenant le programme de démarrage)



Bootstrap

- Un ordinateur doit posséder une mémoire non volatile contenant son programme de démarrage appelé bootstrap.

- Basic Input Output System (BIOS) :
  - Teste le matériel (mémoire, périphériques utiles)
  - Cherche un support (clé USB, disque, CD ROM ou réseau) où trouver le système d'exploitation
  - Copie le noyau du SE en mémoire
  - Exécute ce programme

- A partir de là c'est le SE qui est exécuté, il commence par s'auto compléter (services, pilotes de périphériques …)

- Puis le SE permet, via une interface le plus souvent graphique, de mettre dans la mémoire d'autres programmes (applications) et de les exécuter

Exécution des instructions

- Quand l'unité de contrôle a récupéré l'instruction à faire elle va :
  - **Décoder** le code d'opération pour savoir quel traitement faire
  - **Envoyer** les adresses des opérandes à la mémoire pour récupérer les informations à traiter
  - **Envoyer** ces informations à l'UT ou à l'UE/S
- Il ne lui reste plus qu'à passer à l'instruction suivante

Trouver l'instruction suivante

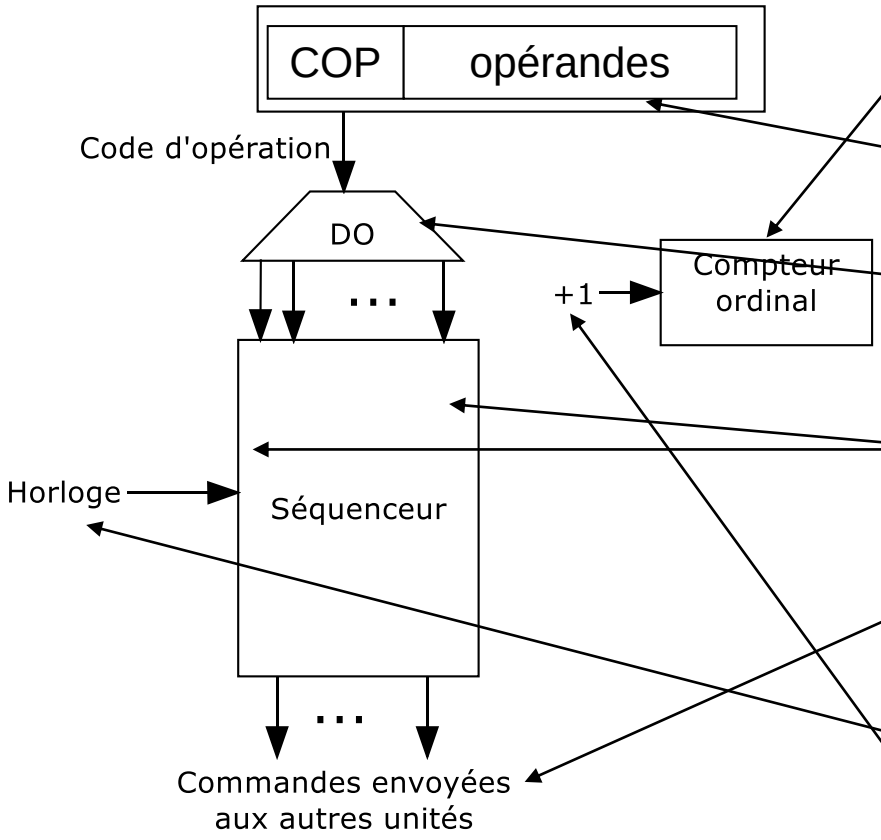
- La machine de Turing choisit l'instruction suivante selon son état et le caractère lu (mémoire associative).
- La machine de Von Neumann trouve l'instruction suivante par son adresse (mémoire adressable).

- 2 solutions :
  - Chaque instruction contient l'adresse de la suivante
  - On choisit de mettre les instructions dans la mémoire à des adresses consécutives et il suffit de passer à l'adresse suivante

- C'est la deuxième solution qui a été choisie
- Conséquence :
  - Vous devez écrire les instructions de vos programmes dans l'ordre
  - Notion de séquence (vue en algo)

Unité de contrôle

Unité de contrôle



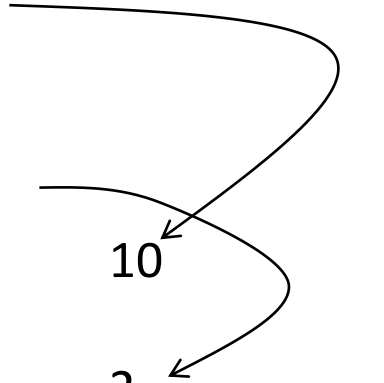
- Le compteur ordinal contient l'adresse de l'instruction
- Avec cette adresse on lit en mémoire l'instruction et on la met dans le registre d'instructions
- Le code d'opération (COP) est décodé par le décodeur d'opérations (DO)
- Chaque opération déclenche la séquence prévue dans le séquenceur qui lui correspond
- Ces séquences consistent à envoyer des ordres aux autres unités
- Le rythme auquel sont envoyés ces ordres est donné par une horloge
- A la fin le compteur ordinal est incrémenté et on recommence

Traduction d'une alternative

```
a ← 4
Si (a<b)

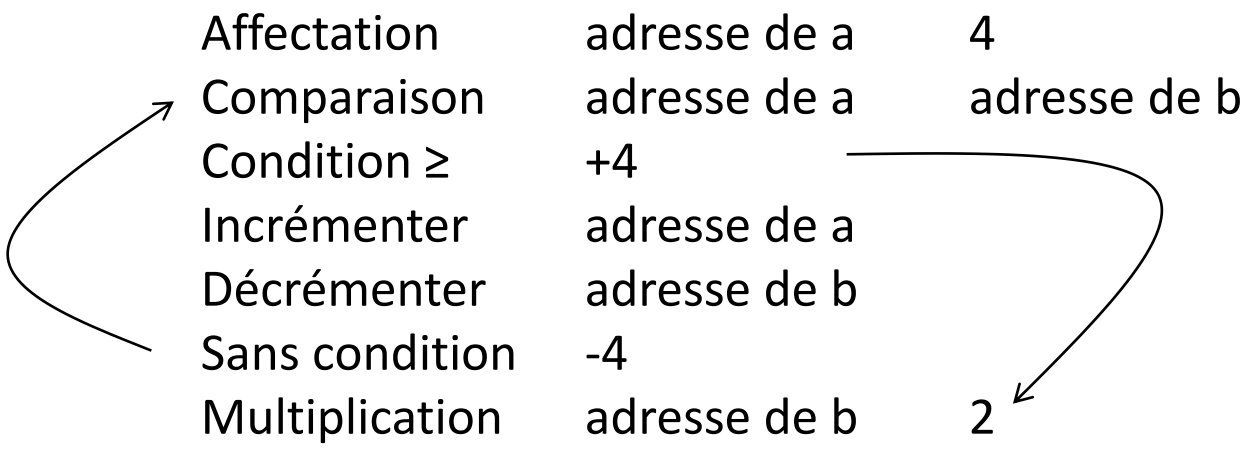
   a ← a+1
   b ← b-1
Sinon
   a ← 10
Fsi
b ← 2*b
```

Affectation	adresse de a	4
Comparaison	adresse de a	adresse de b
Condition ≥	+4	
Incrémenter	adresse de a	
Décrémenter	adresse de b	
Sans condition	+2	
Affectation	adresse de a	10
Multiplication	adresse de b	2



Traduction d’une boucle

```
a ← 4
Tant que (a<b)
    a ← a+1
    b ← b-1
Fin tant que
b ← 2*b
```



Séquenceur de l'UC

- A chaque code d'opération correspond une suite fixe d'ordres à envoyer aux autres unités (microprogramme).

- Cette suite est définie par le constructeur. Elle peut être plus ou moins longue selon l'opération à faire

- Par exemple pour une addition :
  1. Envoyer l'adresse du 1er opérande à la mémoire
  2. Lui donner un ordre de lecture
  3. Récupérer l'information dans le registre de mémoire et l'envoyer dans le registre d'opérande 1 de l'UT
  4. Refaire la même chose pour l'opérande 2
  5. Donner l'ordre d'addition à l'UT
  6. Récupérer le résultat dans le registre de résultats de l'UT

Séquenceur de l'UC

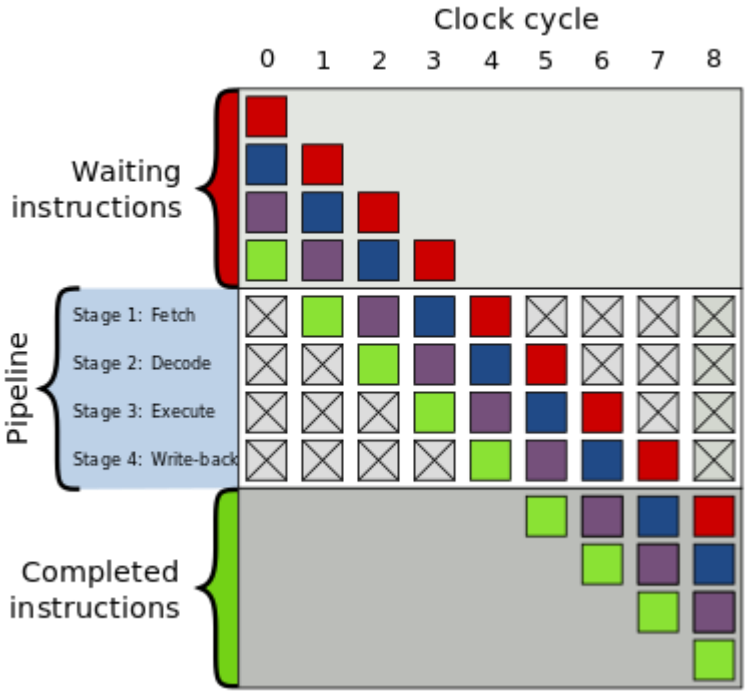
- Ces opérations doivent se faire dans l'ordre

- Certaines peuvent se faire en même temps par exemple on peut faire :
  - Récupérer l'information dans le registre de mémoire et l'envoyer dans le registre d'opérande 1 de l'UT en même temps que envoyer l'adresse du 2ème opérande à la mémoire

- Il faut respecter le temps par exemple on ne peut pas récupérer le résultat de l'addition avant que l'UT ait fini l'addition ni récupérer l'opérande avant que la mémoire ait fini l'opération de lecture

- L'horloge est là pour ça. Plus elle va vite plus l'ordinateur va vite mais elle ne peut pas aller plus vite que ce que supportent les unités et la mémoire (overclocking)

- De plus la consommation électrique des unités augmente comme le carré de la vitesse ⇒ ça chauffe vite !!!



RISC ou CISC

- Au début les UC étaient simples et le nombre d'instructions réduit parce qu'on ne pas savait faire mieux
- Puis on a augmenté le nombre d'instructions (Complex Instruction Set Computer)

- Mais on s'est aperçu que beaucoup d'instructions ne servaient quasiment jamais (compilateurs)

- Alors on a réduit le nombre d'instructions (Reduced Instruction Set Computer) et on met la complexité de l'électronique au service de la vitesse

Vitesse

- Sur les 1ers microprocesseurs (1971) l'horloge du séquenceur fonctionnait à 1 MHz (millions de hertz) et il fallait 2 à 4 cycles pour faire une instruction (on faisait donc environ 300 000 instructions/s)

- Maintenant les horloges sont entre 3 et 5 GHz (3 à 5000 fois plus) et on met souvent moins d'un cycle pour faire une instruction (on verra comment on fait ça plus tard)

- On arrive donc maintenant à faire jusqu'à 30 milliards d'instructions/s (on a donc multiplié la vitesse par 100 000 dont 5000 dus à la vitesse brute de l'horloge)

- Malgré tout certains traitements (éclairage d'une scène 3D par exemple) demandent d'énormes quantités d'opérations.

Instructions +

- On a vu que dans une instruction on désignait les opérandes soit par leur valeur soit par leur adresse

- En réalité on peut les désigner soit par leur valeur soit par n'importe quoi qui permette d'obtenir leur adresse
- Par exemple on peut donner une adresse qui désigne un mot dans la mémoire qui contient l'adresse de l'opérande (indirect)

- Les constructeurs proposent de nombreuses façons d'obtenir les adresses des opérandes ceci permet :
  - D'adapter les instructions aux structures de données utilisées dans les langages (tableaux, pointeurs, …)
  - De mettre en place des mécanismes de sécurité pour les systèmes d'exploitation (protéger certaines adresses par exemple)

Architecture de Von Neumann

Récapitulons :

- Seulement 4 unités (UC, UT, UE/S et mémoire)
- Tout est dans la mémoire (informations et instructions)
- L'UC enchaîne automatiquement les instructions en séquence (compteur ordinal)
- Des instructions spéciales permettent les ruptures de séquence (boucle, alternatives)
- L'UT est capable de faire au moins les opérations de la machine de Turing

Donc l'ordinateur est une machine automatique de traitement d'informations universelle

- Tout ce qui est calculable peut l'être par un ordinateur mais c'est limité par :
  - Le temps de calcul
  - La taille de la mémoire
  - La capacité d'écrire le programme c'est-à-dire de savoir comment arriver au résultat à partir des informations de départ. En particulier lié au fait que l'on n'a pas représenté le domaine des informations

Conséquence

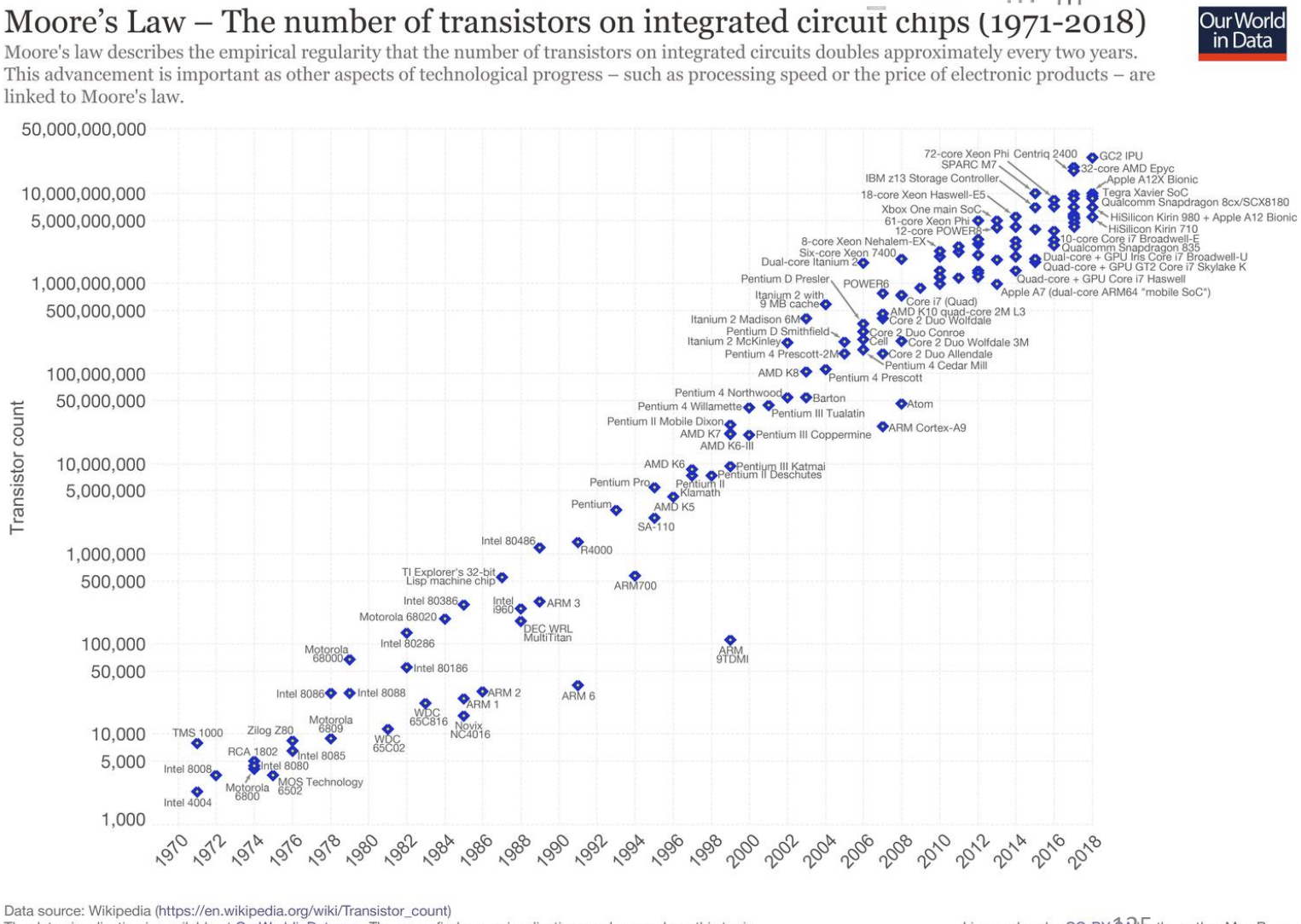
- Les instructions du programme **sont des informations comme les autres** qui se trouvent dans la mémoire avec les autres
- L'ordinateur peut donc **traiter les instructions comme des informations**
- Il peut donc exécuter un programme qui :
  - Modifie ses propres instructions (virus)
  - Fabrique de nouvelles instructions qui seront exécutées :
    - Par le même ordinateur (compilateur)
    - Par un autre (compilation croisée)

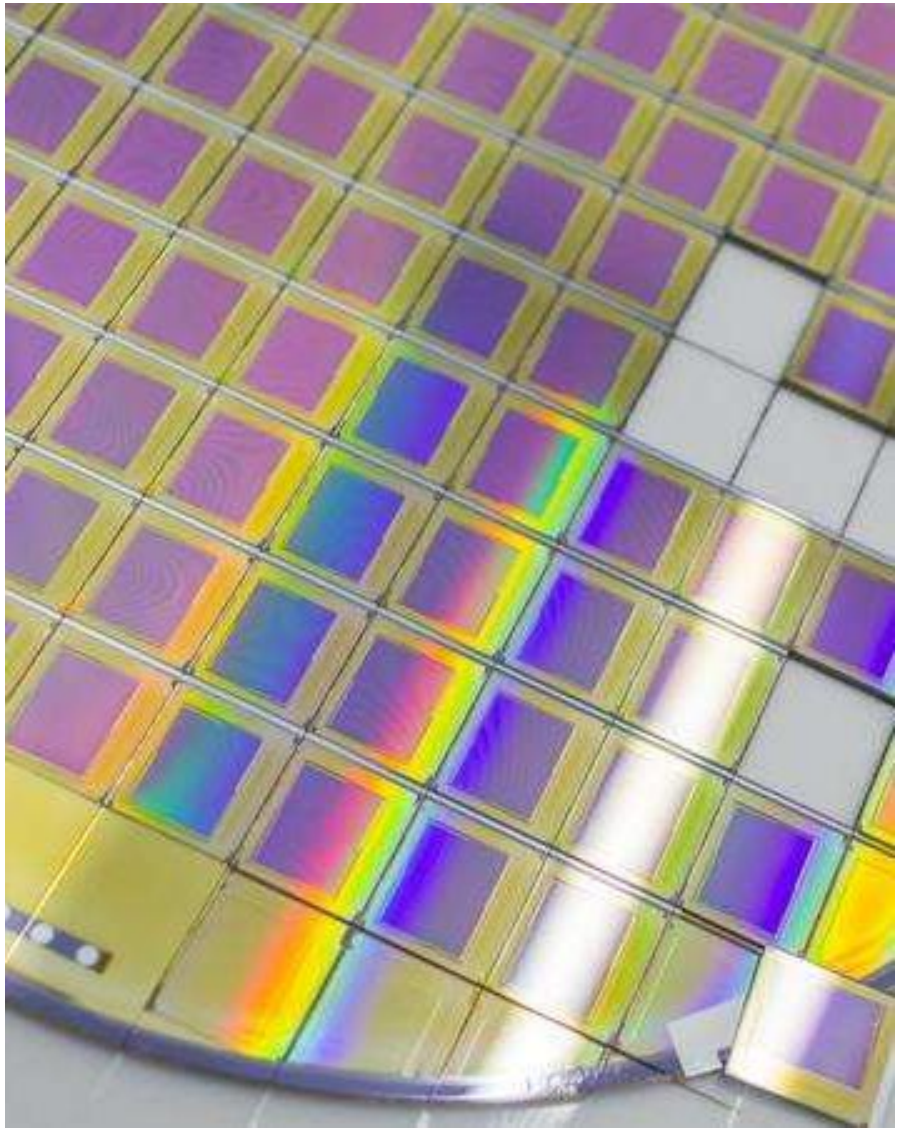
Exécution d’un programme

Puce…



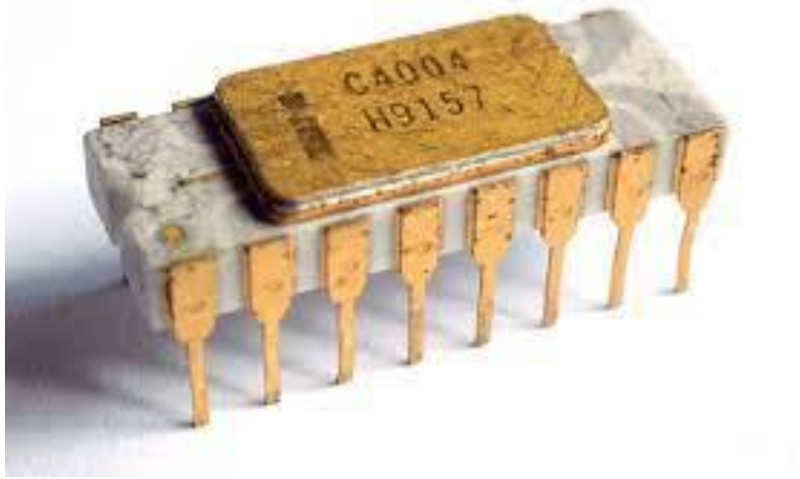
- Une puce est un carré de silicium constitué de millions de transistors
- L'intégration des transistors sur une puce suit la loi de Moore
  - Auj: 50 milliards/processeur





Fabrication

- Du Silicium (SI) …du sable… après l'oxygène, le constituant le plus massif et le plus répandu sur Terre.
- Un barreau de Silicium fabriqué dans un four à très haute température
- Un Wafer (disque de Silicium) 30 cm de diamètre
- Silicium est un semi-conducteur (certaines parties peuvent être plus ou moins conductrices)
- Dépôt de couches de substrat
    - dépôt d'un produit photosensible , zones imprimées par ultraviolet, acide pour le décapage des zones, dépôt d'impuretés pour créer la connectivité
- Création de transistors interconnectés => plusieurs puces sur un wafer
- Découpage de la puce et tests
- Mise en boîtier et connexion des pattes du support avec les points de contacts de la puce devient un le microprocesseur
    - > densité de plusieurs centaines de millions de transistors/mm2

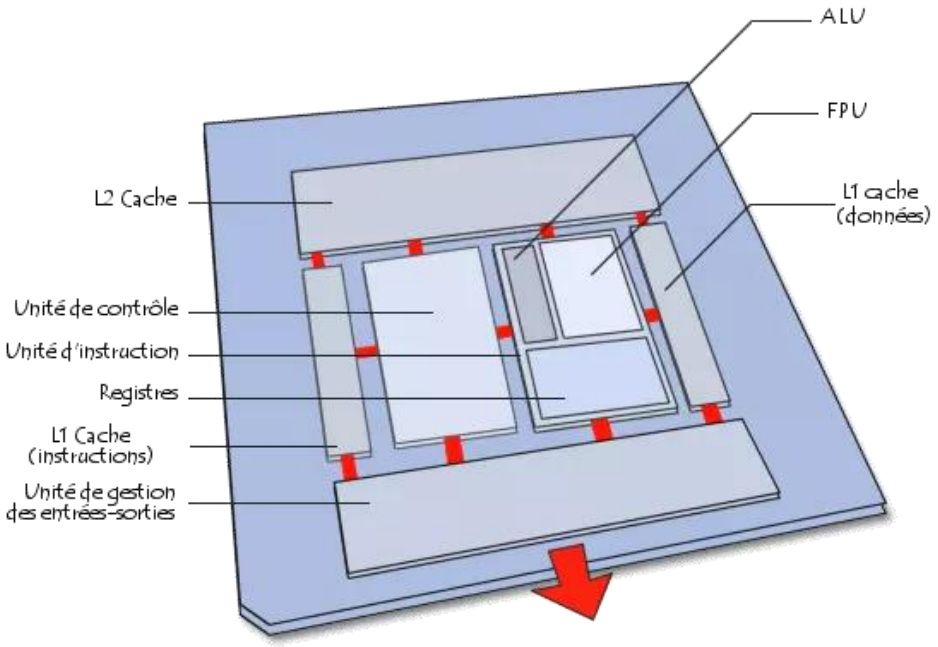


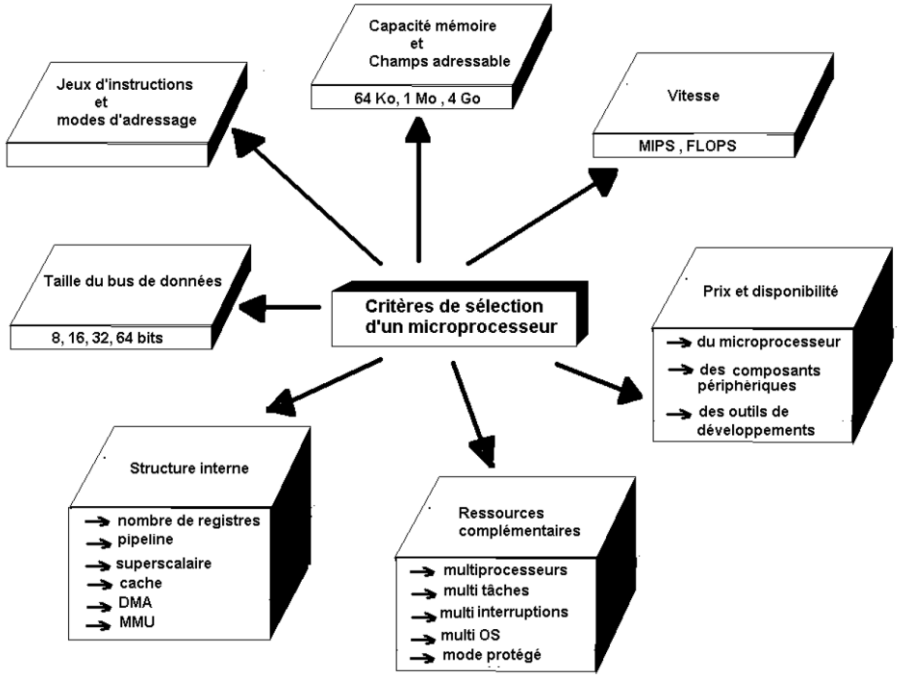


Fabrication

- Le processeur Apple A11 Bionic = 4 300 000 000 de transistors
  - 1 869 565 fois plus que le premier microprocesseur, l'Intel 4004.
  - S'il était toujours gravé à 10 µm, il couvrirait la surface d'un grand salon, mais puisqu'il est gravé à 10 nm, il mesure moins de 90 mm2.
  - => Plus c'est gravé fin…plus ça peut aller vite et consommer moins…mais…plus cela chauffe
    - => stagnation des fréquences

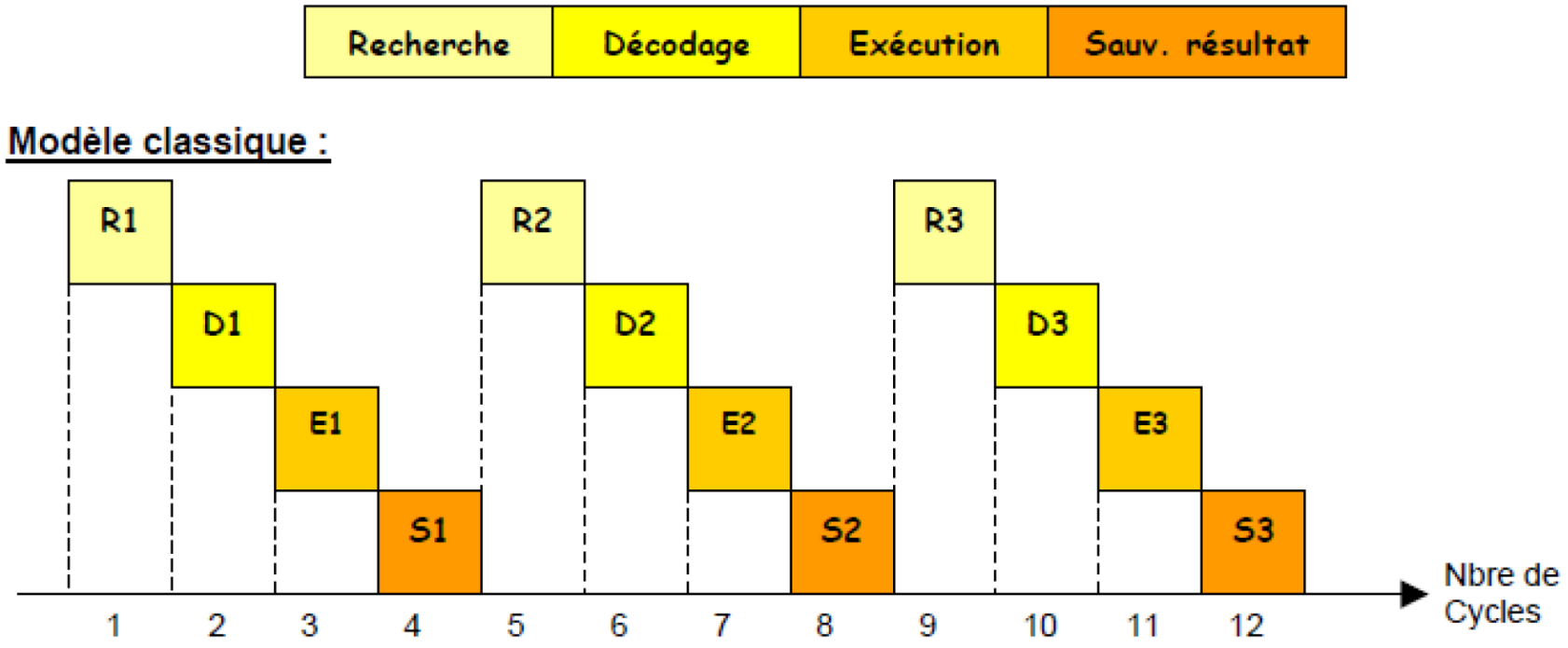
Choix d’un processeur



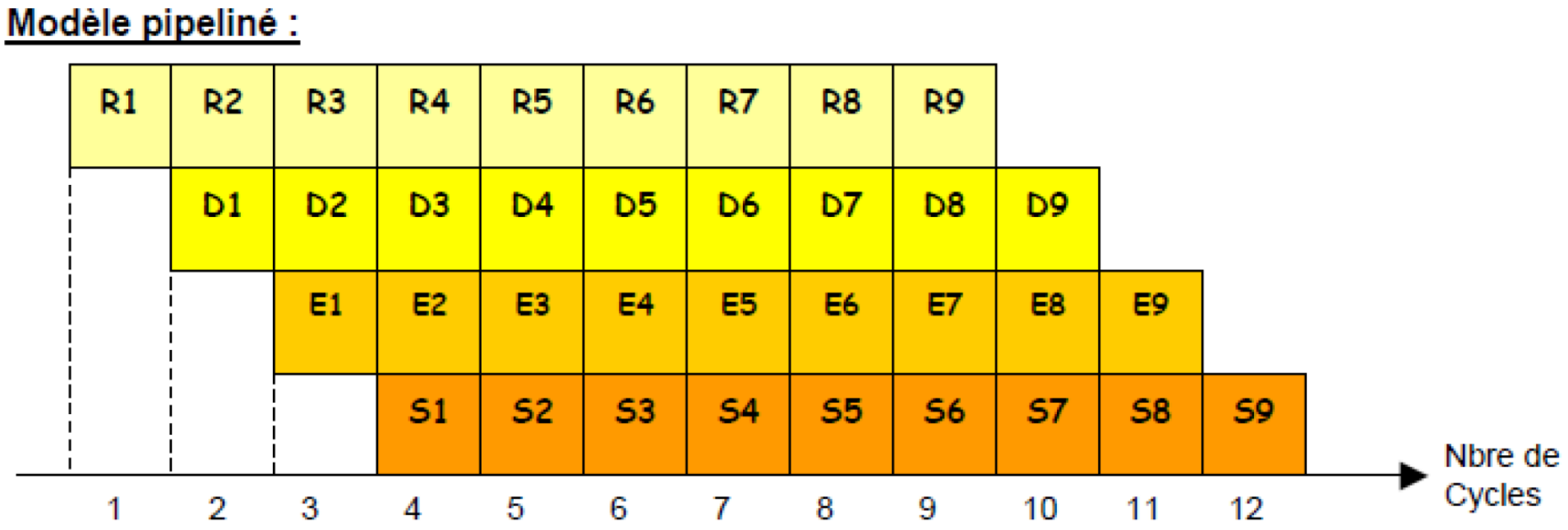


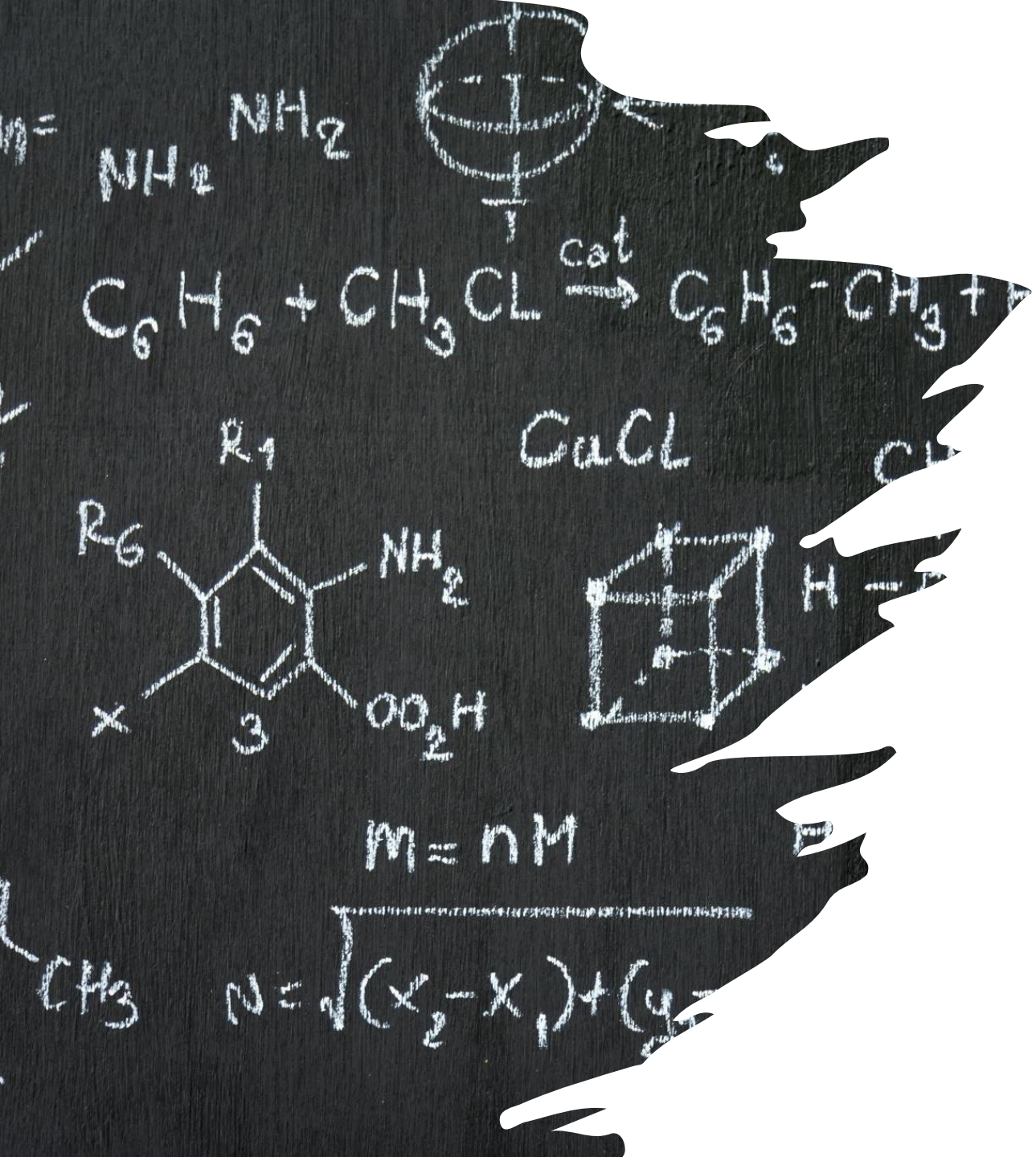
Fonctionnement d’un processeur

Principe du Pipeline



Principe du Pipeline





Pipe line (utilité)

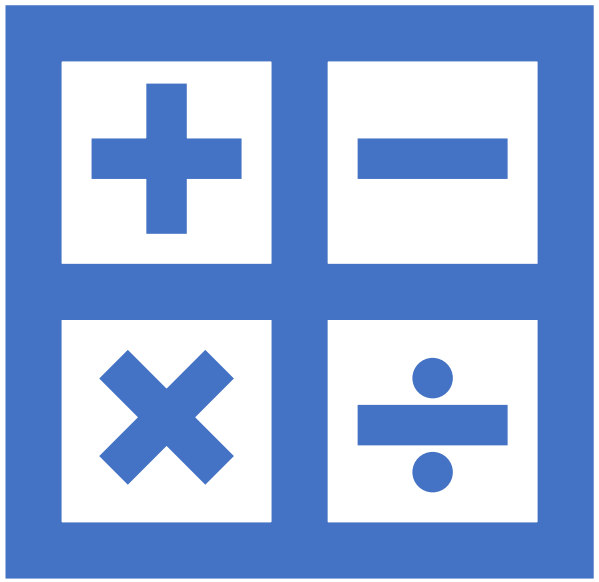
- Adapté aux calculs répétitifs (même opération sur de nombreuses informations)
- C’est le cas du calcul vectoriel ou matriciel
- Utile en traitement d’images, de vidéo, de son, …
- Mais ça ne résout pas le cas des calculs non répétitifs

Parallélisme

- Idée :
  - Mettre **plusieurs unités de calcul en parallèle**
  - On peut faire **plusieurs traitements en même temps si ces traitements sont indépendants** (l'un n'utilise pas le résultat de l'autre)
  - Actuellement on en met une dizaine
  - Mais pour simplifier toutes ne sont pas identiques :
    - Les opérations les plus utilisées sont disponibles sur toutes
    - Les opérations moins utilisées ne le sont que sur quelques unes

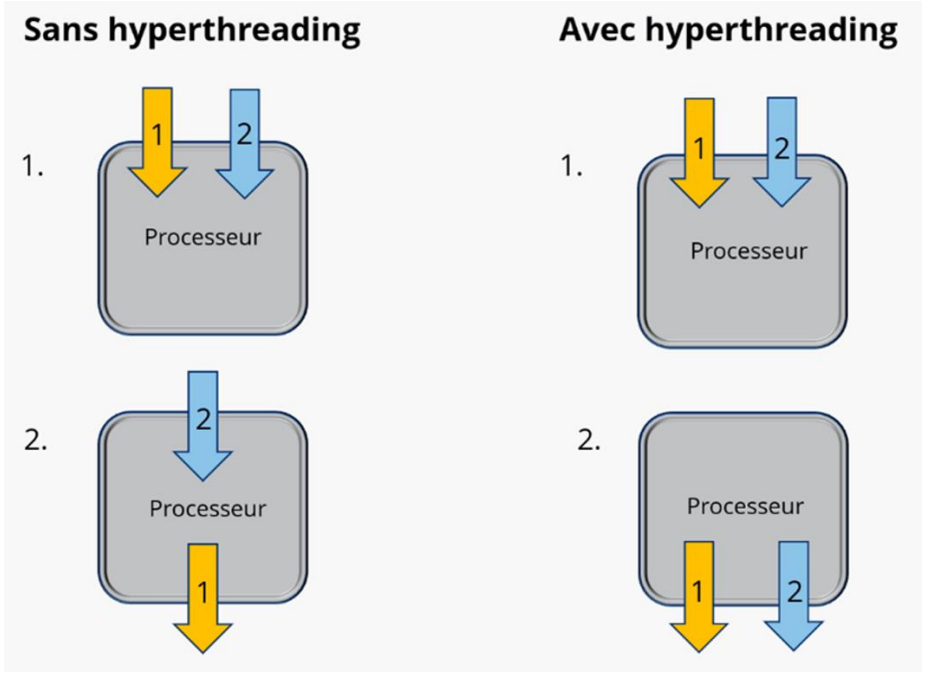
- Le groupe d'unités de calcul possède un distributeur qui reçoit les opérations à faire et les envoie à la première unité libre qui peut le faire

- Les résultats obtenus par une unité de calcul peuvent servir d'opérande à une autre (circulation interne des informations plus rapide)



Hyperthreading (HT)

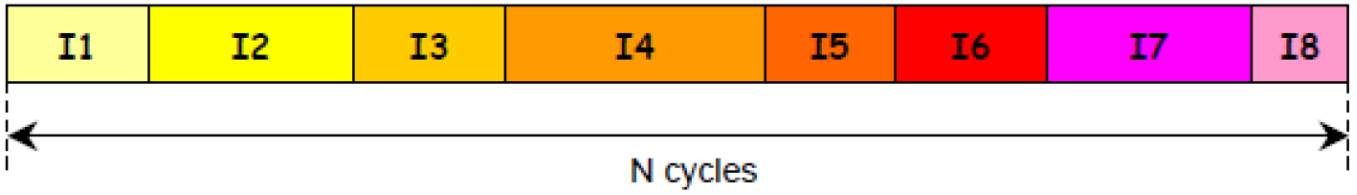
- L'Hyper Threading signifie SMT (Simultaneous Multi Threading) à deux voies.
  - Deux voies signifiant que chaque processeur recevra des instructions à traiter 2 par 2 (1 / thread).
  - Les deux threads partagent des données comme des registres, le cache le pipeline et le bus.
  - Système de contrôle afin de gérer les deads-locks pouvant survenir.
  - Il existe aussi du SMT à 4 voies, mais celui ci est plus généralement utilisé pour des machines plus importantes (super calculateur, serveur).
  - Permet d'accélérer le traitement des instructions et donc les performances des machines basées sur un Core i7. Il est possible de désactiver cette option gourmande en énergie dans le bios (+30 % d'énergie).
  - Gains allant de…rien à 20 %



Architecture de processeur superscalaire

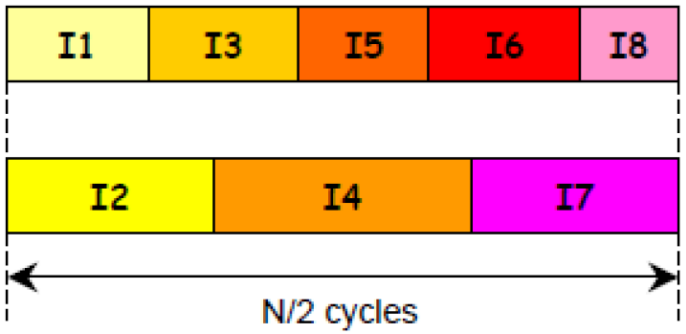
- Technologie consiste à faire travailler plusieurs unités d'exécution en parallèle dans le processeur

Architecture scalaire :



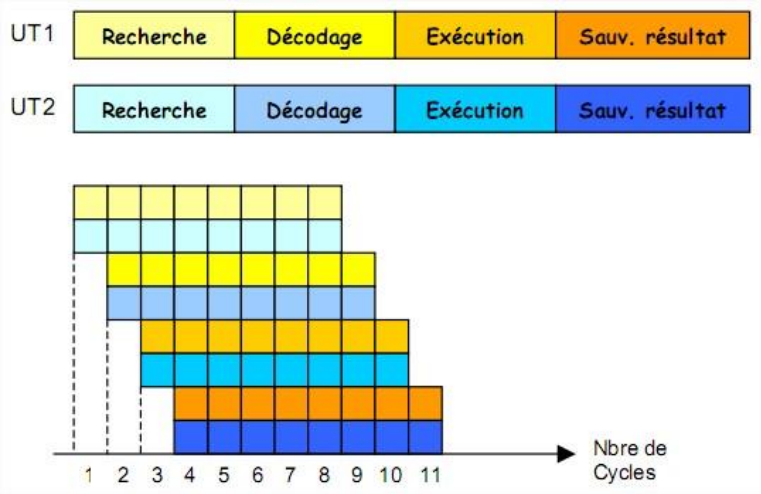
Unité de traitement

Architecture superscalaire :



Unité de traitement 1

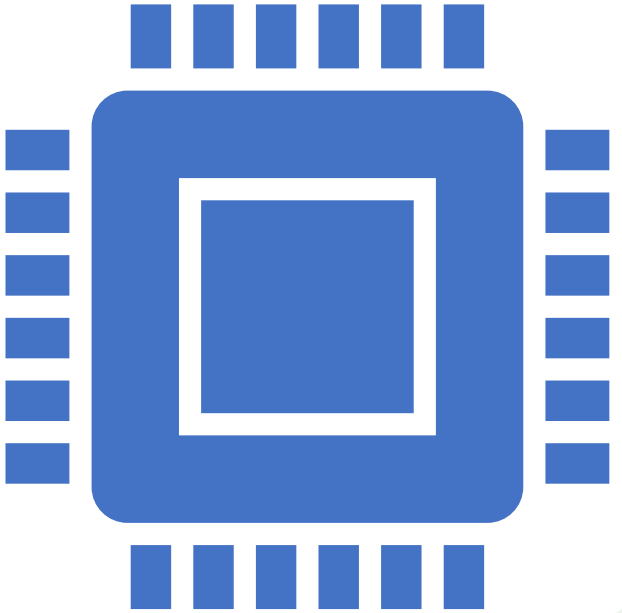
Unité de traitement 2



Superscalaire vs multicoeurs

- Les processeurs super-scalaires signifient
  - On envoie plusieurs instructions au cours d'un même cycle d'horloge.
  - **Toutes les instructions proviennent d'un seul programme.**

- Différent avec le multicœur
  - plusieurs flux d'instructions s'exécutent simultanément.
  - chaque cœur peut également être super-scalaire afin d'exécuter chaque processus plus rapidement!



Prédiction d'instructions

- Le dispositif de recherche d'instruction passe son temps à aller chercher des instructions dans le cache pour les mettre dans la file d'attente
- Quand il a mis une instruction dans le cache il passe à la suivante (adresse + 1)
- Mais s'il s'git d'une instruction de rupture de séquence. L'instruction suivante est peut être en ±N.
- Comment le savoir ?
- Si l' instruction de rupture de séquence n'a pas de condition c'est facile puisqu'il suffit de faire le ±N contenu dans l'instruction mais sinon ?

Prédiction d'instruction

- Première approche :
  - En général
    - dans un programme on passe plus souvent dans le si que dans le sinon parce que c'est comme ça que le programmeur raisonne
    - quand on a une boucle on boucle
  - Donc on peut considérer qu'il y a de fortes chances que :
    - Un saut en avant (+N) n'aura pas lieu (on ne saute pas le si pour aller au sinon)
    - Un saut en arrière (-N) aura lieu (on boucle)

- Le dispositif de prédiction peut faire comme ça en moyenne il aura souvent raison mais quand il a tort ?

Prédiction d'instructions

- Si on a rempli la file d'attente avec des instructions dont une au moins est une rupture de séquence conditionnelle on ne saura si on avait bien prédit que lorsque cette instruction sera exécutée.

- A ce moment là :
  - Si la condition évaluée correspond à la prédiction tout va bien
  - Sinon il faut enlever toutes les instructions en attente dans la file et re-remplir la file à partir de la bonne adresse (perte de temps)

- Plus la prédiction est bonne moins on perd de temps à vider et re-remplir la file.

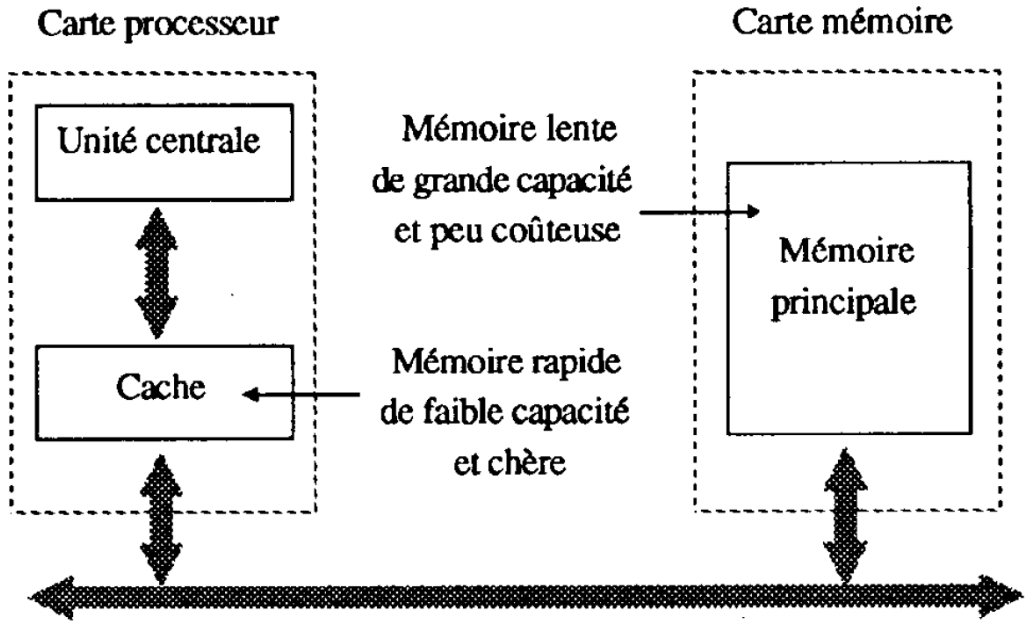
- On va donc cherche à améliorer la prédiction

Prédiction améliorée

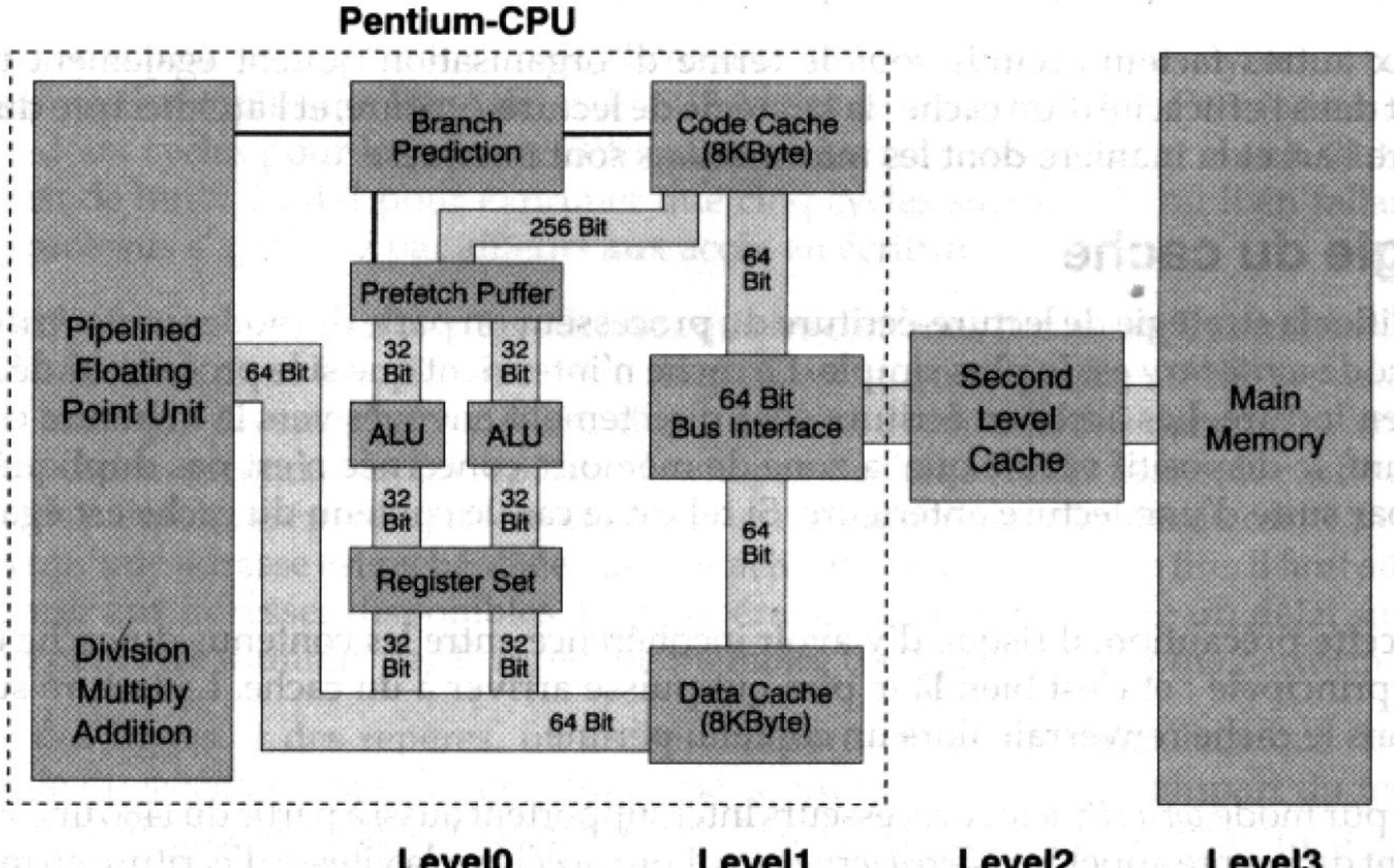
- Exemple :
  - On a une boucle contenant un si sinon mais il se trouve que l'on passe beaucoup plus souvent dans le sinon que dans le si
- Le dispositif de prédiction va se tromper presque à chaque tour de boucle puisqu'il va aller chercher les instructions du si
- Idée :
  - Mémoriser ce qui s'est réellement passé quand on a évalué la condition
- Au premier passage dans la boucle le dispositif se trompe mais pas au deuxième ni, la plupart du temps, aux suivants
- On va donc doter le dispositif d'une mémoire pour garder les résultats des instructions de rupture de séquence que l'on a déjà vues
- La première fois qu'il tombe sur une instruction de rupture de séquence il utilise la prédiction selon le signe de N. Mais quand l'instruction est exécutée il garde le résultat
- Les fois suivantes il utilise ce résultat plutôt que le signe de N et quand l'instruction est exécutée il garde le résultat
- Ce mécanisme diminue beaucoup les mauvaises prédictions pour un programme « normal »

Principe des caches

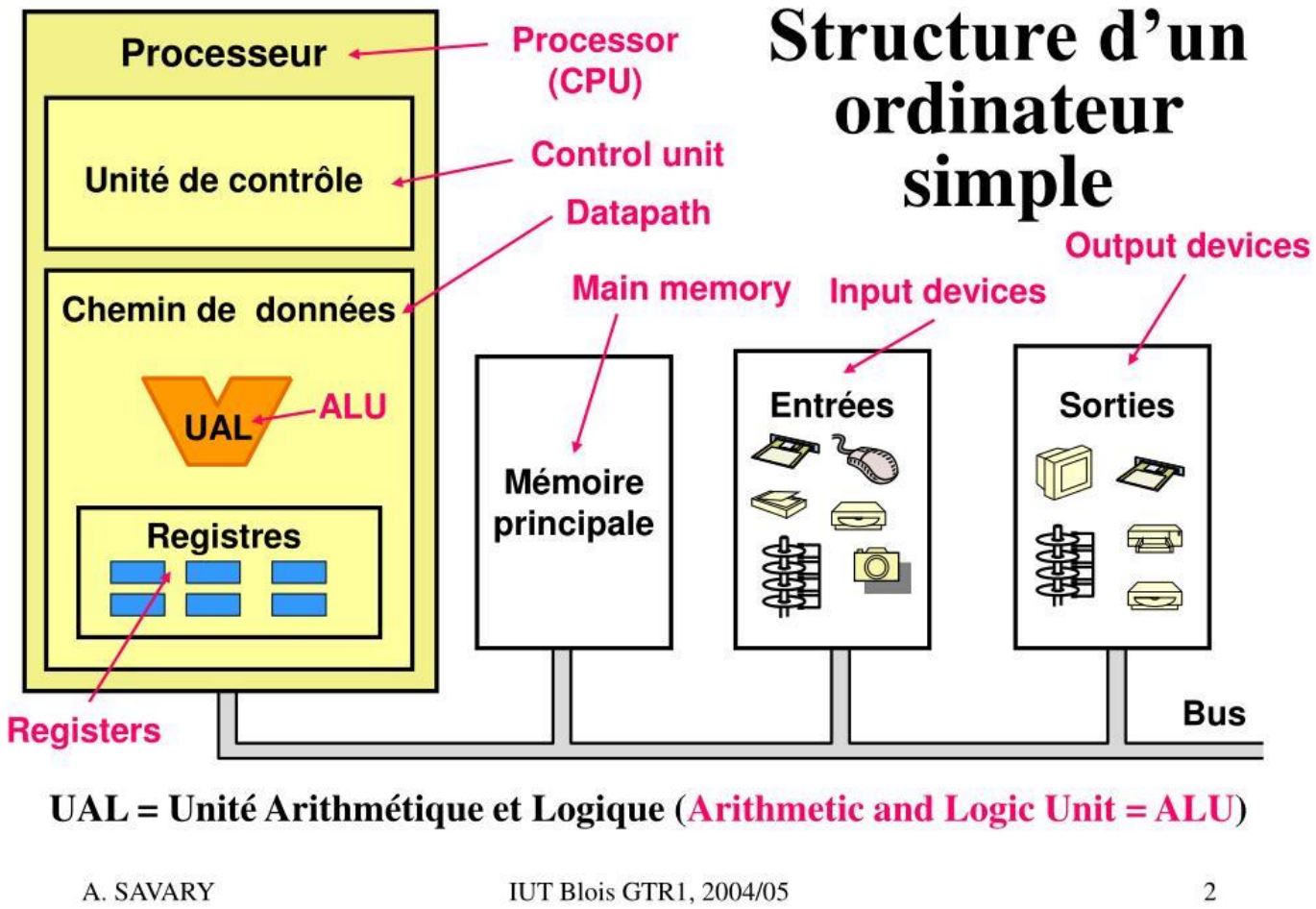
- La mémoire cache est une mémoire ultra rapide destinée à éviter les états d'attente (de 6 à 15 ns de temps d'accès contre 60 pour la mémoire vive).
- La mémoire cache (SRAM pour Static RAM) conserve les données qui y sont inscrites. Pas besoin de rafraîchissement, ce qui évite les états d'attente du processeur.
- Les données qui transitent entre la mémoire vive et le processeur sont stockées dans la mémoire cache qui fait office de tampon. Si le processeur a de nouveau besoin de ces données et cela arrive souvent qu'il fasse deux fois appel aux mêmes, il les obtiendra beaucoup plus rapidement.



Vue schématisée d'un processeur superscalaire (2 niveaux) + cache (3 niveaux)

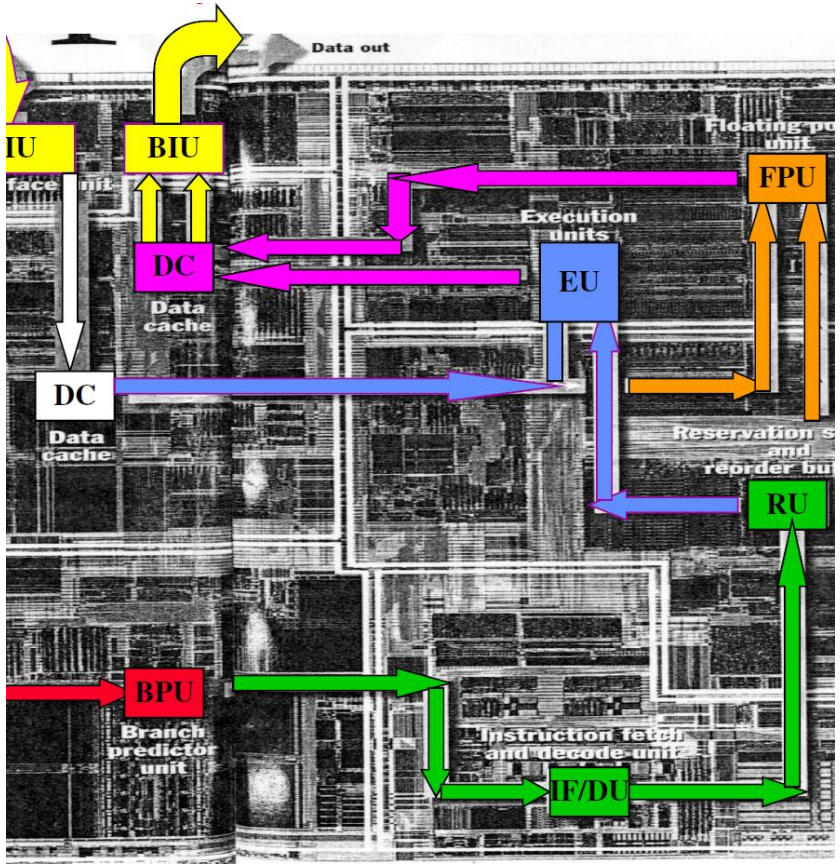


Vue Fonctionnelle



Fonctionnement d'une puce

- Les informations arrivent de la RAM du PC dans la puce par le bus (BIU Bus Interface Unit)
- L'information va dans un cache de code ou de donnée (CC,DC)
- L'unité de prédiction de branchement détermine le chemin optimum pour l'information (BPU)
- la station de réservation et buffer de réordonnancement détermine l'ordre d'exécution le plus efficace (RU)
- l'unité d'exécution exécute les opérations et rend les résultats dans le cache de donnée (EU)
- l'unité de virgule flottante (FPU) effectue les opérations arithmétiques
- le cache de donnée transfert ses résultats vers l'unité d'interface de bus qui les transmet à la RAM.



Fonctionnement d'une puce

- Améliorations continues
  - Prévision de branchement
  - Analyse du flot de données
  - Exécution spéculative
  - Amélioration du transfert processeur-mémoire
    - augmentation de la taille des bus
    - amélioration de l'interface des DRAM
    - utilisation des caches
    - augmentation de la bande passante des bus
  - Amélioration des entrées sorties

Fonctionnement d'une puce

Génération	Dates	Technologie	Opérations/s
1	1946-57	Tubes à vide	40k
2	1958-64	Transistors	200K
3	1965-71	SSI-MSI	1M
4	1972-77	LSI	10M
5	1978-	VLSI	100M

Cas pratique : fonctionnement du iCore i7

Pour différencier les processeurs, ils existent plein de criteres :

- le jeu d'instructions : addition, multiplication, max(nb1,nb2), etc.

Nombre de bits : 32 ou 64

Vitesse de l'horloge : nombre de cycles/ seconde

Nombre de cœurs

Nombre de transistors

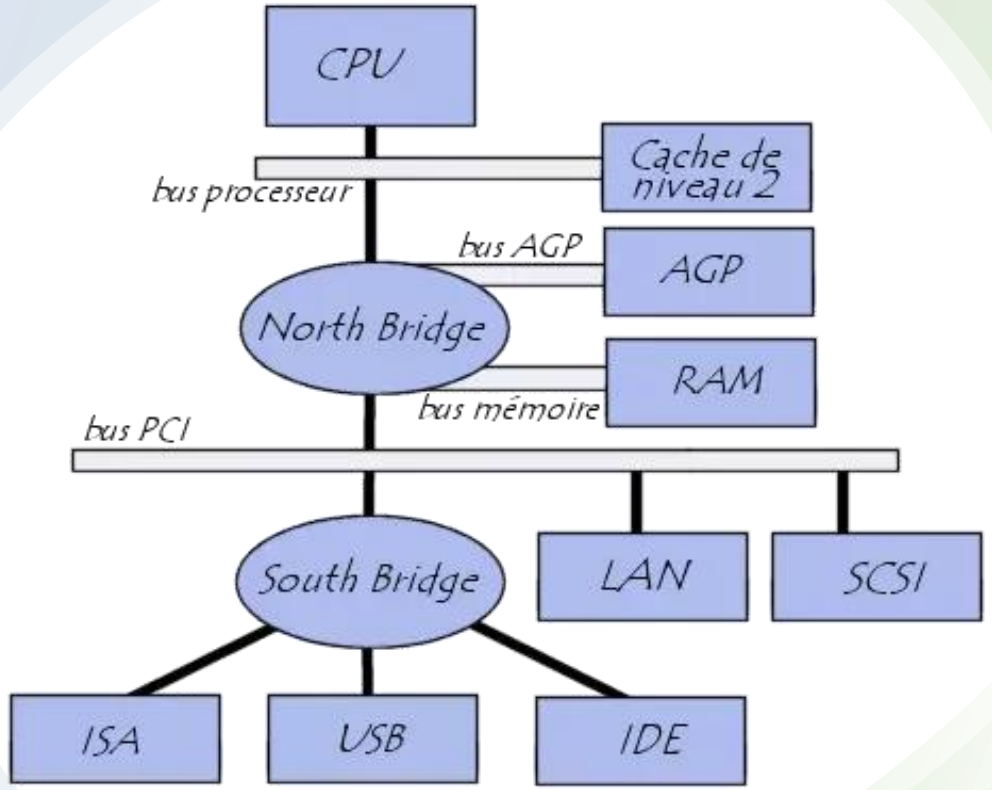
> 700 MIPS (millions d'instructions/secondes)

Quick Path Interconnect (QPI)

- **QPI = bus inventé par Intel permet d'interconnecter les coeurs entre eux ainsi qu'au chipset. Son but est de remplacer le FSB.**
- Actuellement, le proc. doit faire transiter toutes ces requêtes et données via le FSB pour communiquer avec les composants. Il existe un "aiguilleur" afin de fluidifier le trafic, le premier composant servant d'aiguilleur est le NorthBridge.

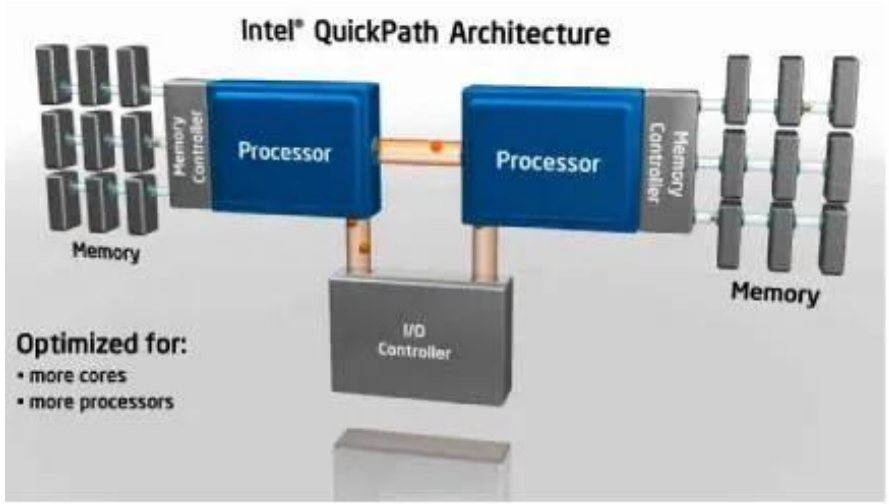
  Ce dernier doit faire le tri entre la mémoire, la carte graphique et le reste. Le reste est traité par le SouthBridge. Il s'occupe des autres composants. Cela va de la simple prise USB, au clavier en passant par l'*ethernet* et le *sata* par exemple.

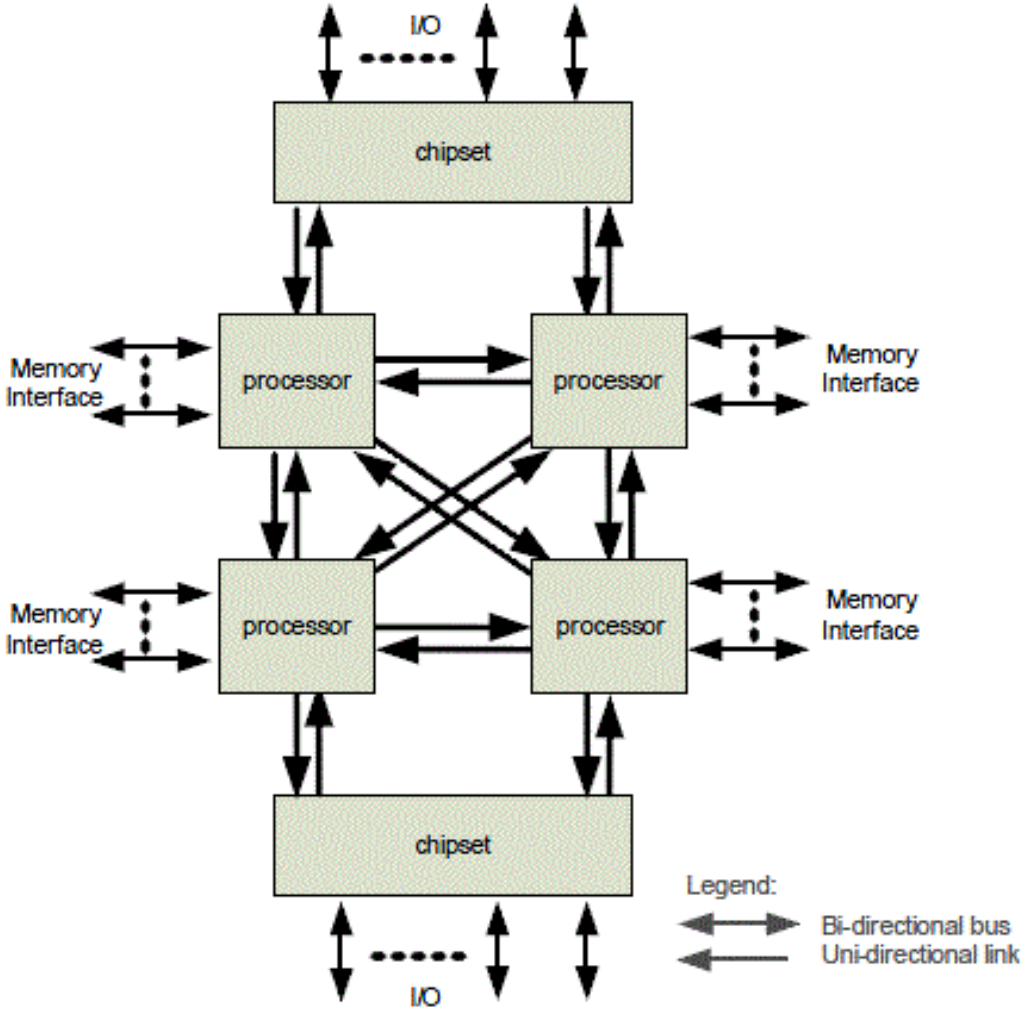
  Le FSB, pour Front Side Bus, doit supporter un trafic très dense. Son débit maximal est de 12.8 Go/s et ce pour le seul canal existant. Toutes les écritures et lectures en mémoire et sur les composants passent par ce FSB.



Quick Path Interconnect (QPI)

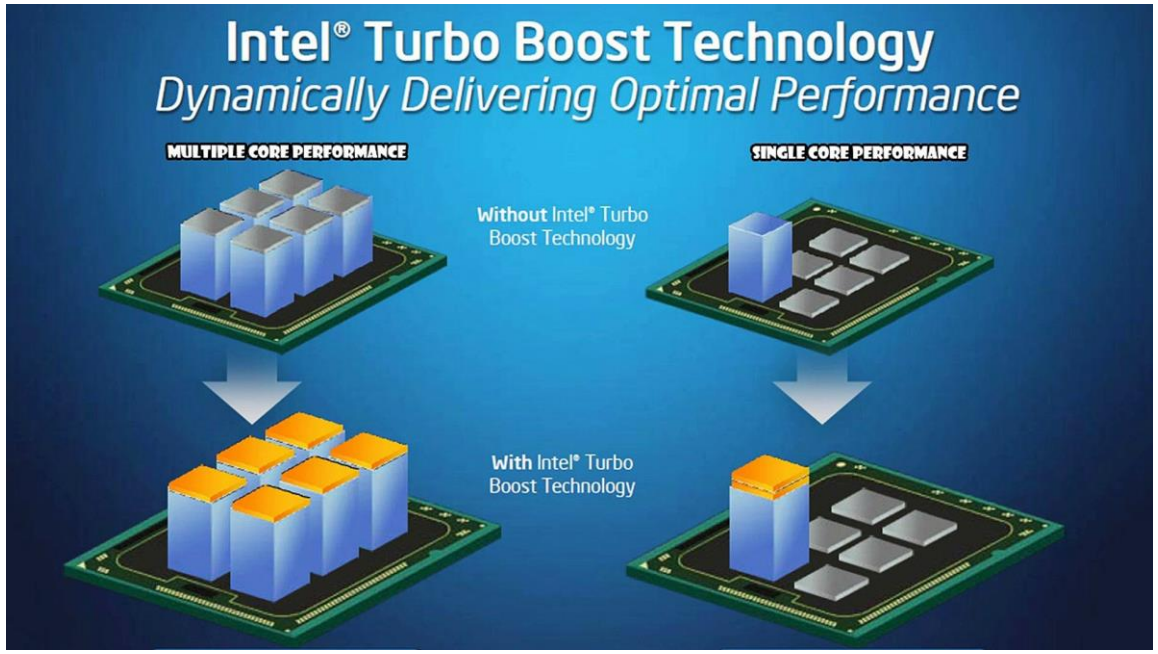
- C'est un bus est composé de deux voies.
- Il relie chaque cœur aux autres cœurs et au chipset. Le fait d'avoir deux voies autorise la lecture et l'écriture en même temps.





Turbo Boost

- Appelé aussi l'OverClocking automatique.
- Technologie qui permet de gérer l'utilisation des processeurs en fonction de la charge de travail à effectuer.
- Permet d'éviter de faire fonctionner plusieurs cœurs lorsqu'un seul suffit. Si jamais ce seul cœur n'est pas suffisant, le Turbo Boost peut augmenter la fréquence du cœur utilisé et ceci à la volée et de façon indépendante (les autres cœurs ne sont pas affectés par ces changements).



Evolution des architectures : du CISC au RISC

- CISC Complex Instruction Set Computer
  - jeu d'instruction complexe
- RISC Reduced Instruction Set Computer
  - jeu d'instruction réduit
  - multiplication des unités d'exécution
  - usage intensif des caches
  - branchements prédictifs ...

Caractéristiques des processeurs CISC

- Jeu d'instruction complexe => décodeur d'instructions
  - ++ programme d'application plus courts
  - ++ moins de place mémoire
  - -- temps d'exécution variables d'une instruction à une autre
  - -- la vitesse du processeur doit s'aligner sur les temps max

- Microcode
  - contient l'ensemble des micro-instructions
  - reçoit les instructions du décodeur et les transforme en opération élémentaires
  - surface de silicium augmentée par le microcode
  - problème des défauts de surface de silicium

Suppression du microcode avec le RISC

- **RISC développé par IBM à partir de 1975**
  - Concept freiné à ses débuts par le prix des mémoires (code + important)
  - Travaux parallèles sur les compilateurs
- Jeu d'instruction réduit => suppression du microcode
- Instructions complexes décomposées => rôle du compilateur
- Instructions complexes maintenues sont câblées
- temps d'exécution constant pour chaque instruction
- taille des instructions constante
- adressage simplifié
- multiplication des registres internes

Evaluation des performances

- Pour l'utilisateur : performance = temps d'exécution
- Difficile à évaluer car elle dépend de :
  - Vitesse d'exécution du microprocesseur
  - Temps de réponse de la mémoire
  - Entrée /sortie
- Programmes de tests Benchmarks
  - valables sur toutes les machines
  - problème de la représentativité des programmes

Evaluation de performances

- SPEC System Performance Evolution Coopérative
  - consortium de constructeurs
  - standards communs de test
  - exécution sur différentes plates-formes
  - => SPECMARK
  - De là viennent les unités de mesure
    - MIPS : Million Instructions Per Second
    - MFlops : Million of Floating Point Opération Per Second
- Problème:
  - le calcul des Mips dépend du constructeur
  - les jeux d'instructions étant différents => résultats non comparables entre eux.
- NB:
  - Ces mesures sont des indicateurs théoriques de performance
  - Choix d'une machine : benchmarks avec programmes types qu'on va utiliser.

Evaluation de performances

- **Systèmes temps-réel :**
  - systèmes critiques
  - dépassement de temps => DANGER
  - Exemple :
    - Détection d'alarme dans une centrale nucléaire
    - Détection d'un décrochage sur un A 310
- Pour ces systèmes
  - Evaluation doit être précise
  - benchmarks
  - analyse statique des programmes
  - simulation

Evolutions de l'architecture de Von Neumann

- **Objectif : se rapprocher le plus possible de la machine de Turing :**
  - Mémoire infinie
  - Vitesse de traitement infinie

- En réalité :
  - La mémoire peut être grande (des Giga octets voire des Téra octets sur les super ordinateurs)
  - Les temps de traitement et de lecture/écriture en mémoire peuvent être très courts (inférieurs à la ns)
  - Les temps de l'UE/S sont moins cruciaux puisqu'il suffit qu'ils soient compatibles avec celui de l'utilisateur (sauf pour les périphériques de stockage et les réseaux)

Contexte

$$\left(A_2\left(B_1C_2-B_2C_1\right)_1-A_3\left(B_3C_1-B_1C_3\right)_2\right)_1=\varepsilon_{1mi}A_n\left(\varepsilon_{ijk}B_jC_k\right)_i$$
$$=\varepsilon_{1m1}A_n\left(B_2C_3-B_3C_2\right)_1+\varepsilon_{1n2}A_n\left(B_3C_1-B_1C_3\right)_2+\varepsilon_{1n3}A_n\left(B_1C_2-B_2C_1\right)_3$$
$$=\varepsilon_{111}A_1\left(B_2C_3-B_3C_2\right)_1+\varepsilon_{121}A_2\left(B_2C_3-B_3C_2\right)_1+\varepsilon_{131}A_3\left(B_2C_3-B_3C_2\right)_1$$
$$+\varepsilon_{112}A_1\left(B_3C_1-B_1C_3\right)_2+\varepsilon_{122}A_2\left(B_3C_1-B_1C_3\right)_2+\varepsilon_{132}A_3\left(B_3C_1-B_1C_3\right)_2$$
$$+\varepsilon_{113}A_1\left(B_1C_2-B_2C_1\right)_3+\varepsilon_{123}A_2\left(B_1C_2-B_2C_1\right)_3+\varepsilon_{133}A_3\left(B_1C_2-B_2C_1\right)_3$$
$$=\varepsilon_{123}A_2\left(B_1C_2-B_2C_1\right)_3+\varepsilon_{132}A_3\left(B_3C_1-B_1C_3\right)_2$$
$$=1A_2\left(B_1C_2-B_2C_1\right)_3-1A_3\left(B_3C_1-B_1C_3\right)_2=A_2\left(B_1C_2-B_2C_1\right)-A_3\left(B_3C_1-B_1C_3\right)$$

- Certains programmes demandent de grandes quantités de calculs (simulations, météorologie, …)



- Certains programme demandent des temps de calcul courts (temps réel)
- Certains programmes cumulent les 2 exigences (éclairage de scènes dans les jeux 3D)
- Si on veut conserver l'architecture de Von Neumann on doit améliorer les 3 unités (Mémoire, UT et UC)
- C'est ce que l'on va faire maintenant



Unité de traitement

- On a vu qu'ajouter des opérations à l'UT ne servait pas à grand-chose (CISC)
- Il vaut mieux accélérer les opérations (RISC)

- Par exemple, faire une addition consiste à :
  - Faire la somme des 2 chiffres de droite
  - Faire la somme des 2 chiffres suivants en reportant la retenue
  - Continuer jusqu'au dernier chiffre à gauche

- On doit donc faire ces sommes élémentaires l'une après l'autre. On ne peut pas ajouter les centaines avant d'avoir ajouté les dizaines puisqu'on ne connaît pas la retenue éventuelle. Ça prend du temps



Accélérer l'UT

- On peut trouver quelques astuces de calcul pour aller plus vite (prédiction de retenue) mais c'est limité

- Solution : modifier l'architecture de l'UT :
  - Pipe line
  - Parallélisme

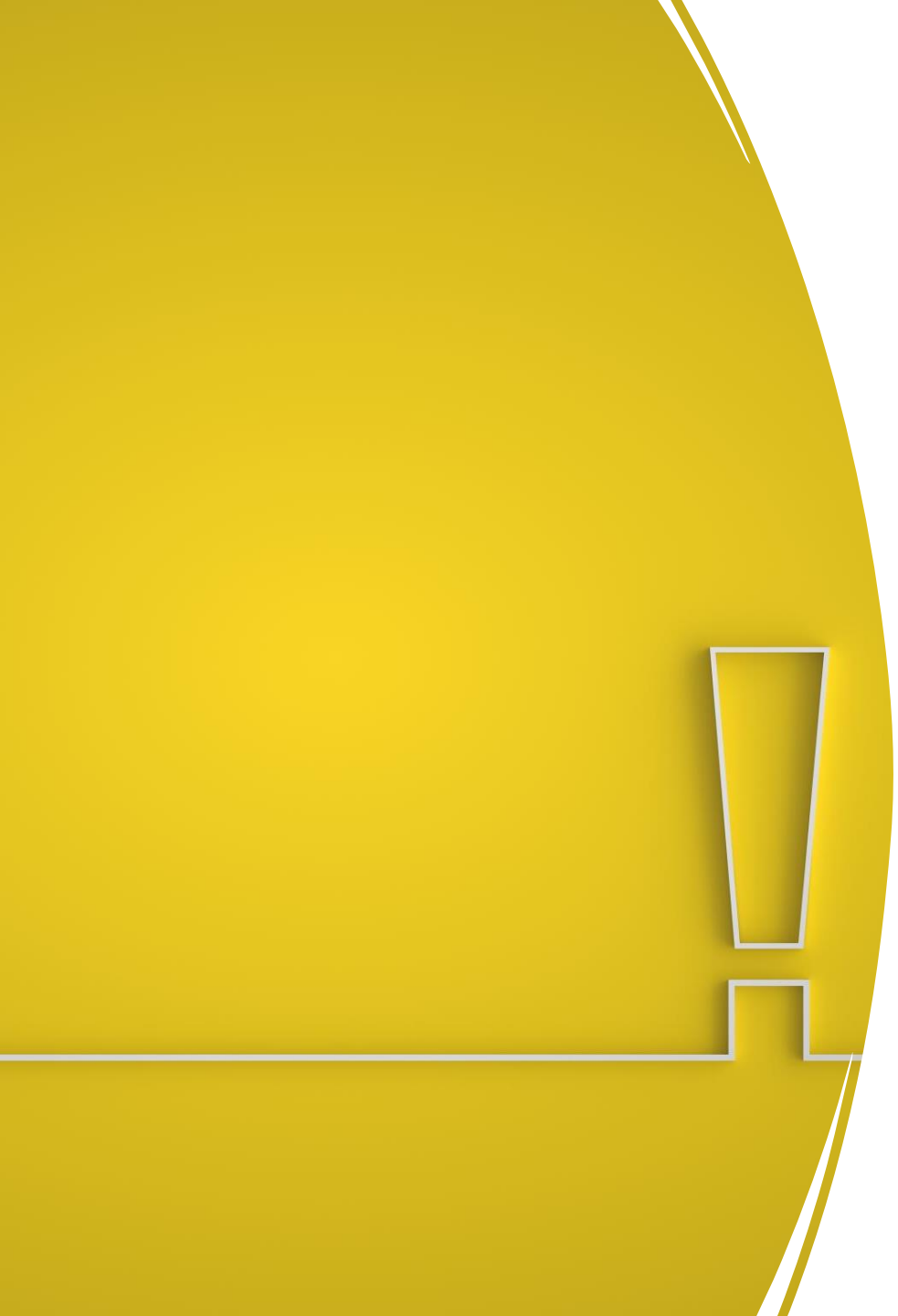


Résultat

- Tout ce qui pouvait être parallélisé l'a été
  - Pendant l'exécution d'une instruction
    - On prépare les suivantes
    - On prépare les opérandes
    - On range les résultats
  - Les traitements sont faits en parallèle dans l'UT et en pipe line dans les unités de calcul
  - L'exécution des instructions est faite en pipe line

- On arrive donc a avoir plusieurs instructions (jusqu'à 10) en cours d'exécution même temps (en recouvrement)
- On peut donc exécuter plus d'une instruction par cycle d'horloge

- Le nombre d'exécutions en parallèle dépend des instructions (liées)
- Il est impossible de savoir combien on fera d'instructions par seconde cela dépend de trop de choses (prédiction, remplissage des files, remplacement dans les caches, occupation des unités de calcul, …)



Peut-on faire mieux ?

- Mémoire
  - Plus de niveaux de caches : inutile (on ne gagne rien)
  - Caches plus grands, plus rapides : trop cher
  - Disques plus rapides : peu utile car le disque est 100 000 fois plus lent que la mémoire
- UT
  - Plus d'unités : on ne parvient pas à les occuper
- UC
  - Meilleure prédiction : plutôt un problème de compilateurs (optimiser le code généré pour éviter les mauvaises prédictions)
  - Plus d'instructions en parallèle : plutôt un problème de compilateurs (ordre des instructions, anticipation, …)



Peut-on faire mieux ?

- UC (séquenceur)
  - Accélérer l'horloge :
    - La puissance consommée croit comme le carré de la vitesse $\Rightarrow$ la température augmente
    - On peut accélérer l'horloge pendant un temps court et la ralentir si ça chauffe trop (turbo)
    - Les processeurs le permettent, le SE passe en mode turbo quand le processeur est très chargé mais revient en mode normal si la température monte trop ou si le processeur n'est plus chargé
- Alors on fait quoi ?



Multi cœurs

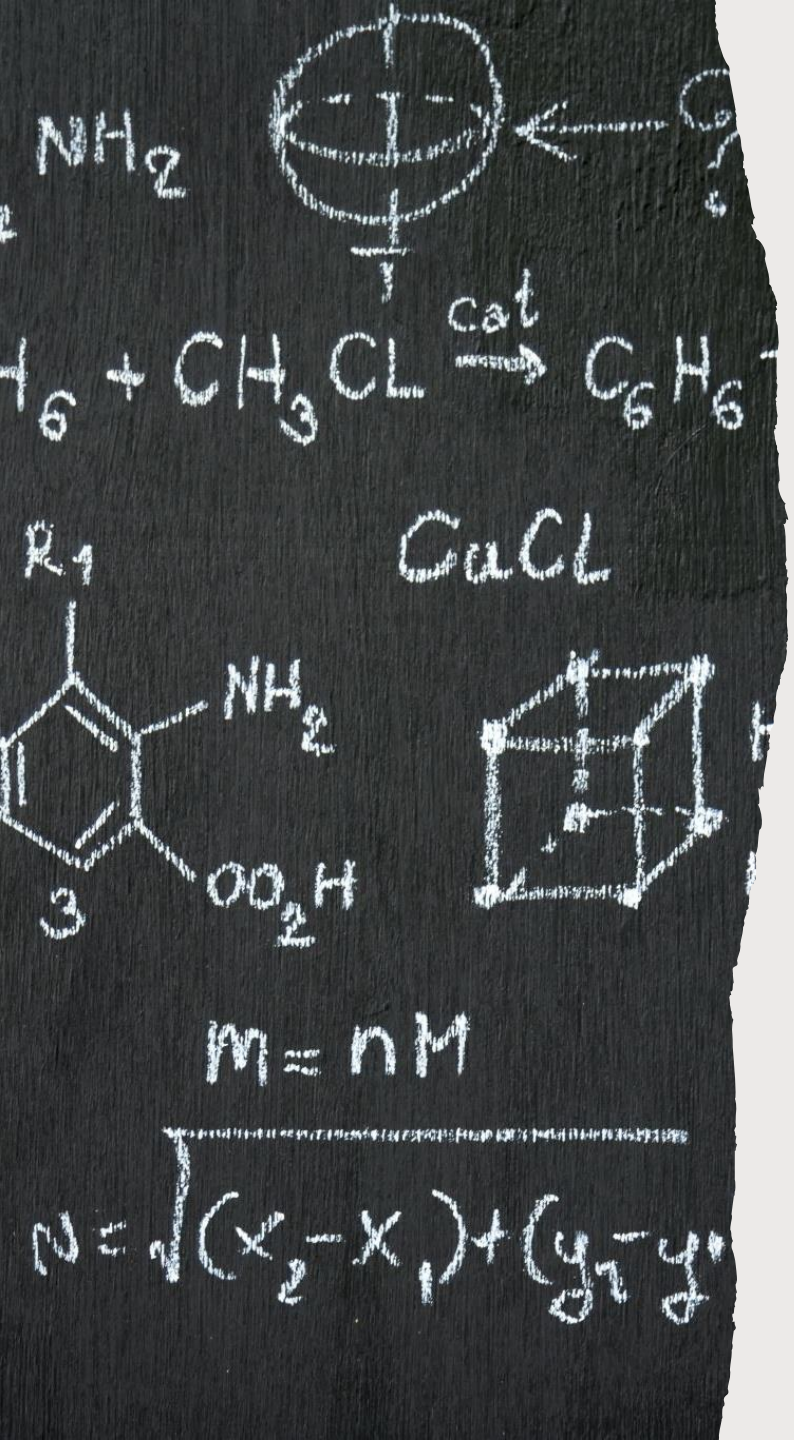
- La solution est de mettre plusieurs processeurs (UC+UT)
- Chacun exécute plus d'une instruction par cycle donc s'il y en a N on peut exécuter N fois plus d'instructions par cycle
- Principe : Mettre N cœurs c'est dire :
    - N UC + UT
    - N caches de niveau 1
    - N caches de niveau 2
  - En général un seul cache de niveau 3
  - Une seule mémoire
  - Et, bien sûr, une seule unité d'E/S

- Le parallélisme n'est plus dans l'exécution des instructions mais dans l'exécution des programmes (plusieurs programme en même temps)
- **C'est un problème de système d'exploitation** : utiliser tous les cœurs de façon optimale en répartissant les tâches entre eux au mieux



Multi cœurs et multi threading

- Les processeurs actuels sont multi cœurs
- Chaque cœur est lui-même multi thread donc peut se faire passer pour 2 cœurs
- Combien de cœurs physiques peut-on mettre ?
  - C'est un problème d'intégration mais surtout de consommation électrique donc de température
  - On trouve des processeurs à 4 cœurs mais aussi à 64 cœurs ou plus
- Le problème de température est une limite forte de l'intégration
- S'il faut un système de refroidissement plus cher que l'ordinateur est-ce que c'est viable ?
- Le problème n'est pas le même sur les serveurs qui sont mis dans des salles climatisées
- Du point de vue écologique les serveurs consomment plus en climatisation que par leur fonctionnement propre



Super ordinateurs

- Disposer d'une puissance de calcul élevée pour des problèmes spécifiques (simulation, météorologie, médical, …)

- Un seul processeur même multicœur ne suffit pas
- Il faut en mettre plusieurs qui travaillent ensemble
  - Partagent une mémoire commune (peu utilisé car limité)
  - Communiquent par des réseaux ultra rapides

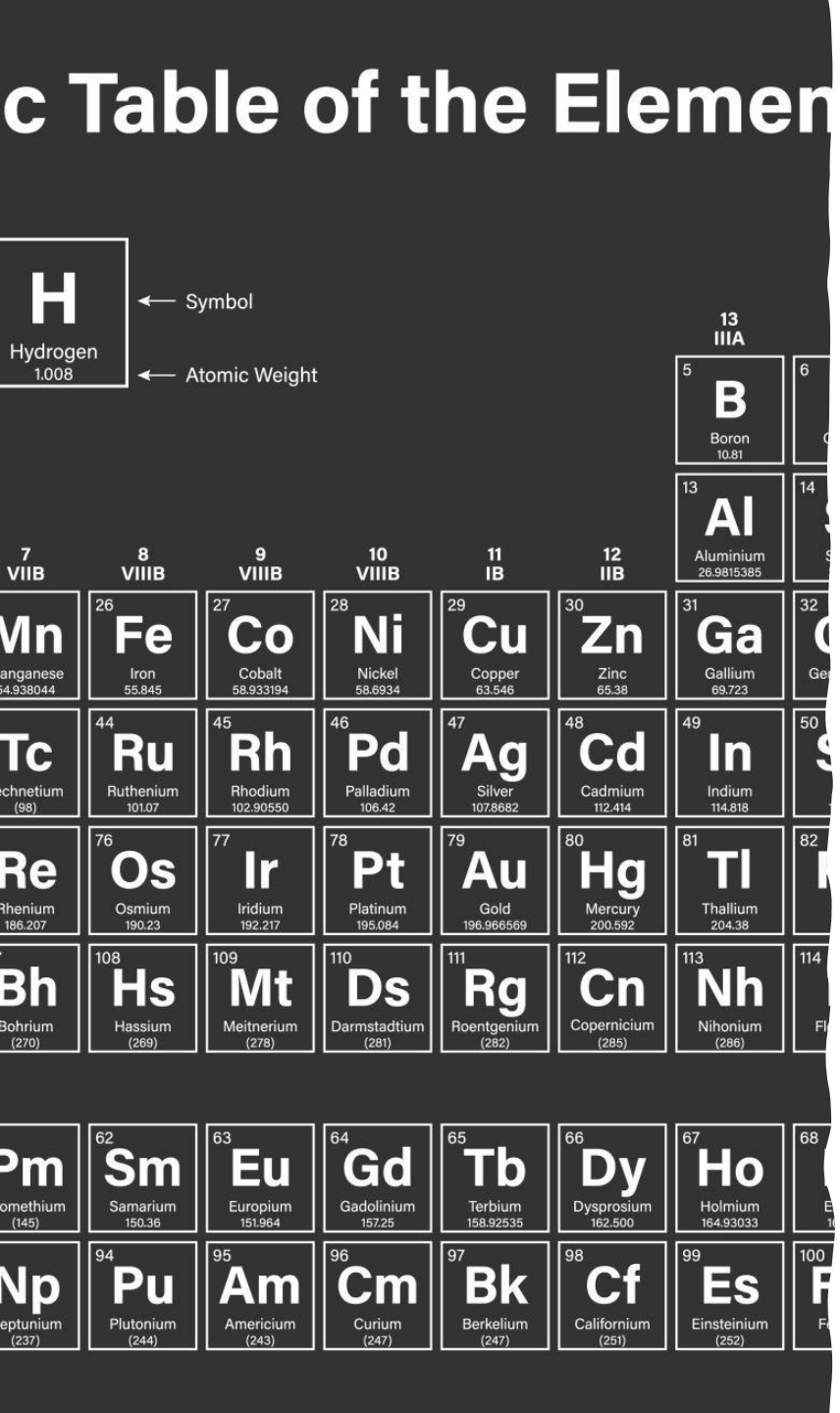
- Ces machines sont peu nombreuses et limitées à des usages très spécifiques (militaire, industrie, recherche)



Super ordinateurs

- Puissance de calcul :
  - Ces machines sont utilisées pour faire du calcul intensif
  - On ne mesure pas le nombre d'instructions par seconde mais le nombre d'opérations sur des nombres réels par seconde
  - **L'unité est le TFlop/s : c'est à dire 1000 milliards d'opérations sur des nombres réels par seconde**
  - Ces mesures sont faites par des programmes spécifiques qui utilisent simultanément tous les cœurs de la machine

- Architecture :
  - Les superordinateurs sont constitués de cartes sur lesquelles se trouvent un ou plusieurs processeurs multicœurs et de la mémoire
  - La mémoire totale disponible est donc la somme de celle présente sur chaque carte
  - Ces cartes sont reliées par un réseau local ultra rapide

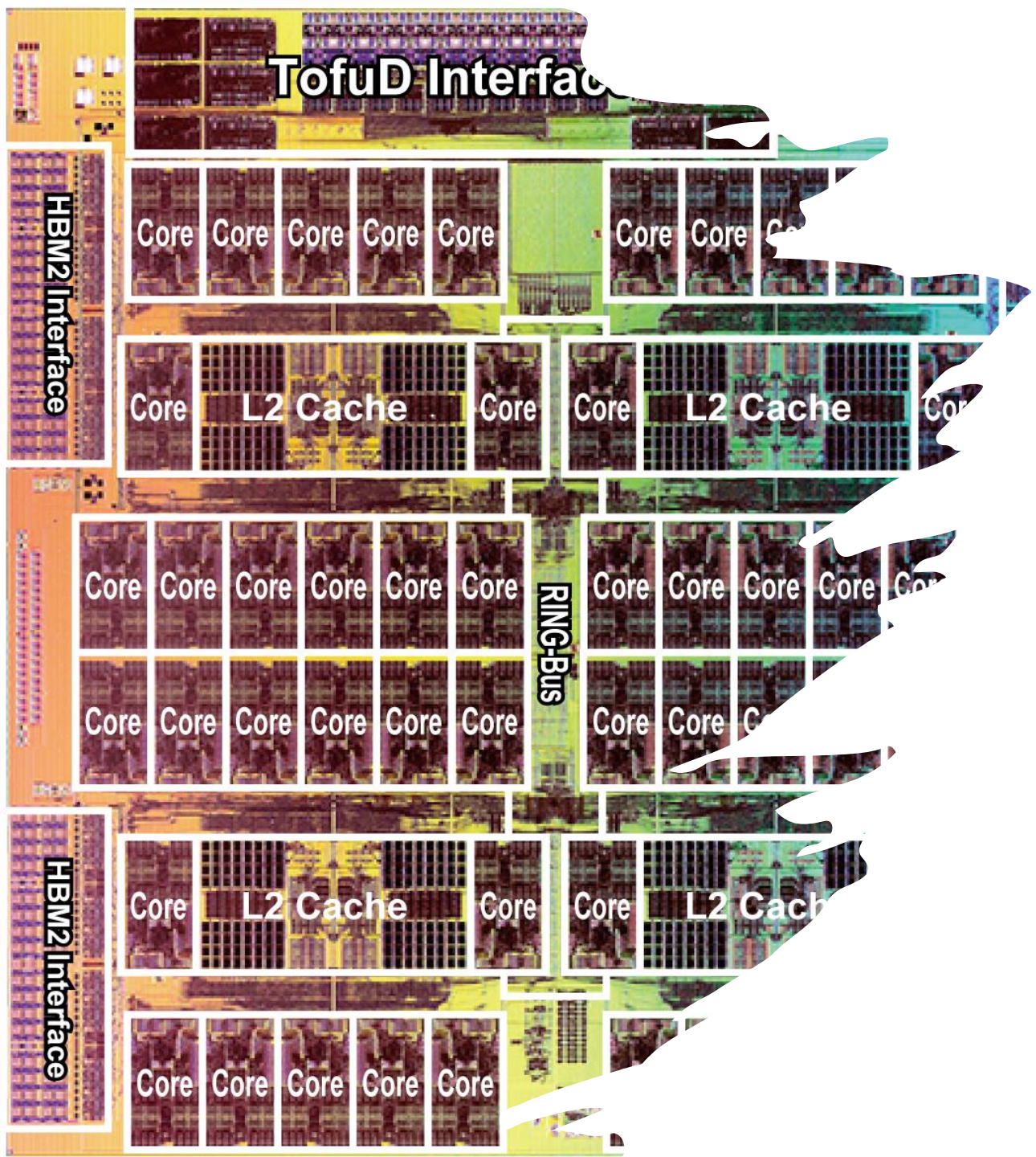


Et le gagnant est :

L'ordinateur les plus puissant au monde

- Constructeur : Fujitsu
- Nom : Fugaku
- Lieu : RIKEN Center for Computational Science, Kobe (Japon)
- Nombre de cœurs : 7 299 072
- 152 064 processeurs : A64FX 48 cœurs à 2 GHz (turbo à 2,2 GHz)
- Mémoire : 4 866 048 Go (32 Go par processeur)
- Interconnexion : Tofu interconnect D (28 Gb/s)

- Performance :
  - Vitesse théorique max : 513 855 TFlop/s
  - Vitesse mesurée : 415 530 TFlop/s
- Système :
  - Red Hat Enterprise Linux 8



Processeur du Fugaku

- Processeur Armv8.2-A SVE (64 bits mais calcul vectoriel sur 128 bits)
- 48 cœurs de calcul + 2 pour le SE
- Cache de niveau 1 : 2*64Ko (256 Go/s en lecture, 128 Go/s en écriture)
- Cache de niveau 2 : 8 Mo (128 Go/s en lecture, 64 Go/s en écriture)
- Pas de cache de niveau 3 car mémoire RAM rapide



Le Fugaku en image

- 152 064 processeurs Armv8.2-A placés dans 396 armoires en contenant 384 chacune
- Consommation = 28 334 KW (un TGV au démarrage : 8 800 KW)

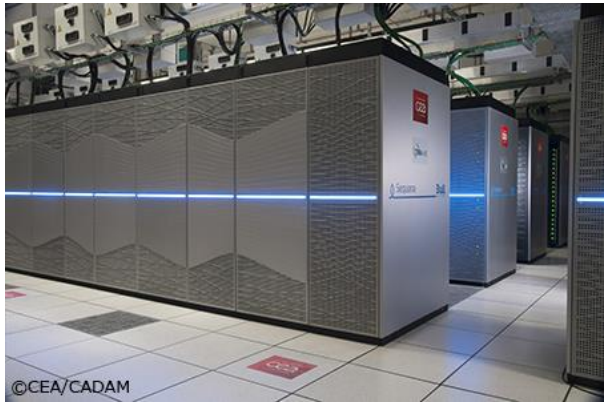


Et en France ?

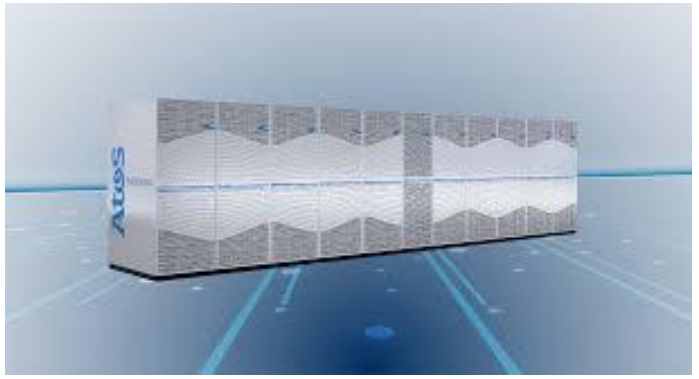
L'ordinateur les plus puissant en France

- Constructeur : Cray – Base Modèle HPE Cray EX
- Nom : Adastra - *Per aspera ad astra « Par des chemins ardus vers les étoiles »*
- Lieu : Centre informatique national de l'enseignement supérieur (Cines), à Montpellier (Hérault).
- Objectif: Modélisation environnementale, médicale, et même astrophysique
- Nombre de cœurs : 319 072 cœurs
- Processeurs : CPU AMD EPYC
- Puissance: 74 TFLops (millions de milliards d'opérations par seconde)
  - Equivalent de deux millions de smartphones haut de gamme ou 200 000 consoles de jeu dernier cri
- Mémoire : 768 Go DDR5 (partition 1), 256 Go (partition 2) en DDR4 avec 4 accélérateurs de calcul AMD Instinct™ MI250X OAM, chacun avec 128 Go de mémoire HBM2e pour un total de 512 Go de mémoire rapide
- 338 nœuds GPU comprenant chacun 4 cartes graphiques = 1 352 cartes graphiques interconnectées via un réseau très performant à 200 Gb/s (10 000 fois plus rapide qu'une box ADSL).
- Consommation: 97% de la chaleur générée par la machine est refroidit via un refroidissement liquide à eau chaude associé à PUE de 1,10 amenant à une consommation soutenue maximale à 1,59 MW.
- 11ème place mondile du TOP 500, 3ème meilleure rapport puissance/conso.
- Dimension de 8 mètres sur 2 pour un poids de 2 tonnes
- Coût: 30 M €

Les autres en France



- Tera-1000-2
  - Commissariat à l'Energie Atomique
  - 294 916 cœurs : 4 337 Intel Xeon Phi 7250 68 cœurs à 1.4GHz
  - 589 824 Go de mémoire
  - Mesuré à 11 965 TFlop/s

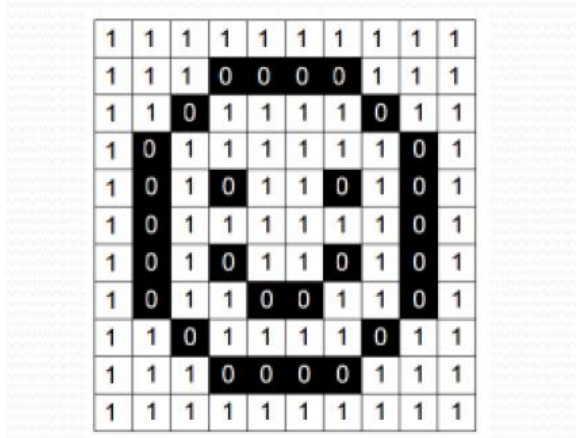


- Belenos
  - Météo France
  - 561 408 cœurs : 8 772 AMD EPYC 7742 64 cœurs à 2.25GHz
  - 1 585 152 Go de mémoire
  - Mesuré à 7 683 TFlop/s

Codages Images/Vidéos

Codage des images

- Formats matriciels(composés de pixels) (abusivement appelé Bitmap, propriétaire)
- Image de 1024x768
  - Monochrome (noir et blanc) :) : 786432 bits (98 Ko)
  - Niveau de gris (8 bits) - 770 Ko
  - Couleurs indexées (8 bits) - 770 Kio
  - Couleurs vraies (24 bits) - 2,25 Mio
- Formats vectoriels:
  - Formes géométriques (équation mathématiques)





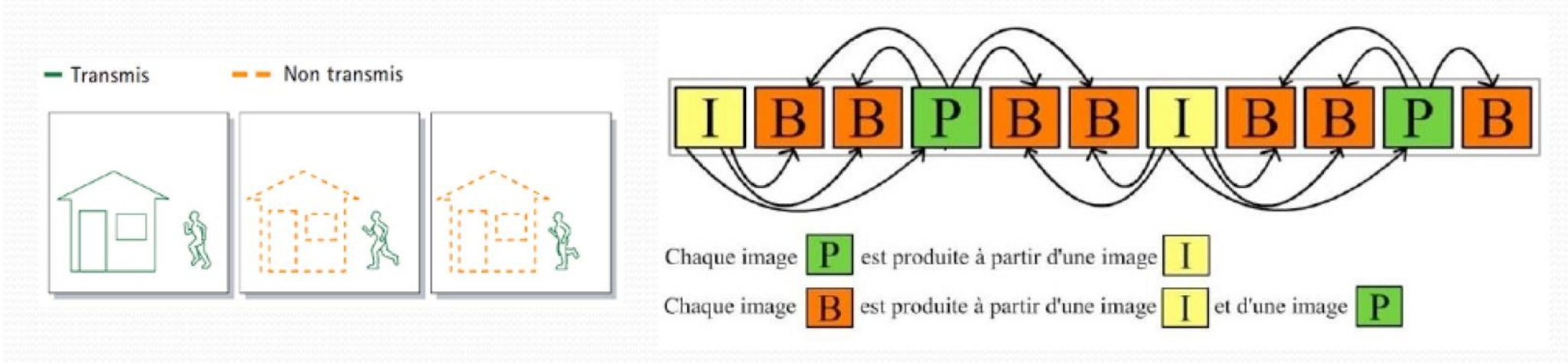
Techniques de compression

- Sans perte (exemple RLE) :SSYYYYYYYYYPHHHHHHHERRRRRRRRRR⇒SS@9YP@7HE@10R
  - Documents, exécutables, etc.

- Avec perte (exemple JPEG) : Image décomposée en carrés de 8 x 8 pixels
  - Suppression des fréquences auxquelles l'oeil (ou l'oreille) est peu sensible
  - Suppression des redondances spatiales

Codages des vidéos

- Nombre d'images par seconde (24, 25, 50, 60, 120, etc.)
- Sans compression : 22500 images de 2,25 Mo= 50,625 Go
- M-JPEG : JPEG sur chaque image de la vidéo
- Idem avec suppression de redondances temporelles :
  - MPEG-1, MPEG-2, MPEG-4



Identification des objets

- Code barre
- Code QR
- RFID
  - Ondes radio
  - Visibilité inutile

